Progressive Education Society's
Modern College of Engineering, Pune-05.

DEPARTMENT OF COMPUTER ENGINEERING

COMPUTER NETWORK AND SECURITY

LAB MANUAL
2025-2026

MS. POOJA HANDE

(SUB-INCHARGE)

DR. MRS. S.A. ITKAR
(HOD)

SR.NO	TITLE
	GROUP A
1.	Setup a wired LAN using Layer 2 Switch. It includes preparation of cable, testing of cable using line tester, configuration machine using IP addresses, testing using PING utility and demonstrating the PING packets captured traces using Wireshark Packet Analyzer Tool.
2.	Demonstrate the different types of topologies and types of transmission media by using a packet tracer tool.
3.	Setup a WAN which contains wired as well as wireless LAN by using a packet tracer tool. Demonstrate transfer of a packet from LAN 1 (wired LAN) to LAN2 (Wireless LAN)
4.	Write a program for error detection and correction for 7/8 bits ASCII codes using Hamming Codes or CRC.
5.	Write a program to simulate Go back N and Selective Repeat Modes of Sliding Window Protocol in Peer-to-Peer mode.
	GROUP B
6.	Write a program to demonstrate Sub-netting and find subnet masks.
7.	Write a program to implement link state /Distance vector routing protocol to find suitable path for transmission.
8.	Use packet Tracer tool for configuration of 3 router network using one of the following protocol RIP/OSPF/BGP
9.	Write a program using TCP socket for wired network for following a. Say Hello to Each other b. File transfer c. Calculator
10.	Write a program using UDP Sockets to enable file transfer (Script, Text, Audio and Video one file each) between two machines.
	GROUP C
11.	Write a program for DNS lookup. Given an IP address as input, it should return URL and vice-versa.

12.	Installing and configure DHCP server and write a program to install the software on remote machine
13.	Capture packets using Wireshark, write the exact packet capture filter expressions to accomplish the following and save the output in file: 1. Capture all TCP traffic to/from Facebook, during the time when you log in to your Facebook account 2. Capture all HTTP traffic to/from Facebook, when you log in to your Facebook account 3. Write a DISPLAY filter expression to count all TCP packets (captured under item #1) that have the flags SYN, PSH, and RST set. Show the fraction of packets that had each flag set. 4. Count how many TCP packets you received from / sent to Face book, and how many of each were also HTTP packets
14.	Study and Analyze the performance of HTTP, HTTPS and FTP protocol using Packet tracer tool.
15.	To study the SSL protocol by capturing the packets using Wireshark tool while visiting any SSL secured website (banking, e-commerce etc.).
16.	Illustrate the steps for implementation of S/MIME email security through Microsoft® Office Outlook.
17.	To study the IPsec (ESP and AH) protocol by capturing the packets using Wireshark tool.

ASSIGNMENT NO- 1

Title: Setup a wired LAN using Layer 2 Switch. It includes preparation of cable, testing of cable using line tester, configuration machine using IP addresses, testing using PING utility and demonstrating the PING packets captured traces using Wireshark Packet Analyzer Tool.

Objectives: To establish a wired LAN for four computers.

Problem Statement:

Setup a wired LAN using Layer 2 Switch and then IP switch of minimum four computers. It includes preparation of cable, testing of cable using line tester, configuration machine using IP addresses, testing using PING utility and demonstrate the PING packets captured traces using Wireshark Packet Analyzer Tool.

Outcomes:

Develop and demonstrate a wired LAN for four computers.

Tools Required:

Hardware: Computer, LAN Cards, RJ-45 Connectors, Switch, CAT-6 Cable, Cable tester, Crimping tool, Line tester etc.

Software: Open source OS and Wireshark

Theory:

Introduction: -

Computer Networks, the widespread sharing of information among groups of computers and their users, a central part of the information age. The popular adoption of the personal computer (PC) and the local area network (LAN) during the 1980s has led to the capacity to access information on a distant database; download an application from overseas; send a message to a friend in a different country; and share files with a colleague—all from a personal computer.

The networks that allow all this to be done so easily are sophisticated and complex entities. They rely for their effectiveness on many cooperating components. The design and deployment of the worldwide computer network can be viewed as one of the great technological wonders of recent decades.

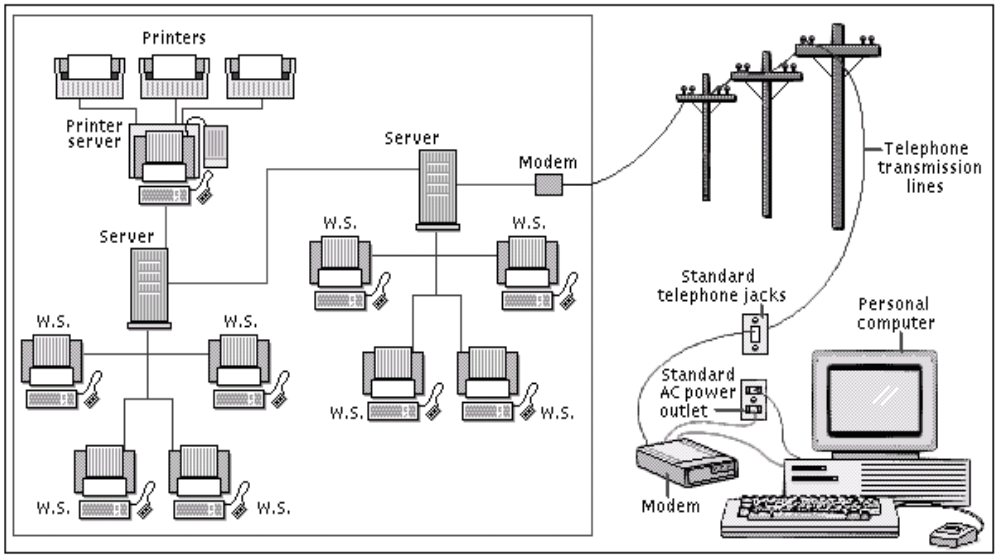


Fig-Computer Network

Networks are connections between groups of computers and associated devices that allow users to transfer information electronically. The local area network shown on the left is representative of the setup used in many offices and companies. Individual computers, called work stations (WS), communicate to each other via cable or telephone line linking to servers. Servers are computers exactly like the WS, except that they have an administrative function and are devoted entirely to monitoring and controlling WS access to part or all of the network and to any shared resources (such as printers). The red line represents the larger network connection between servers, called the backbone; the blue line shows local connections. A modem (modulator/demodulator) allows computers to transfer information across standard telephone lines. Modems convert digital signals into analogue signals and back again, making it possible for computers to communicate, or network, across thousands of miles.

Network Devices :-

1]NIC (**Network Interface Card)** :- A network interface card (NIC) is a hardware component without which a computer cannot be connected over a network. It is a circuit board installed in a computer that provides a dedicated network connection to the computer. It is also called network interface controller, network adapter, or LAN adapter. NIC allows both wired and wireless communications.

NIC allows communications between computers connected via local area network (LAN) as well as communications over large-scale network through Internet Protocol (IP).
NIC is both a physical layer and a data link layer device, i.e. it provides the necessary hardware circuitry so that the physical layer processes and some data link layer processes can run on it.



Fig-NIC

2]Hub-

A hub is a hardware device in networking that is used to connect multiple computers or connections in a network.

- In other words, hub is a multiport repeater.
- It broadcasts the information to all the computers connected to it, i.e., it automatically sends all the information to all the connected devices via ports.
- It usually used to connect computers in a LAN.
- A hub cannot filter data.
- Hub works on the half-duplex transmission mode, i.e., it cannot receive and send information at the same time.
- Hub does not provide any security



Fig-Hub

3]Switch -

Switches are networking devices that allow us to let things connect locally in a building.

For example, if there is a computer in a building that wants to communicate with another computer in that building, it can do so via a switch.

A switch receives and forwards data to the destination device. A PC is connected via an **ethernet cable** to the switch. You must know that switches operate at **Layer 2** of the TCP/IP or the OSI model.

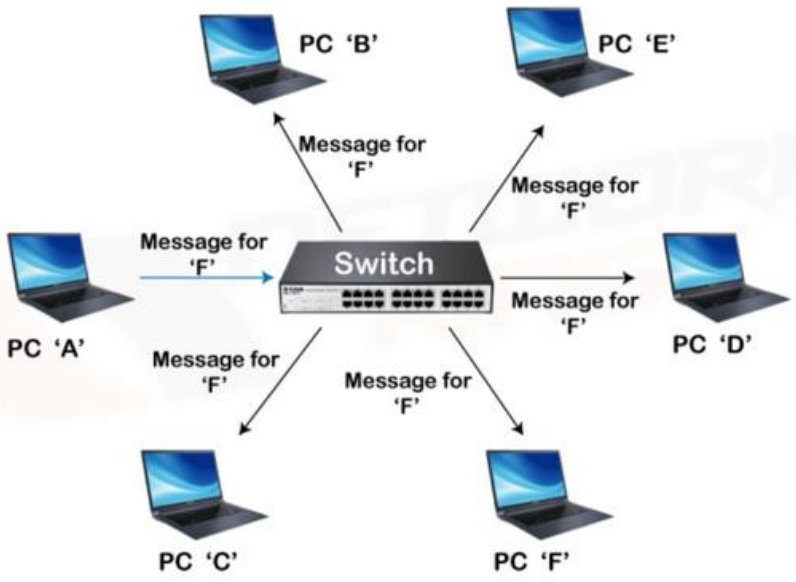


Fig-Switch

Connector-RJ-45-

RJ45 connector is a standard connector, mainly used in Ethernet to connect various network devices, such as computers, routers, switches, etc. It has the characteristics of standardized interfaces, reliable structure, and easy installation, making it a common and cost-effective network connection technology that plays an important role in modern network and communication fields.



Fig-RJ 45 Connector

Paring Rules and Color Code: When working with network cables, understanding color codes is crucial. Ethernet cables, like Cat5, Cat5e, and Cat6, follow specific color codes depending on their type and use.

Color Codes

- **568A:**
  - Pin 1: White/Green
  - Pin 2: Green
  - Pin 3: White/Orange
  - Pin 4: Blue
  - Pin 5: White/Blue
  - Pin 6: Orange
  - Pin 7: White/Brown
  - Pin 8: Brown
- **568B**
  - Pin 1: White/Orange
  - Pin 2: Orange
  - Pin 3: White/Green
  - Pin 4: Blue
  - Pin 5: White/Blue
  - Pin 6: Green
  - Pin 7: White/Brown
  - Pin 8: Brown

Straight-Through vs. Crossover Cables

- **Straight-Through Cables:** Commonly used to connect different devices like computers to routers, switches, or hubs. Both ends of the cable use the same color code (either 568A or 568B).
- **Crossover Cables:** Used to connect similar devices. One end uses the 568A color code and the other end uses 568B.

Line Tester-

- Cable testers are used **to identify the wiring configuration of network cables**. This is particularly important for ensuring that Ethernet cables are wired correctly and to the applicable standard. Correct wiring is crucial for the proper operation of network devices.



Fig-Line Tester

Ping Command- The full form of PING is the Packet InterNet Groper. It is a computer network management system software or utility software used to test the network communication between two devices. Ping works by sending Internet Control Message Protocol (ICMP) packets to the target host and waiting for a response. After the command completes, you will see a small error report, packet loss, and a statistical summary of the results, usually including the minimum, maximum, average round trip time in ms, and the standard deviation of the average.

- In Network Utility, choose Ping. Enter a destination into the box, either an **IP Address** or a **Domain Name**, and click Ping.
- e.g. ping 8.8.8.8
- **OR** ping www.google.com

Procedure for LAN-

Setting up a local network and file sharing, Use the steps in this section to set up a home network using standard RJ-45 hardware and cables.Confirm that you have the following hardware and follow these steps:

- Network interface card (NIC) or an on-the-motherboard network port for each computer. HP and Compaq computers are network ready with NICs installed. Confirm the computer has a RJ-45 networkport on the back.



Figure : Shape of RJ-45 connector

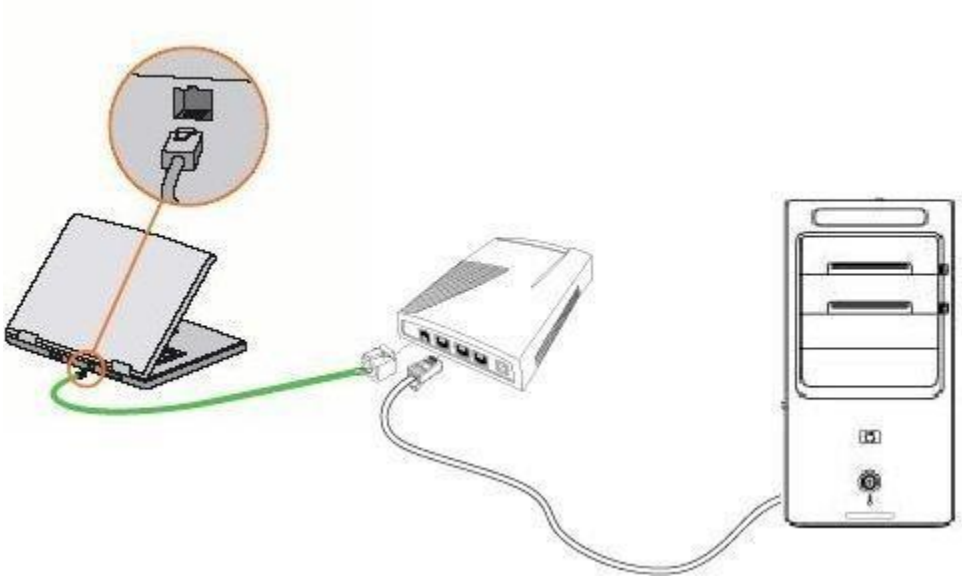
- Disconnect the Internet. If you have a DSL or cable modem, disconnect it.
- Disable any firewall software. Firewall software may interfere with network setup. You can enable thefirewall after network setup is complete.

Step 1: Connecting the network hardware and cables to set up a local network

Do the following to set up the network hardware and connect the networking cables.

1. Set up and turn on the power for the switch or other networking device.
2. Connect the computers to the networking device. If a crossover cable is used,
3. connect the cable to theRJ45 network ports on each computer.

Fig- Example of one possible setup configuration



4. Connect the computer power cords and turn the computers on.

Step 2: Turning on Network discovery and file sharing in Windows 7

Turn on Network discovery and file sharing on each computer that you want to access on the network. Followthese steps to begin setting up the network:

1. Click **Start**, and then click **Control Panel**.
2. Under Network and Internet, click **Choose Homegroup and sharing options**.
3. In the Homegroup settings window, click **Change advanced sharing settings**.
4. Turn on network discovery and file and printer sharing. Review the other settings and turn them on oroff.
5. Click **Save changes**.

Step 3: Sharing drives, folders, and files in a network.

Wireshark Tool-

Wireshark is a network protocol analyzer, or an application that captures packets from a network connection, such as from your computer to your home office or the internet. Packet is the name given to a discrete unit of data in a typical Ethernet network.Wireshark is the most often-used packet sniffer in the world. Like any other packet sniffer, Wireshark does three things:

1. **Packet Capture:** Wireshark listens to a network connection in real time and then grabs entire streams of traffic – quite possibly tens of thousands of packets at a time.
2. **Filtering:** Wireshark is capable of slicing and dicing all of this random live data using filters. By applying a filter, you can obtain just the information you need to see.
3. **Visualization:** Wireshark, like any good packet sniffer, allows you to dive right into the very middle of a network packet. It also allows you to visualize entire conversations and network streams.

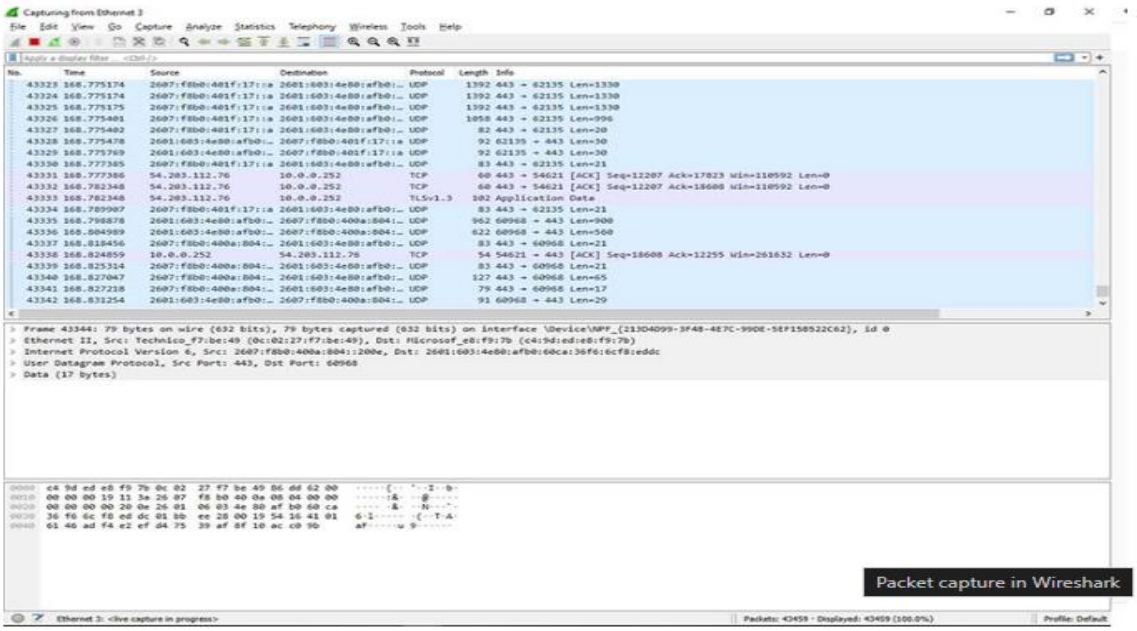


Figure 1: Viewing a packet capture in Wireshark

Conclusion: Hence ,we have demonstrate a wired LAN ,from the physical layer up to the network layer. By preparing and testing cables, configuring network devices, and analyzing network traffic, the fundamental concepts of computer networking are applied in a real-world scenario. The use of both Layer 2 and IP

switches demonstrates how data can be managed and routed within a LAN, while tools like PING and Wireshark help in testing and troubleshooting the network.

ASSIGNMENT NO- 2

Title: Demonstrate the different types of topologies and transmission media by using a packet tracer tool.

Objectives: Configure the network using any topology

Problem Statement: To setup network topology using packet tracer tool.

Tools Required:

Software: Packet Tracer Tool

Theory:

What is a Topology?

- Network topologies give us an overview of logical and physical network layouts containing links and nodes.
- The **physical topology** refers to the configuration of computers, cables, or other peripherals, etc.
- The **logical topology** allows us to pass information between workstations.
- The different types of network topologies are:
    - Bus topology
    - Mesh topology
    - Star topology
    - Ring topology
    - Hybrid/Tree Topology

1.Bus Topology-Bus topology employs a single cable (Bus) to connect all the nodes. The main cable serves as the network's spine. All nodes in a Bus Topology are linked to the Taps and Drop Lines via the bus. Drop Lines are the connections between the central wire or bus and the nodes in this case. The Taps are the three-way connector that aids in connecting the drop line to the main central cable.

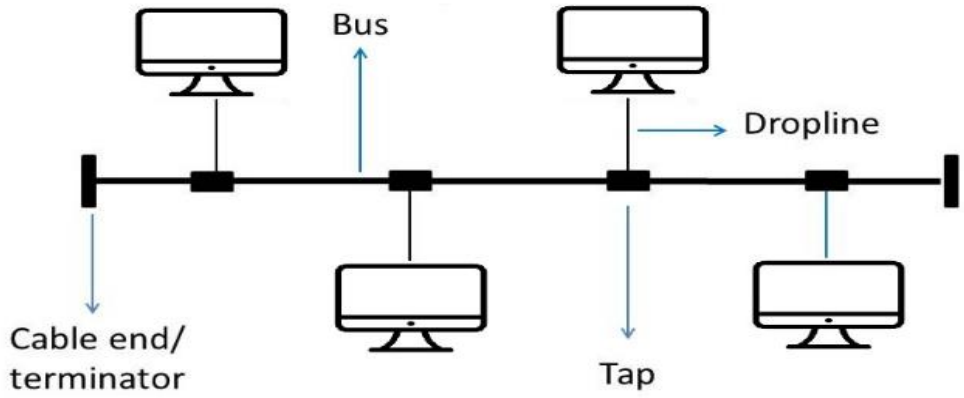


Fig-Bus Topology

The data travels only in one direction, and when it reaches the far end of the line, the terminator removes it from the line. In a bus topology, one computer acts as a server, and the other computers act as clients. A bus topology example is connecting two floors with a single line.

Advantages -

1. **Less cabling:** A common wire connects all nodes in a bus topology.
2. **Less Expensive:** Bus topology is less expensive because it uses a common wire.
3. **Small network:** This is best suited for situations where only a few computers are required for connection establishment.
4. **Upgradeable:** A new node can be added or removed in this topology without affecting the other nodes.

Disadvantages -

1. **Reduced signal strength**: To connect a more significant number of nodes, we must increase the number of Taps, Drop Lines, and the central cable. And increasing these things will weaken the signal.
2. **Core failure:** If the main central cable becomes damaged or faulty, the entire network will fail.
3. **Low security:** This is a significant security issue because all nodes in the network can hear what data is transmitted to other nodes in the network.

2.Star Topologies:-

In Star Topology, all the devices are connected to a single hub through a cable. This hub is the central node and all other nodes are connected to the central node. The hub can be passive in nature i.e., not an intelligent hub such as broadcasting devices, at the same time the hub can be intelligent known as an active hub. Active hubs have repeaters in them. Coaxial cables or RJ-45 cables are used to connect the computers. In Star Topology, many popular Ethernet LAN protocols are used as CD(Collision Detection), CSMA (Carrier Sense Multiple Access), etc.

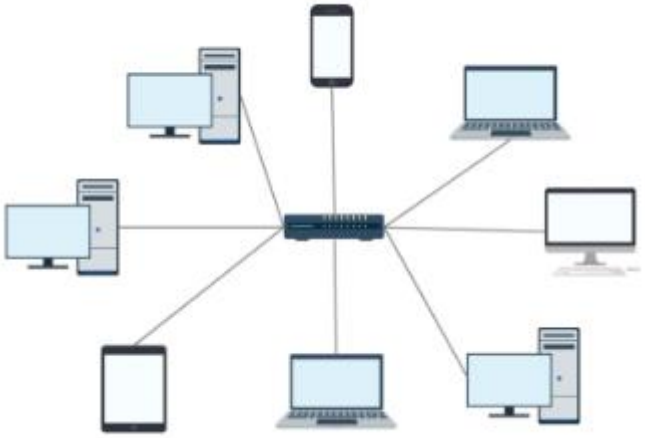


Fig-Star Topology

Advantages-

1. Device failure (computer, cable) doesn't damage the entire network. Each device has its own dedicated connection, isolating issues.
2. Adding or removing devices requires only a single cable connection to the central hub. There is no need to reconfigure the entire network.
3. Dedicated connections prevent data packets from colliding, leading to smoother network traffic and improved performance.

4. Individual device connections make it harder for unauthorized users to access the network, as they need to breach the central hub/switch security.

Disadvantages-

1. The central hub is critical. If it fails, the entire network becomes inoperable, impacting all connected devices.
2. Requires more cabling than bus topology, leading to higher installation and maintenance expenses. Central hubs can also be costly, especially for high-performance models.
3. Network performance depends heavily on the capacity and capabilities of the central device. A limited hub can hinder overall network speed and performance.
4. Managing and maintaining the central hub requires more resources and technical expertise,

3. **Ring Topology:-**

Token Ring Network, in computer science, a LAN formed in a ring (closed loop) topology that uses token passing as a means of regulating traffic. On a token ring network, a token governing the right to transmit is passed from one station to the next in a physical circle. If a station has information to transmit, it "seizes" the token, marks it as being in use, and inserts the information. The "busy" token, plus message, is then passed around the circle, copied when it arrives at its destination, and eventually returned to the sender. The sender removes the attached message and then passes the freed token to the next station in line. Token ring networks are defined in the IEEE 802.5 standards.

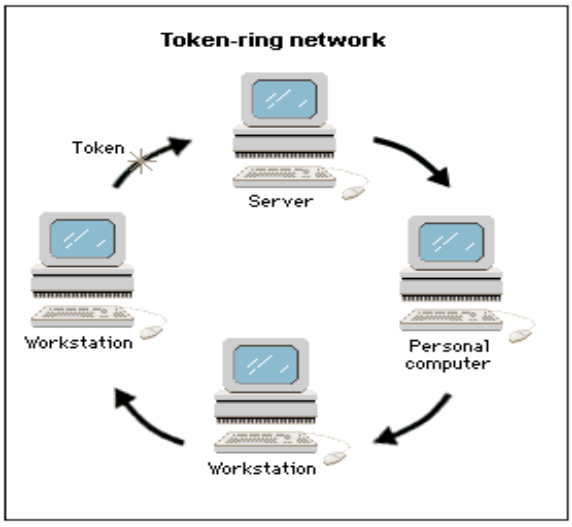


Fig-Ring Topology

Advantages-

1. It reduces the chance of packet collisions as all data flows in one direction in this topology.
2. Any network server is not required in the ring topology to network connectivity between each workstation.
3. It has the ability to send data at high speeds.
4. In this network, if you add additional workstations, they do not impact the performance of the network.

Disdvantages-

1. It is much slower as compared to a star topology, as all data in ring topology must pass through each workstation on the network that making it slower.
2. If one workstation gets failure, the entire network will be impacted.
3. It is more expensive as compared to Ethernet cards, hubs, or switches because, in this network, the hardware is required to connect each workstation to the network.

4.Mesh Topology:-

The type of network topology in which each of the nodes of the network is connected to each of the other nodes in the network with a point-to-point link – this makes it possible for data to be simultaneously transmitted from any single node to all of the other nodes.

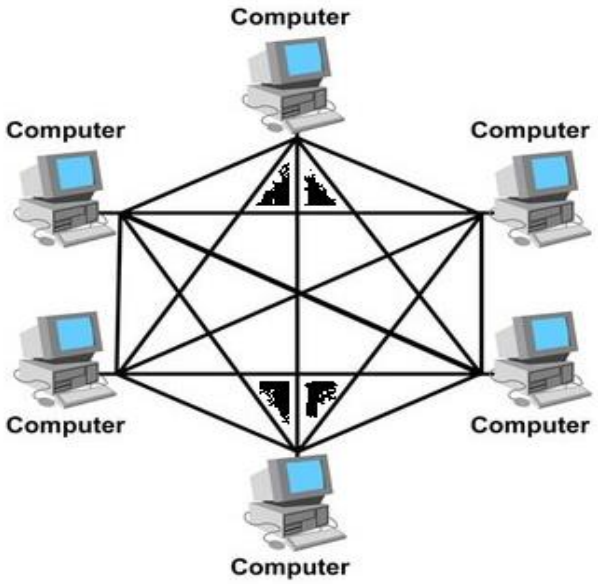


Fig-Mesh Topology

Note: The physical fully connected mesh topology is generally too costly and complex for practical networks, although the topology is used when there are only a small number of nodes to be interconnected.

Advantages-

There are various benefits of mesh topology, which are discussed below:

1. **Scalable:** In a mesh topology, each node acts like Router. However, there are no exclusive Routers. It easy to add an additional node in this topology and connect it to the network. Additionally, to scale up the network, extra effort is not needed.
2. **Robust:** If any single node gets fails in the system, the network availability will not be affected and will be maintained. And, robust features are included in this topology to overcome any situation. Furthermore, this topology has no total shutdown.
3. **Lower Cost:** It needs less investment in infrastructure as it is a highly decentralized system. Also, to manage the network, there are no central servers.
4. **Redundancy:** This topology built a lot of redundancy to keep maximum uptime and offers numerous paths to reach the destination.

Disadvantages-

1. **Complex:** In this topology, each node works as a router that increases complexity.
2. **Planning:** This topology offers flexibility and scalability as it allows the addition of new devices in the existing network, which will have to ensure uniform latency across all nodes. Hence, network planning makes it a little difficult.
3. **Power Consumption:** All the time in this network setup, each node will have to remain active that led to high power consumption and increase the load.
4. As compared to other network topologies, such as point to point, star, bus, the cost of mesh topology is high.

5.Hybrid/Tree Topology:-

A tree topology combines characteristics of linear bus and star topologies. It consists of groups of star-configured workstations connected to a linear bus backbone cable.

These topologies can also be mixed. For example, a bus-star network consists of a high-bandwidth bus, called the **backbone,** which connects a collection of slower-bandwidth star segments.

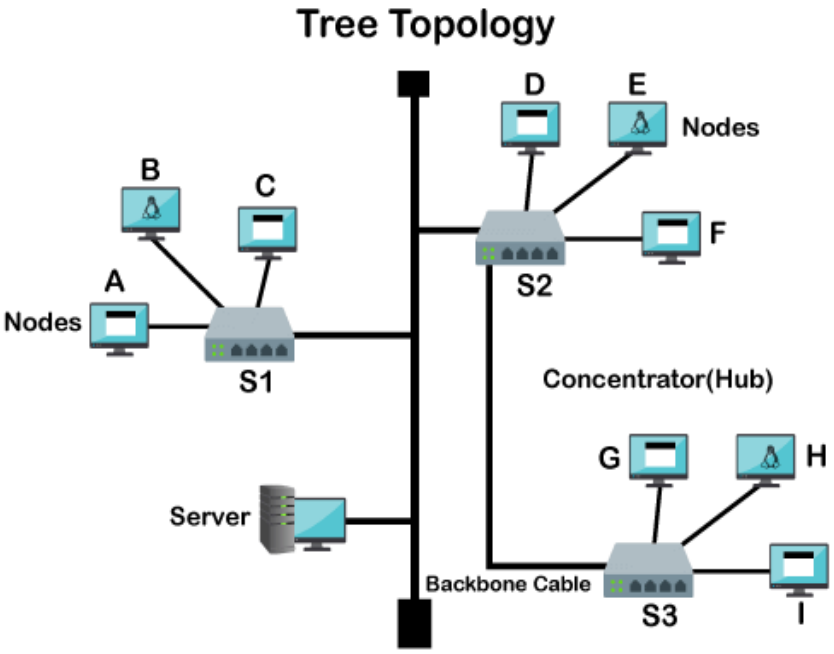


Fig-Tree Topology

Advantages-

1. It is more scalable and flexible as it does not require puncturing the main backbone cable to add devices, unlike the Bus Topology. Even if less space is available, you can easily add a new device to the hub or switch. You can create more space through Star Network.
2. If one or more nodes have failed in the network, they do not affect the entire network. And, the network can still continue to operate, even if a single Switch or Hub has been damaged.
3. A tree topology network can be managed and maintained easily.

4. In this topology, Node expansion is easy and fast.

Disadvantages-

1. This network is very difficult to configure as compared to the other network topologies.
2. The length of a segment is limited & the limit of the segment depends on the type of cabling used.
3. Due to the presence of a large number of nodes, the network performance of tree topology becomes a bit slow.

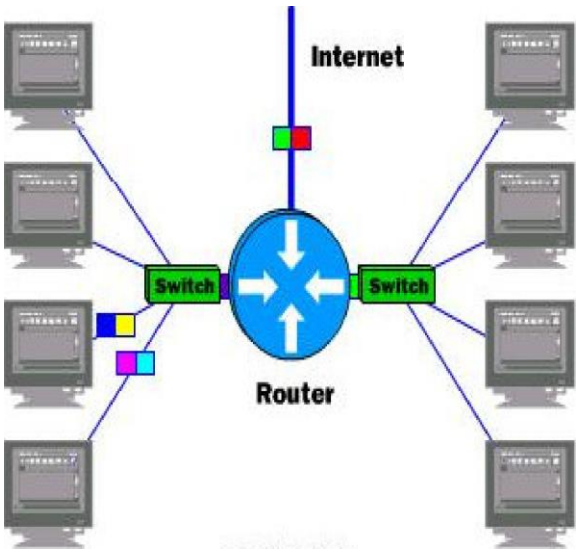
Packet Tracer Tool

Packet Tracer is a cross-platform visual simulation tool designed by Cisco Systems that allows users to create network topologies and imitate modern computer networks. The software allows users to simulate the configuration of Cisco routers and switches using a simulated command line interface.

Cisco Packet Tracer as the name suggests, is a tool built by Cisco. This tool provides a network simulation to practice simple and complex networks.

The main purpose of Cisco Packet Tracer is to helps in learning the principles of networking with hands-on experience as well as develop Cisco technology specific skills. Since the protocols are implemented in software only method, this tool cannot replace the hardware Routers or Switches.

How Routers Work



Routers are the traffic cops of intranets. They make sure that all data gets sent to where it's supposed to go and that it gets sent via the most efficient route. Routers are also useful tools to make the most efficient use of the intranet. Routers are used to segment traffic and provide redundancy of routes. Routers use encapsulation to permit different protocols to be sent across otherwise incompatible networks.

Just as routers direct traffic on the Internet, sending information to its proper destination, routers on an intranet perform the same function. Routers-equipment that is a combination of hardware and software-can send the data to a computer on the same subnetwork inside the intranet, to another network on the intranet, or outside to the Internet. They do this by examining header information in IP packets, and then sending the data on its way. Typically, a router will send the packet to the next router closest to the final destination, which in turn sends it to an even closer router, and so on, until the data reaches its intended recipient.

A router has input ports for receiving IP packets, and output ports for sending those packets toward their destination. When a packet comes to the input port, the router examines the packet header, and checks the destination in it against a routing table-a database that tells the router how to send packets to various destinations.

Based on the information in the routing table, the packet is sent to a particular output port, which sends the packet to the next closest router to the packet's destination.

If packets come to the input port more quickly than the router can process them, they are sent to a holding area called an input queue. The router then processes packets from the queue in the order they were received. If the number of packets received exceeds the capacity of the queue (called the length of the queue), packets may be lost.

In a simple intranet that is a single, completely self-contained network, and in which there are no connections to any other network or the intranet, only minimal routing need be done, and so the routing table in the router is exceedingly simple with very few entries, and is constructed automatically by a program called *ifconfig*

Implementation-

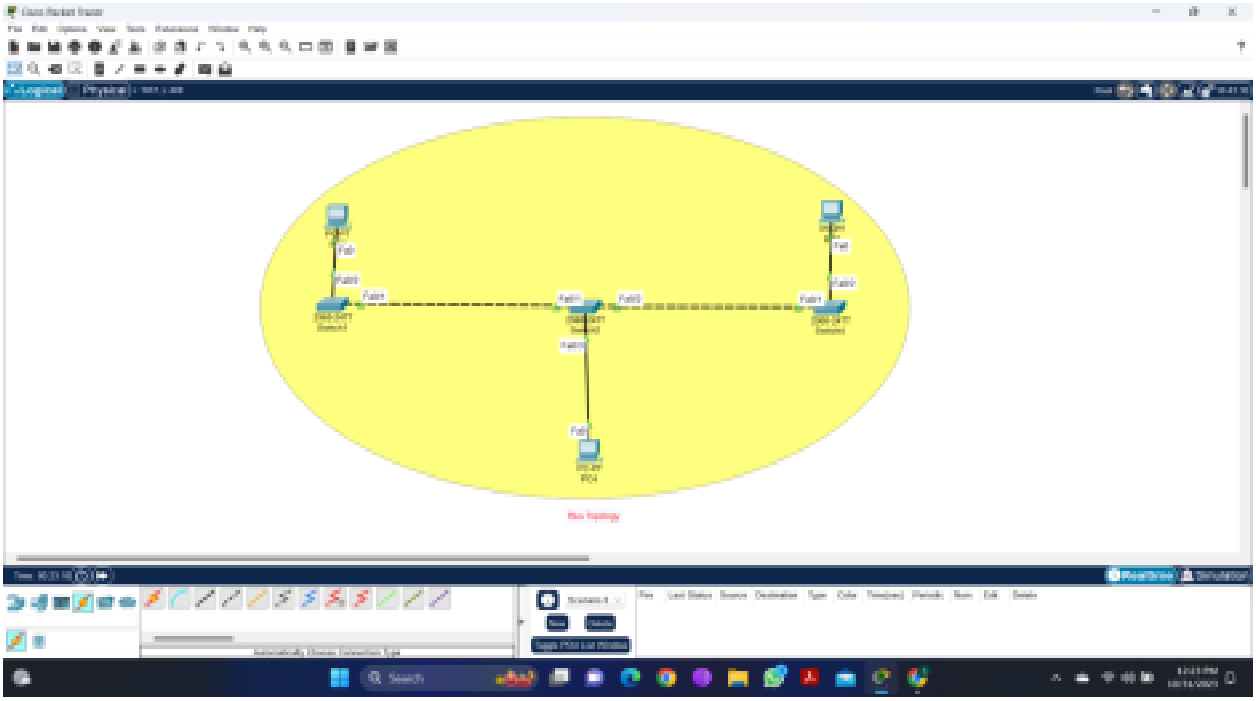


Fig-BUS Topology

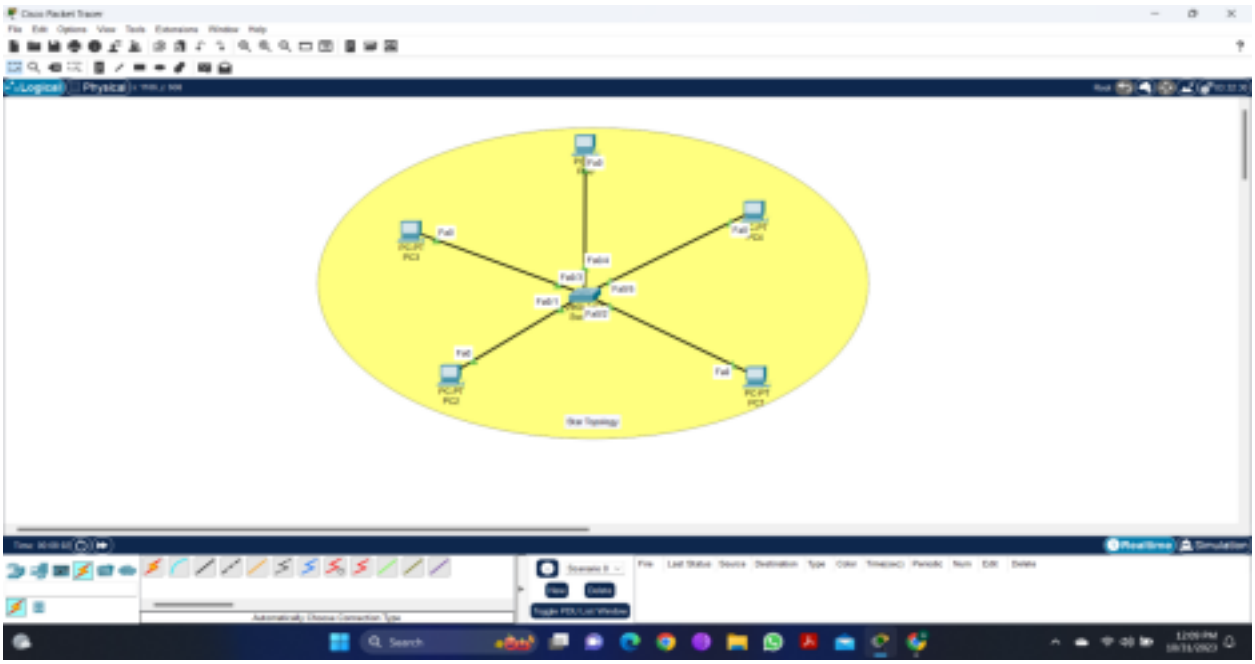


Fig- Star Topology:

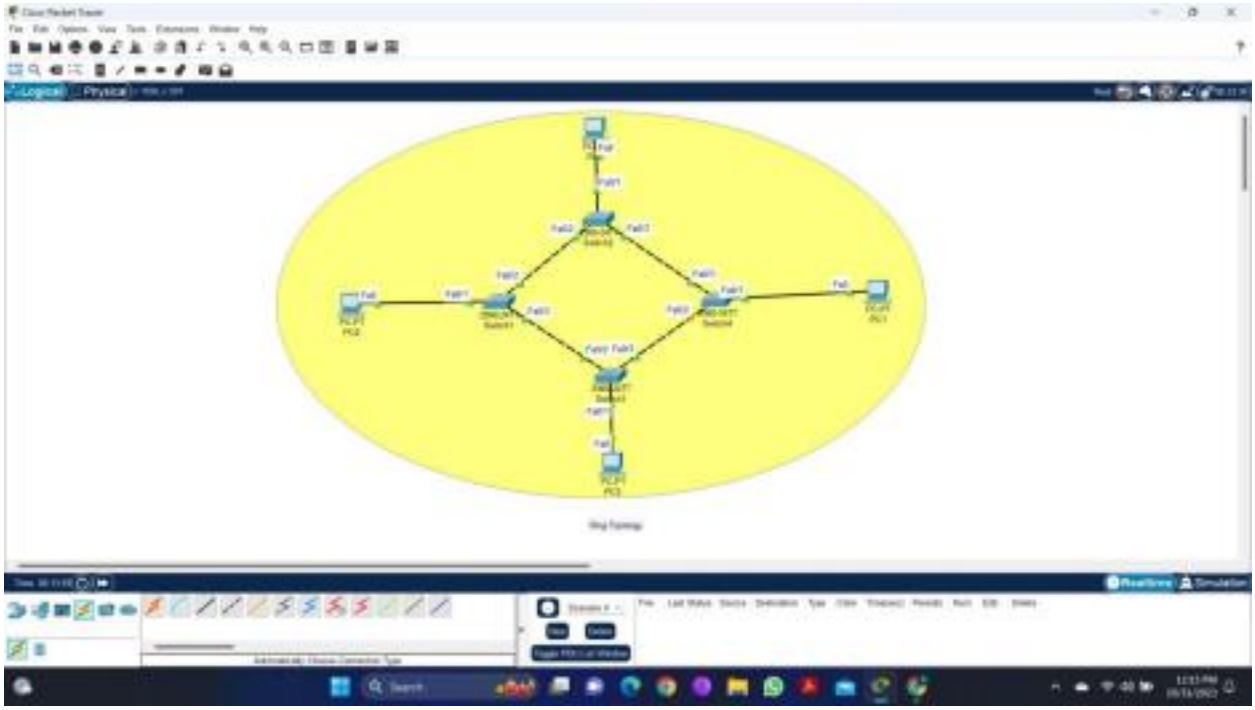


Fig-3. Ring Topology

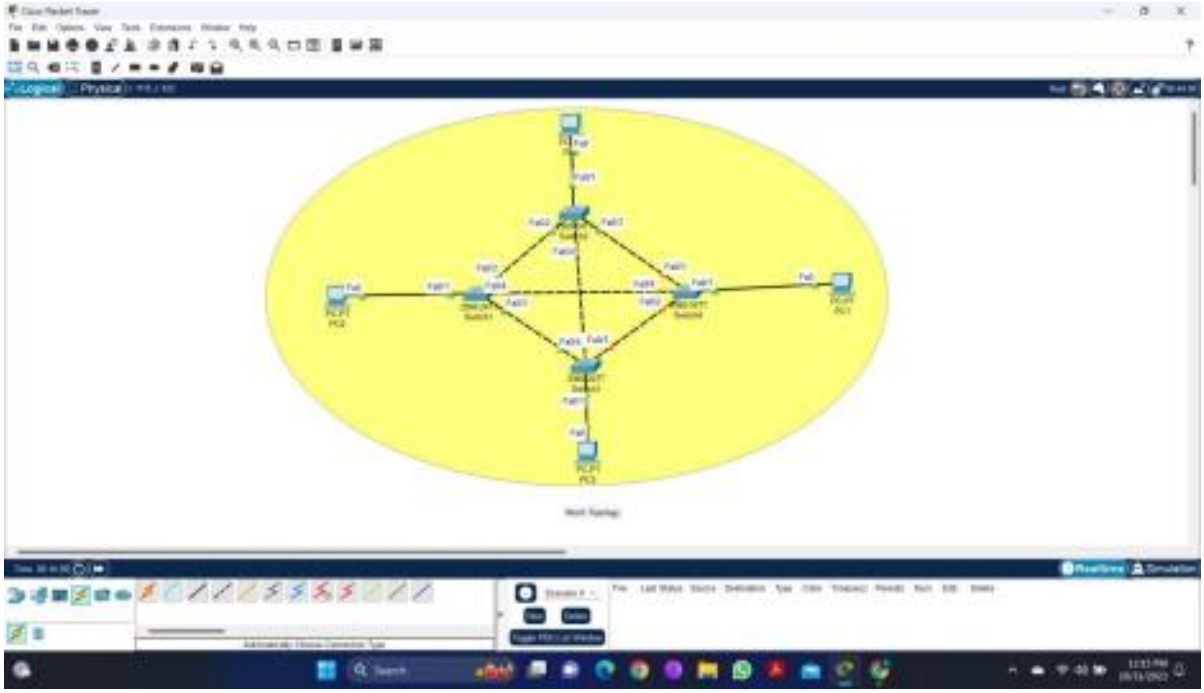


Fig- Mesh Topology

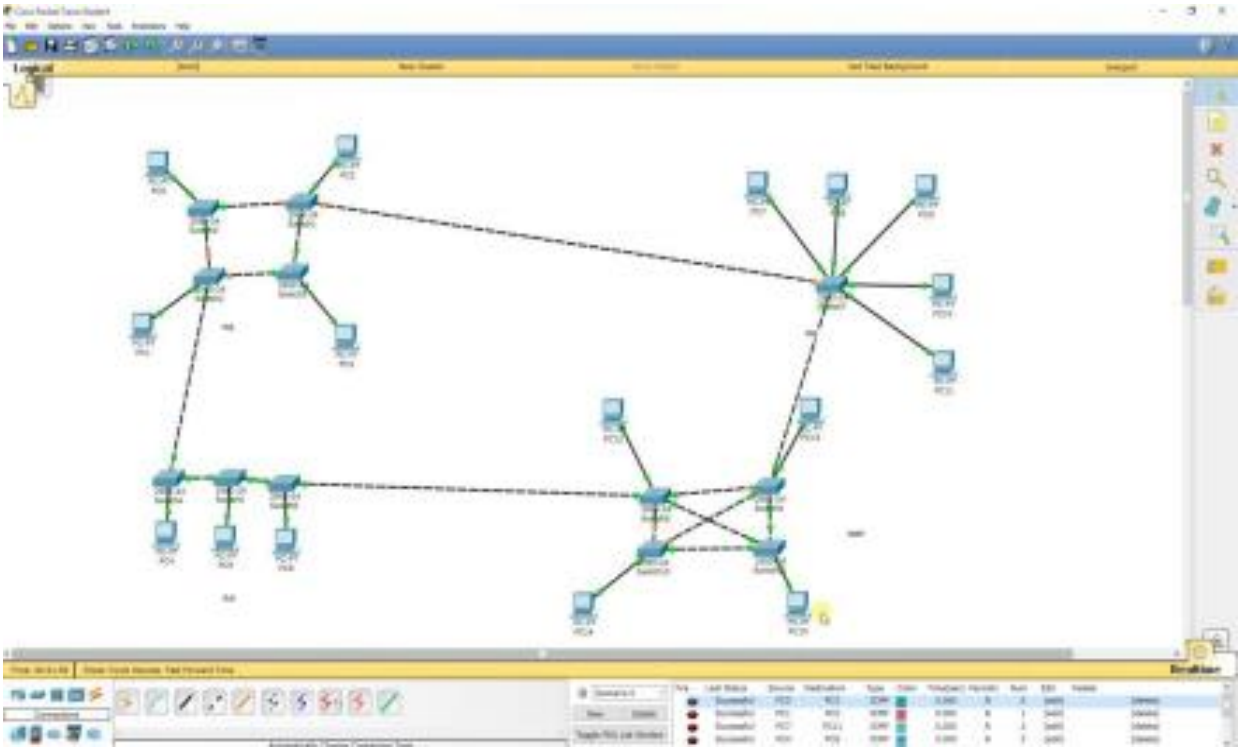


Fig- Hybrid Topology

Conclusion: Hence, we have configured a network topology using CPT.

ASSIGNMENT NO- 3

Title: Setup a WAN which contains wired as well as wireless LAN by using a packet tracer tool. Demonstrate transfer of a packet from LAN 1 (wired LAN) to LAN2 (Wireless LAN)

Objectives : To Configure wired and wireless LAN using CPT

Problem Statement: Setup a WAN which contains wired as well as wireless LAN by using a packet tracer tool. Demonstratetransfer of a packet from LAN 1 (wired LAN) to LAN2 (Wireless LAN).

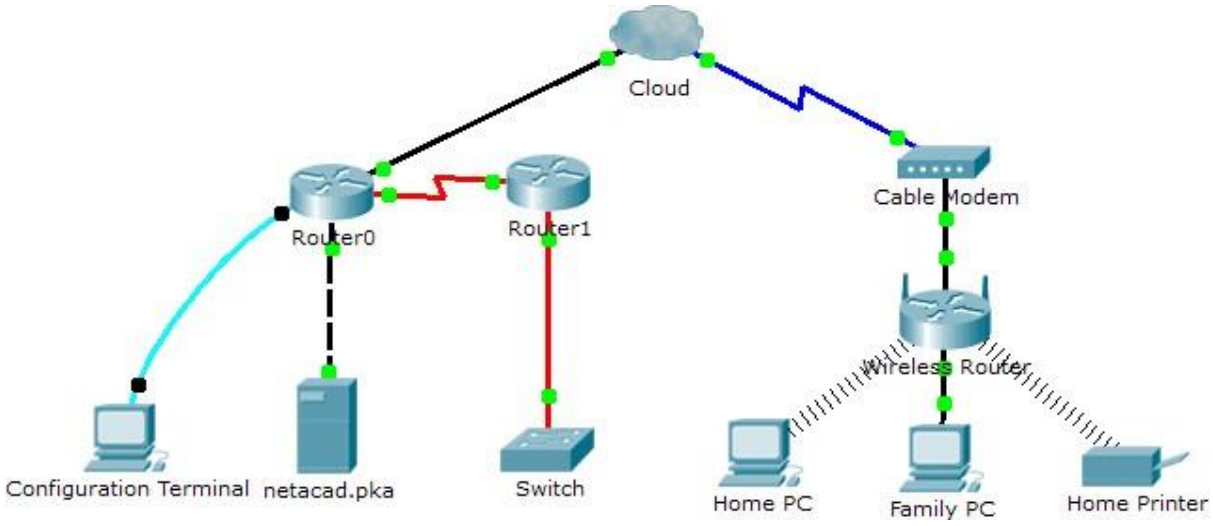
Tools Required:

Software: Packet Tracer Tool

Theory:

Packet Tracer

Connecting a wired and wireless LAN. Red font color or gray highlights indicate text that appears in the Answer copy only.



When working in Packet Tracer (a lab environment or a corporate setting), you should know how to select the appropriate cable and how to properly connect devices. This activity will examine device configurations in Packet Tracer, selecting the proper cable based on the configuration, and connecting the devices. This activity will also explore the physical view of the network in Packet Tracer.

There are many methods of deploying wide area network (WAN) when using Cisco routers. Some of these methods are HDLC, PPP, Frame Relay, and MPLS. The Point to point protocol is an industry standard WAN protocol that allows for the deployment of secured wide area connections between routers from different vendors. In this post, I will share with us on how to successfully configure PPP with authentication between two Cisco routers stationed miles apart from each other.

IMPLEMENTATION:

ROUTER A

```
Router#conf t

Router(config)#host RouterA
RouterA(config)#int s0/3/0
RouterA(config-if)#clock rate 64000


RouterA(config-if)#ip add 192.168.1.1 255.255.255.252
RouterA(config-if)#no shut
RouterA(config-if)#encapsulation ppp
 RouterA(config-if)#ppp authentication chap
RouterA(config-if)#username RouterB pass cisco
RouterA(config)#
```

Router B-

```
Router>en
Router#conf t

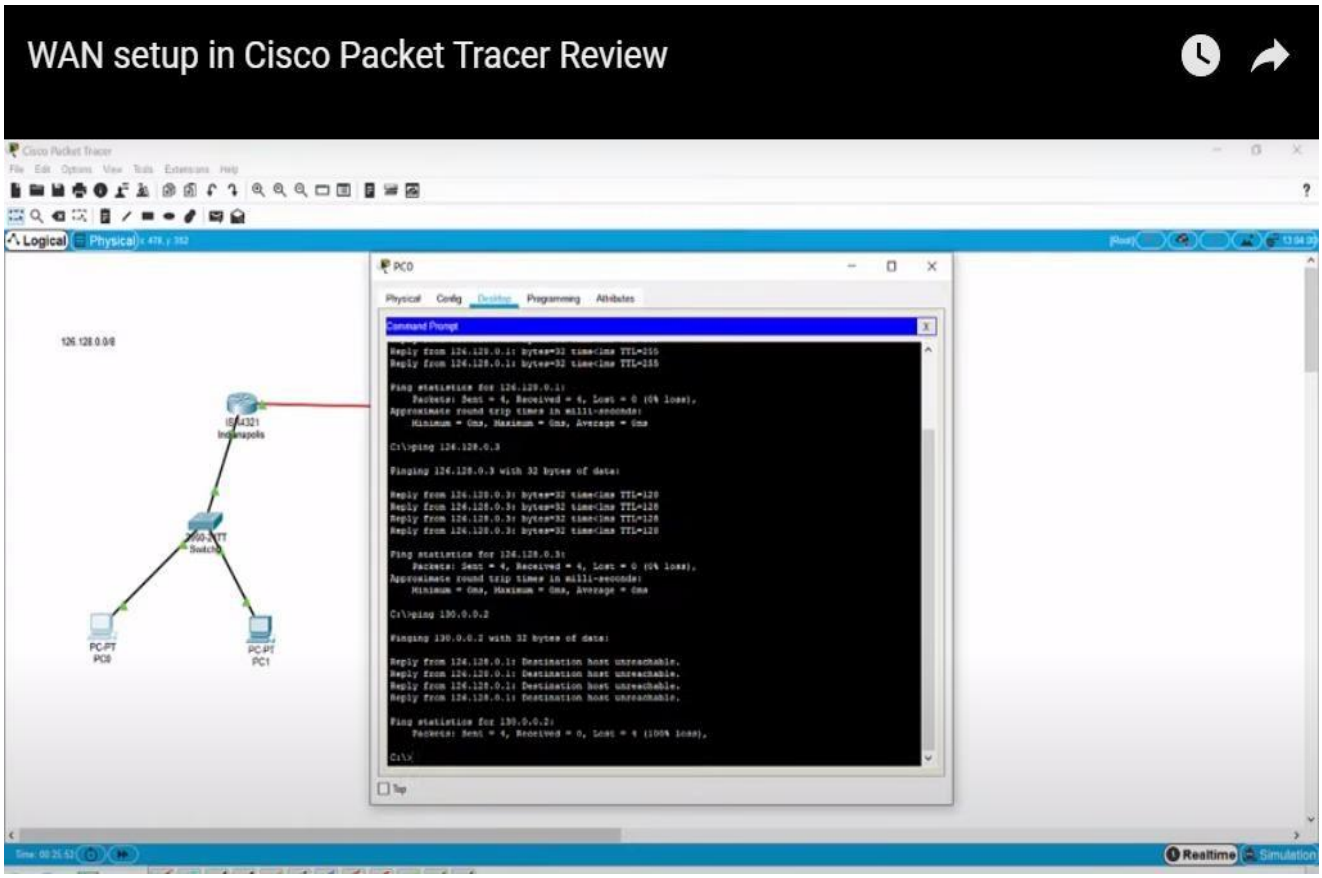
Router(config)#host RouterB
RouterB(config)#int s0/3/0

RouterB(config-if)#ip add 192.168.1.2 255.255.255.252
RouterB(config-if)#no shut
RouterB(config-if)#encapsulation ppp
RouterB(config-if)#ppp authentication chap
```

RouterB(config-if)#username

RouterA pass ciscoRouterB(config)#

Output/Screenshot-



Conclusion: Packet Tracer tool is used for configuration WLAN

ASSIGNMENT NO-4

Title: Write a program for error detection and correction for 7/8 bits ASCII codes using Hamming Codes or CRC.

Objectives : To implement error detection and correction techniques

Problem Statement: Write a program for error detection and correction for 7/8 bits ASCII codes using Hamming Codes or CRC.

Outcome : Demonstrate Hamming Codes or CRC with example.

Software Requirements :gcc

Hardware Requirements : Open source linux operating system.

Hamming Code

Hamming codes can detect up to two-bit errors or correct one-bit errors without detection of uncorrected errors. By contrast, the simple parity code cannot correct errors, and can detect only an odd number of bits in error In this context, an extended Hamming code having one extra parity bit is often used.

In Hamming codes the bits of the codeword are numbered consecutively, starting with bit 1 at the left end, bit 2 to its immediate right, and so on. The bits that are powers of 2 (1, 2, 4, 8, 16, etc.) are check bits. The rest (3, 5, 6, 7, 9, etc.) are filled up with the m data bits. This pattern is shown for an (11,7)

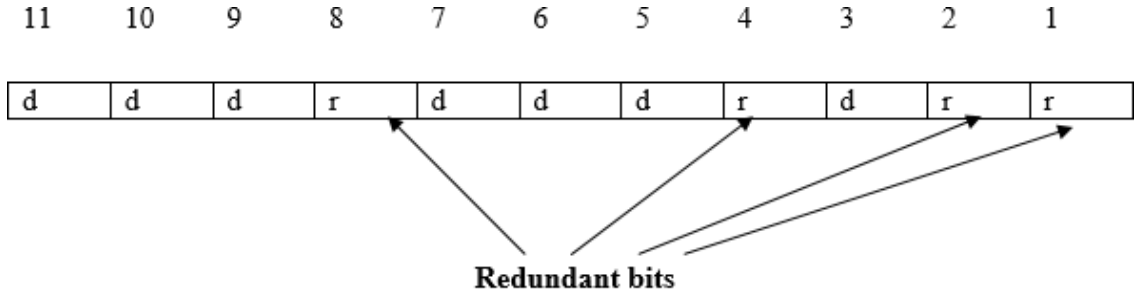


Figure Positions of redundancy bits in hamming code

Hamming code with 7 data bits and 4 check .

- Basic approach for error detection by using Hamming code is as follows:
- To each group of m information bits k parity bits are added to form (m+k) bit code.
- Location of each of the (m+k) digits is assigned a decimal value.
- The k parity bits are placed in positions 1, 2, …, 2k-1 positions.–K parity checks are performed on selecteddigits of each codeword.

- At the receiving end the parity bits are recalculated. The decimal value of the k parity bits provides the bit-position in error, if any.

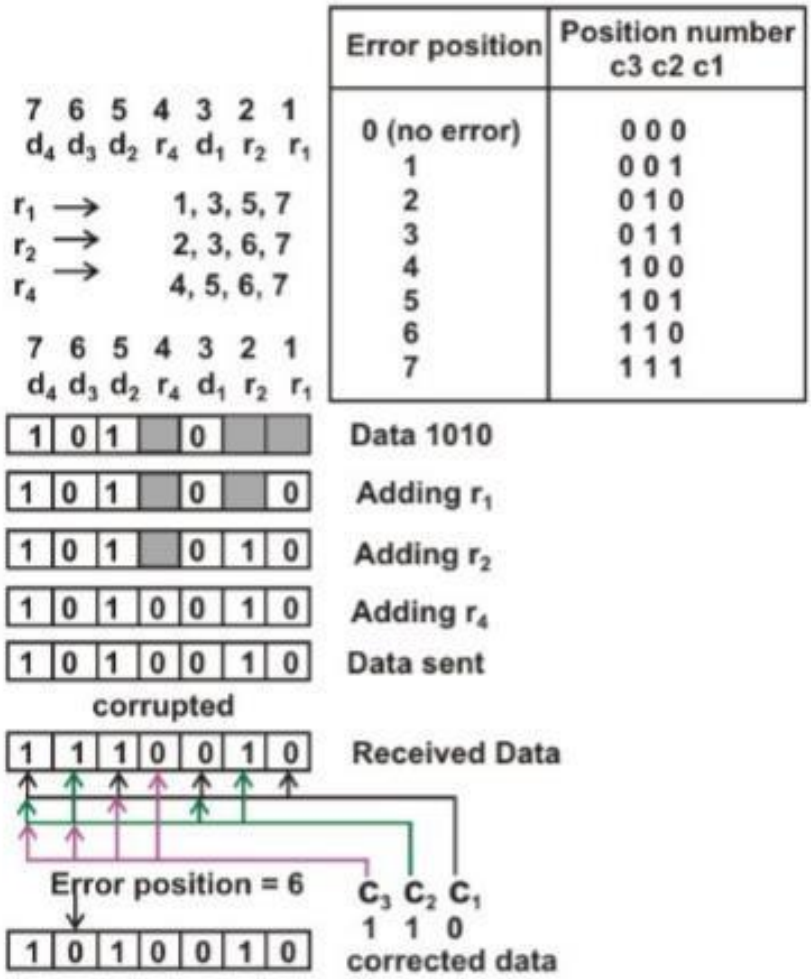


Figure Use of Hamming code for error correction for a 4-bit data

Cyclic Redundancy Checks (CRC)

This Cyclic Redundancy Check is the most powerful and easy to implement technique. Unlike checksum scheme, which is based on addition, CRC is based on binary division. In CRC, a sequence of redundant bits, called cyclic redundancy check bits, are appended to the end of data unit so that the resulting data unit becomesexactly divisible by a second, predetermined binary number. At the destination, the incoming data unit is divided by the same number. If at this step there is no remainder, the data unit is assumed to be correct and is therefore accepted. A remainder indicates that the data unit has been damaged in transit and therefore must be rejected. The

generalized technique can be explained as follows.

If a k bit message is to be transmitted, the transmitter generates an r-bit sequence, known as Frame Check Sequence (FCS) so that the (k+r) bits are actually being transmitted. Now this r-bit FCS is generated by dividingthe original number, appended by r zeros, by a predetermined number. This number, which is (r+1) bit in length,can also be considered as the coefficients of a polynomial, called Generator Polynomial. The remainder of thisdivision process generates the r-bit FCS. On receiving the packet, the receiver divides the (k+r) bit frame by the same predetermined number and if it produces no remainder, it can be assumed that no error has occurred during the transmission. Operations at both the sender and receiver end are shown in Fig.

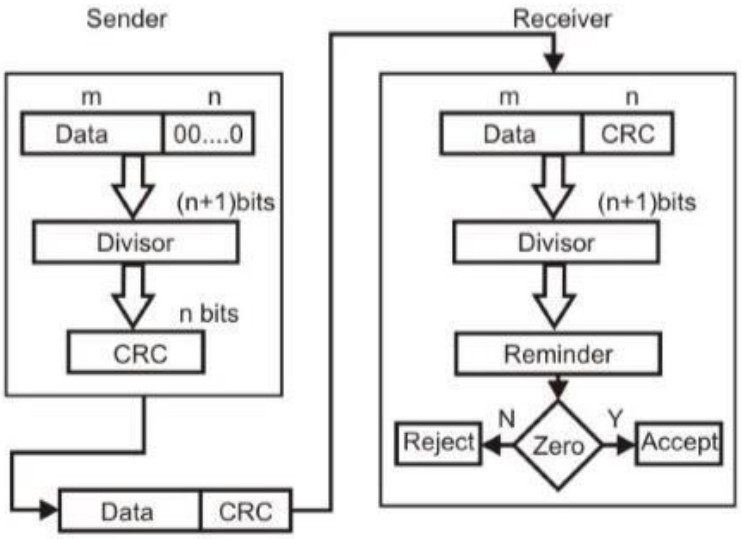


Figure Basic scheme for Cyclic Redundancy Checking

This mathematical operation performed is illustrated in Figure. by dividing a sample 4bit number by the coefficient of the generator polynomial x3+x+1, which is 1011, using the modulo-2 arithmetic. Modulo-2 arithmetic is a binary addition process without any carry over, which is just the Exclusive-OR operation. Consider the case where k=1101. Hence we have to divide 1101000 (i.e. k appended by 3 zeros) by 1011, which produces the remainder r=001, so that the bit frame (k+r) =1101001 is actually being transmitted throughthe communication channel. At the receiving end, if the received number, i.e., 1101001 is divided by the samegenerator polynomial 1011 to get the remainder as 000, it can be assumed that the data is free of errors.

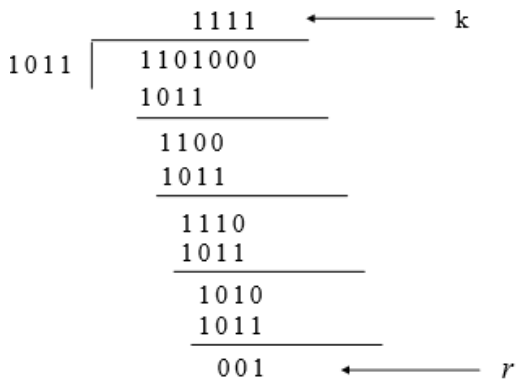


Figure Cyclic Redundancy Checks (CRC)

The transmitter can generate the CRC by using a feedback shift register circuit. The same circuit can also be used at the receiving end to check whether any error has occurred. All the values can be expressed as polynomials of a dummy variable X.

Implementation :

CRC

```
#include <iostream>
using namespace std;

int main()
{
   int i,j,k,l;

      //Get Frame
      int fs;
      cout<<"\n Enter Frame size: ";
      cin>>fs;

      int f[20];

      cout<<"\n Enter Frame:";
```

```
for(i=0;i<fs;i++)
{
cin>>f[i];
}

//Get Generator
int gs;
cout<<"\n Enter Generator size: ";
cin>>gs;

int g[20];

cout<<"\n Enter Generator:";
for(i=0;i<gs;i++)
{
cin>>g[i];
}
cout<<"\n Sender Side:";
cout<<"\n Frame: ";
for(i=0;i<fs;i++)
{
cout<<f[i];
}
cout<<"\n Generator :";
for(i=0;i<gs;i++)
{
cout<<g[i];
}

//Append 0's
```

```
int rs=gs-1;
cout<<"\n Number of 0's to be appended: "<<rs;
for (i=fs;i<fs+rs;i++)
{
f[i]=0;
}

int temp[20];
for(i=0;i<20;i++)
{
temp[i]=f[i];
}

cout<<"\n Message after appending 0's :";
for(i=0; i<fs+rs;i++)
{
cout<<temp[i];
}

//Division
for(i=0;i<fs;i++)
{
j=0;
k=i;
//check whether it is divisible or not
if (temp[k]>=g[j])
{
   for(j=0,k=i;j<gs;j++,k++)
   {
      if((temp[k]==1 && g[j]==1) || (temp[k]==0 && g[j]==0))
```

```
        {
           temp[k]=0;
        }
        else
        {
           temp[k]=1;
        }
     }
}
}

//CRC
int crc[15];
for(i=0,j=fs;i<rs;i++,j++)
{
crc[i]=temp[j];
}

cout<<"\n CRC bits: ";
for(i=0;i<rs;i++)
{
cout<<crc[i];
}

cout<<"\n Transmitted Frame: ";
int tf[15];
for(i=0;i<fs;i++)
{
tf[i]=f[i];
}
```

```
for(i=fs,j=0;i<fs+rs;i++,j++)
{
tf[i]=crc[j];
}
for(i=0;i<fs+rs;i++)
{
cout<<tf[i];
}

cout<<"\n Receiver side : ";
cout<<"\n Received Frame: ";
for(i=0;i<fs+rs;i++)
{
cout<<tf[i];
}

for(i=0;i<fs+rs;i++)
{
temp[i]=tf[i];
}

//Division
for(i=0;i<fs+rs;i++)
{
j=0;
k=i;
if (temp[k]>=g[j])
{
   for(j=0,k=i;j<gs;j++,k++)
   {
```

```
        if((temp[k]==1 && g[j]==1) || (temp[k]==0 && g[j]==0))
        {
           temp[k]=0;
        }
        else
        {
           temp[k]=1;
        }
     }
}
}

cout<<"\n Reaminder: ";
int rrem[15];
for (i=fs,j=0;i<fs+rs;i++,j++)
{
rrem[j]= temp[i];
}
for(i=0;i<rs;i++)
{
cout<<rrem[i];
}

int flag=0;
for(i=0;i<rs;i++)
{
if(rrem[i]!=0)
{
   flag=1;
}
```

```
    }

    if(flag==0)
    {
    cout<<"\n Since Remainder Is 0 Hence Message Transmitted From Sender To Receiver Is Correct";
    }
    else
    {
    cout<<"\n Since Remainder Is Not 0 Hence Message Transmitted From Sender To Receiver Contains Error";
    }
    return 0;
}

/* OUTPUT
iotlab@iotlab-Veriton-M200-B360:~$ g++ crc1.cpp
iotlab@iotlab-Veriton-M200-B360:~$ ./a.out

 Enter Frame size: 8

 Enter Frame:1
0
1
1
0
1
1
1

 Enter Generator size: 4
```

Enter Generator:1

0

1

0

Sender Side:

Frame: 10110111

Generator :1010

Number of 0's to be appended: 3

Message after appending 0's :10110111000

CRC bits: 110

Transmitted Frame: 10110111110

Receiver side :

Received Frame: 10110111110

Reaminder: 000

Since Remainder Is 0 Hence Message Transmitted From Sender To Receiver Is Correct

*/

2. Hamming Code

```
#include<iostream>

using namespace std;

int main()
{
   int data[10];
   int dataatrec[10],c,c1,c2,c3,i;
```

```
    cout<<"Enter 4 bits of data one by one\n";
    cin>>data[7];
    cin>>data[6];
    cin>>data[5];
    cin>>data[3];

    //Calculation of even parity
    data[4]=data[5]^data[6]^data[7];
    data[2]=data[3]^data[6]^data[7];
    data[1]=data[3]^data[5]^data[7];

    cout<<"\nEncoded data is\n";
    for(i=1;i<=7;i++)
       cout<<data[i];

    cout<<"\n\nEnter received data bits one by one\n";
    for(i=1;i<=7;i++)
       cin>>dataatrec[i];

    c1=dataatrec[1]^dataatrec[3]^dataatrec[5]^dataatrec[7];
    c2=dataatrec[2]^dataatrec[3]^dataatrec[6]^dataatrec[7];
    c3=dataatrec[4]^dataatrec[5]^dataatrec[6]^dataatrec[7];
    c=c3*4+c2*2+c1;
    if(c==0)
    {
    cout<<"\ncongratulations there is no error: ";
}
else
{
```

```
        cout<<"\nerror on the postion:"<<c;
        cout<<"\nCorrect message is:";
        if(dataatrec[c]==0)
                dataatrec[c]=1;
        else
                dataatrec[c]=0;
        for (i=1;i<=7;i++)
        {
           cout<<dataatrec[i];
        }
    }

        return 0;
}

/*OUTPUT
iotlab@iotlab-Veriton-M200-B360:~$ g++ Ham.cpp
iotlab@iotlab-Veriton-M200-B360:~$ ./a.out
Enter 4 bits of data one by one
1
1
0
0

Encoded data is
1000011

Enter received data bits one by one
1
1
```

0

0

0

1

1

error on the postion:2

Correct message is:1000011

CONCLUSION: Hamming code and CRC code executed successfully.

ASSIGNMENT NO- 5

Title: Write a program to simulate Go back N and Selective Repeat Modes of Sliding Window Protocol in Peer-to-Peer mode

Objectives: To demonstrate Go back N and Selective Repeat Modes of Sliding Window Protocol in peer to peer mode .

Problem Statement:

Write a program to simulate Go back N and Selective Repeat Modes of Sliding Window Protocol in peer to peer mode and demonstrate the packets captured traces using Wireshark Packet Analyzer Tool for peer to peer mode.

Outcomes:

Demonstrate Go back N and Selective Repeat Modes and also captured packets using Wireshark Packet Analyzer Tool for peer to peer mode.

Tools Required:

Software Requirements :jdk

Hardware Requirements : Open source linux operating system.

Theory:

Sliding Window Protocols assumes two-way communication (full duplex).

It uses two types of frames: 1. Data 2. Ack (sequence number of last correctly received frame)

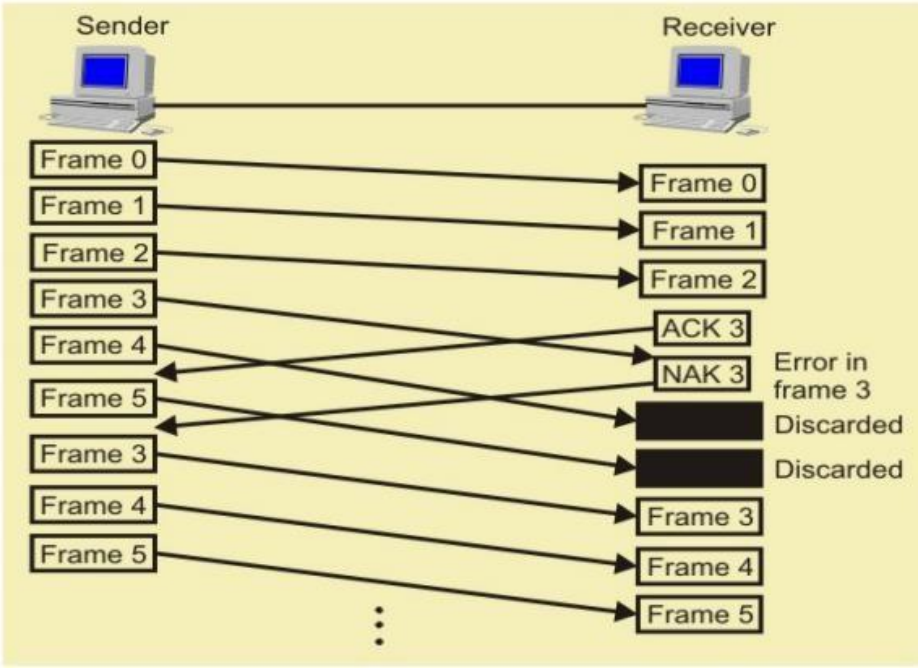
The basic idea of sliding window protocol is that both sender and receiver keep a ``window'' of acknowledgment. The sender keeps the value of expected acknowledgment; while the receiver keeps the value of expected receiving frame. When it receives an acknowledgment from the receiver, the sender advances the window. When it receives the expectedframe, the receiver advances the window.In transmit flow control, sliding window is a variable-duration window that allows a sender to transmit a specified numberof data units before an acknowledgement is received or before a specified event occurs.An example of a sliding window in packet transmission is one in which, after the sender fails to receive an acknowledgement for the first transmitted packet, the sender "slides" the window, i.e. resets the window, and sends a second packet. This process is repeated for the specified number of times before the sender interrupts transmission. Slidingwindow is sometimes (loosely) called acknowledgement delay period.

For example, supposing a fixed window size of m packets, a sender may send out packets before receiving any acknowledgement. If acknowledgement arrives from the receiver for packet n, then the range (window) of unacknowledged packets slides to , and the sender is able to send out packet (n + m). In some way, "sliding" signifies a FIFO operation, trimming the range at one end, extending it at the other end.

The purpose of the sliding window is to increase throughput. Let's denote the round trip time with RTT. The time necessaryto transfer and acknowledge K (a big number of) packets is roughly (in one round trip, 2m packets and 2m ACKs are delivered). However, the size of the window (in bytes) should not grow above "capacity of the path" (the sum of affectednetwork buffer sizes of all hops along the path): windows that are too big do not increase throughput; they only increaselatency, the number of packets transmitted out-of-order, and memory usage.

Go-Back N ARQ Protocol

The most popular ARQ protocol is the go-back-N ARQ, where the sender sends the frames continuously without waitingfor acknowledgement. That is why it is also called as continuous ARQ. As the receiver receives the frames, it keeps on sending ACKs or a NACK, in case a frame is incorrectly received. When the sender receives a NACK, it retransmits theframe in error plus all the succeeding frames.



In practice, protocols often adapt the window size to the link's speed and actual saturation or congestion.

Algorithm *Go-Back-N sender algorithm*

```
1  Sw = 2^m - 1;
2  Sf = 0;
3  Sn = 0;
4
5  while (true)                          //Repeat forever
6  {
7   WaitForEvent();
8    if(Event(RequestToSend»             //A packet to send
9    {
10       if(Sn-Sf >= Sw)                 //If window is full
11             Sleep();
12       GetData();
13       MakeFrame(Sn);
14       StoreFrame(Sn);
15       SendFrame(Sn);
16       Sn = Sn + 1;
17       if(timer not running)
18             StartTimer{);
19   }
20
21   if{Event{ArrivalNotification»       //ACK arrives
22   {
23       Receive(ACK);
24       if{corrupted{ACK»
25             Sleep();
26       if{{ackNo>sf)&&{ackNO<=Sn»      //If a valid ACK
27       While(Sf <= ackNo)
28         {
29          PurgeFrame{Sf);
30          Sf = Sf + 1;
31         }
32         StopTimer();
33   }
34
35   if{Event{TimeOut»                   //The timer expires
36   {
37    StartTimer();
38    Temp = Sf;
39    while{Temp < Sn);
40      {
41       SendFrame(Sf);
42       Sf = Sf + 1;
43      }
44   }
45 }
```

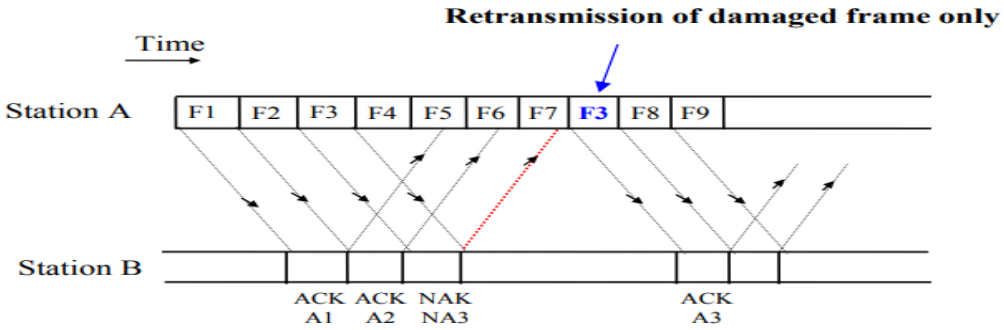
Algorithm *Go-Back-N receiver algorithm*

```
1  Rn = 0;
2
3  while (true)                          //Repeat forever
4  {
5    WaitForEvent();
6
7    if(Event{ArrivalNotification»       /Data frame arrives
8    {
9       Receive(Frame);
10      if(corrupted(Frame»
11            Sleep(};
12      if(seqNo == Rn)                  //If expected frame
13      {
14        DeliverData()i                 //Deliver data
15        Rn = Rn + 1;                   //Slide window
16        SendACK(Rn);
17      }
18   }
19 }
```

Selective Repeat ARQ

Selective Repeat ARQ is a specific instance of the Automatic Repeat-reQuest (ARQ) Protocol, in which the sending process continues to send a number of frames specified by a window size even after a frame loss. Unlike Go-Back-N ARQ, the receiving process will continue to accept and acknowledge frames sent after an initial error.

The selective-repetitive ARQ scheme retransmits only those for which NAKs are received or for which timer has expired,this is shown in the Figure. This is the most efficient among the ARQ schemes, but the sender must be more complex sothat it can send out-of-order frames. The receiver also must have storage space to store the post NAK frames and processing power to reinsert frames in proper sequence.



The receiver process keeps track of the sequence number of the earliest frame it has not received, and sends that numberwith every ACK it sends. If a frame from the sender does not reach the receiver, the sender continues to send subsequent frames until it has emptied its window. The receiver continues to fill its receiving window with the subsequent frames, replying each time with an ACK containing the sequence number of the earliest missing frame. Once the sender has sent all the frames in its window, it resends the frame number given by the ACKs, and then continues where it left off.

The size of the sending and receiving windows must be equal, and half the maximum sequence number (assuming that sequence numbers are numbered from 0 to n-1) to avoid miscommunication in all cases of packets being dropped. The sender moves its window for every packet that is acknowledged.

Algorithm *Sender-site Selective Repeat algorithm*

```
1     = 2^m-1  i
2     = Oi
3     = Oi
4
5   hile  (true)                          //Repeat forever
6   {
7     WaitForEvent()i
8     if(Event(RequestToSend)}            //There is a packet to sen
9     {
10        if{Sn-S;E  >= Sw)               I/If window is full
11              Sleep{);
12        GetData{) ;
13        MakeFrame (Sn) ;
14        StoreFrame{Sn);
15        SendFrame (Sn) ;
16        Sn = Sn + 1;
17        StartTimer{Sn);
18    }
19
20    if(Event{ArrivalNotification»   IIACK arrives
21    {
22        Receive{frame);               I/Receive ACK or NAK
23        if{corrupted{frame»
24              Sleep ();
25        if (FrameType == NAK)
26           if (nakNo between Sf and So)
27           {
28            resend{nakNo);
29            StartTimer{nakNo);
30           }
31        if (FrameType == ACK)
32           if (ackNo between Sf and So)
33           {
34             while{sf < ackNo)
35             {
36              Purge (sf);
37              stopTimer(Sf) ;
38              Sf = Sf + 1;
39             }
40           }
41    }
42
43    if(Event{TimeOut{t»)            liThe timer expires
44    {
45     StartTimer{t);
46     SendFrame{t);
47    }
48  }
```

Algorithm *Receiver-site Selective Repeat algorithm*

```
1  Rn = 0;
2  NakSent = false;
3  AckNeeded = false;
4  Repeat(for all slots)
5       Marked(slot) = false;
6
7  !while (true)                                   IIRepeat forever
8  {
9    WaitForEvent()i
10
11   if{Event{ArrivalNotification»                 jData frame arrives
12   {
13       Receive(Frame);
14       if(corrupted(Frame»&& (NOT NakSent)
15       {
16        SendNAK(Rn);
17        NakSent = true;
18        Sleep{};
19       }
20       if(seqNo <> Rn)&& (NOT NakSent)
21       {
22        SendNAK(Rn);
23        NakSent = true;
24        if {(seqNo in window)&&(IMarked(seqNo»
25        {
26         StoreFrame{seqNo)
27         Marked(seqNo)= true;
28         while(Marked(Rn)
29         {
30          DeliverData(Rn);
31          Purge(Rn);
32          Rn = Rn + 1;
33          AckNeeded = true;
34         }
35          if(AckNeeded);
36          {
37          SendAck(Rn);
38          AckNeeded = false;
39          NakSent = false;
40          }
41        }
42       }
43   }
44 }
```

Implementation-

//-----AT CLIENT SIDE------

```
import java.lang.System;
import java.net.*;
import java.io.*;
public class Client {
    static Socket connection;
    public static void main(String a[]) throws SocketException {
        try {
            int v[] = new int[9];
            //int g[] = new int[8];
            int n = 0;
            InetAddress addr = InetAddress.getByName("Localhost");
            System.out.println(addr);
            connection = new Socket(addr, 8011);
            DataOutputStream out = new DataOutputStream(
                    connection.getOutputStream());
            DataInputStream in = new DataInputStream(
                    connection.getInputStream());
            int p = in.read();
            System.out.println("No of frame is:" + p);
               for (int i = 0; i < p; i++) {
               v[i] = in.read();
               System.out.println(v[i]);
               //g[i] = v[i];
            }
            v[5] = -1;
            for (int i = 0; i < p; i++)
             {
```

```
            System.out.println("Received frame is: " + v[i]);


            }
          for (int i = 0; i < p; i++)
            if (v[i] == -1) {
          System.out.println("Request to retransmit packet no "
                    + (i+1) + " again!!");
              n = i;
              out.write(n);
              out.flush();
            }
            System.out.println();
                v[n] = in.read();
           System.out.println("Received frame is: " + v[n]);
          System.out.println("quiting");
        } catch (Exception e) {
           System.out.println(e);
        }
        }
    }
```

/* OUTPUT

iotlab@iotlab-Veriton-M200-B360:~/sliding window$ javac Client.java

iotlab@iotlab-Veriton-M200-B360:~/sliding window$ java Client

Localhost/127.0.0.1

No of frame is:9

30

40

50

60

70

80

90

100

110

Received frame is: 30

Received frame is: 40

Received frame is: 50

Received frame is: 60

Received frame is: 70

Received frame is: -1

Received frame is: 90

Received frame is: 100

Received frame is: 110

Request to retransmit packet no 6 again!!

Received frame is: 80

quiting

iotlab@iotlab-Veriton-M200-B360:~/sliding window$

*/

//-----AT SERVER SIDE-----

```
import java.io.DataInputStream;
import java.io.DataOutputStream;
import java.io.IOException;
import java.net.ServerSocket;
import java.net.Socket;
import java.net.SocketException;
  public class Server {
   static ServerSocket Serversocket;
```

```
static DataInputStream dis;
static DataOutputStream dos;
  public static void main(String[] args) throws SocketException {

try {
   int a[] = { 30, 40, 50, 60, 70, 80, 90, 100, 110 };
   Serversocket = new ServerSocket(8011);
   System.out.println("waiting for connection");
   Socket client = Serversocket.accept();
   dis = new DataInputStream(client.getInputStream());
   dos = new DataOutputStream(client.getOutputStream());
   System.out.println("The number of packets sent is:" + a.length);
   int y = a.length;
   dos.write(y);
   dos.flush();
  for (int i = 0; i < a.length; i++) {
      dos.write(a[i]);
      dos.flush();
   }
     int k = dis.read();
     dos.write(a[k]);
   dos.flush();
     } catch (IOException e) {
      System.out.println(e);
   } finally {
      try {
         dis.close();
         dos.close();
      } catch (IOException e) {
         // TODO Auto-generated catch block
```

```
                e.printStackTrace();
             }
            }
        }
    }
/*OUTPUT
iotlab@iotlab-Veriton-M200-B360:~/sliding window$ javac Server.java
iotlab@iotlab-Veriton-M200-B360:~/sliding window$ java Server
waiting for connection
The number of packets sent is:9
iotlab@iotlab-Veriton-M200-B360:~/sliding window$
*/
```

Conclusion: Implemented Go Back N and Selective Repeat Sliding Window protocol.

ASSIGNMENT NO-6

Tittle: Write a program to demonstrate Sub-netting and find subnet masks.

Objectives: To understand subnetting concepts and also find subnet mask of network.

Problem Statement: Write a program to demonstrate subnetting and find subnet mask.

Outcome: Demonstrate subnetting concepts with examples.

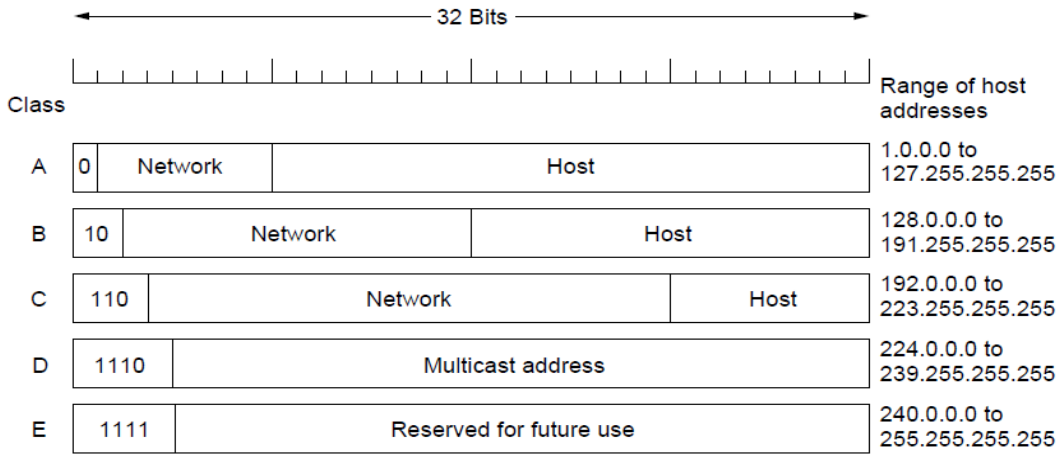
Software Requirements : Jdk and python

Hardware Requirements : Open source linux operating system.

THEORY:

What is IP address?

An Internet Protocol address (IP address) is a numerical label assigned to each device (e.g., computer, printer) participating in a computer network that uses the Internet Protocol for communication. An IP address serves two principal functions: host or network interface identification and location addressing. IP address is a 32 bit number. It is universally unique



An IPv4 address has two components, the network part and the host part. IPv4 address is a combination of IPv4 address and Subnet mask. The purpose of subnet mask is to identify which part of anIPv4 address is the network part and which part is the host part. Subnet mask is also a 32 bit number where allthe bits of the network part are represented as "1" and all the bits of the host part are represented as "0".

If we take an example for a Class C network , 192.168.10.0, the address part and the subnet mask can be represented as below

Component	Binary	Decimal
Address Part	11000000.10101000.00001010.00000000	192.168.10.0
SN Mask	11111111.11111111.11111111.00000000	255.255.255.0

For a Class C IPv4 address , the first three octets are used to represent the Network part and the last octet is used to represent the host part. From the above table, we can see all "1" in the network part and all "0" in the host part. When this subnet mask is converted to a decimals, it will become 255.2555.255.0. The default subnetmask for a Class C network is 255.255.255.0, Class B network is 255.255.0.0 and Class A network is 255.0.0.0.

What is a Network Address?

A network address is used to identify the subnet that a host may be placed on and is used to represent that network. We can find the network address by assigning all bits in the host part as 0.

Class C Subnetting

Subnetting is done by taking the bit/s from host part and adding it to the network part. Consider the same Class C example given above. Remember, the first three octets of a Class C network is used to represent the network and the last octet is used to represent the host. The default format for a Class CIPv4 address is **Network.Network.Network.Host.**

To make things easy, you may remember this.

- If all the bits in the host part are "0", that represents the network id.
- If all the bits in the host part are "0" except the last bit, it is the first usable IPv4 address.
- If all the bits in the host part are "1" except the last bit, it is the last usable IPv4 address.
- If all the bits in the host part are "1", that represents the directed broadcast address.
- All the IPv4 addresses between the first and last IPv4 addresses(including the first and last) canbe used to configure the devices.

Class C

One Bit Subnetting

Consider the network shown above. If we include one bit from the host part to the network part,The subnet mask is changed into 255.255.255.128. The single bit can have two values in last octet, either 0 or 1.

11000000.10101000.00001010.0|0000000

11111111.11111111.11111111.1 | 0000000

That means, we can get two subnets if we do a single bit subnetting.

The network 192.168.10.0 is divided into two networks, each network has 128 total IPv4 addresses and 126 usableIPv4 addresses (two IPv4 addresses are used in each subnet to represent the network address and the directed broadcast address). The subnet mask for one bit subnetting is 255.255.255.128.

Class C - Two Bit Subnetting (expected solution of problem statement)

If we include two bits from the host part to the network part, the subnet mask is changed into 255.255.255.192.The two bits added to network part can have four possible values in last octet and that are 00, 01, 10 and 11. That means, we can get four networks if we do a two bit subnetting (as expected in the problem statement).

11000000.10101000.00001010.00 | 000000

11111111.11111111.11111111.11 | 000000

SubNet No	Description	Binaries	Decimal
1	Network Address	11000000.10101000.00001010.00000000	192.168.10.0
	First IPv4 address	11000000.10101000.00001010.00000001	192.168.10.1
	Last IPv4 address	11000000.10101000.00001010.00111110	192.168.10.62
	Broadcast Address	11000000.10101000.00001010.00111111	192.168.10.63
2	Network Address	11000000.10101000.00001010.01000000	192.168.10.64
	First IPv4 address	11000000.10101000.00001010.01000001	192.168.10.65
	Last IPv4 address	11000000.10101000.00001010.01111110	192.168.10.126
	Broadcast Address	11000000.10101000.00001010.01111111	192.168.10.127
3	Network Address	11000000.10101000.00001010.10000000	192.168.10.128
	First IPv4 address	11000000.10101000.00001010.10000001	192.168.10.129
	Last IPv4 address	11000000.10101000.00001010.10111110	192.168.10.190
	Broadcast Address	11000000.10101000.00001010.10111111	192.168.10.191
4	Network Address	11000000.10101000.00001010.11000000	192.168.10.192
	First IPv4 address	11000000.10101000.00001010.11000001	192.168.10.193
	Last IPv4 address	11000000.10101000.00001010.11111110	192.168.10.254
	Broadcast Address	11000000.10101000.00001010.11111111	192.168.10.255

usableIPv4 addresses (two IPv4 addresses are used in each subnet to represent the network address and the directed broadcast address). The subnet mask for two bit subnetting is 255.255.255.192.

Class C Subnetting can be summerized as below:

Subnet Bits	Subnet Mask	CIDR	Total Subnets	Usable IPs/Subnet
0	255.255.255.0	/24	1	254
1	255.255.255.128	/25	2	126
2	255.255.255.192	/26	4	62
3	255.255.255.224	/27	8	30
4	255.255.255.240	/28	16	14
5	255.255.255.248	/29	32	6
6	255.255.255.252	/30	64	2

Implementation-

```
import java.io.*;
import java.net.InetAddress;
public class Subnet1 {
    public static void main(String[] args) throws IOException {
```

```
System.out.println("ENTER IP:");
BufferedReader br = new BufferedReader(new InputStreamReader(System.in));
String ip = br.readLine();
String checkclass = ip.substring(0, 3);
int cc = Integer.parseInt(checkclass);
String mask = null;
if(cc>0)
{
   if(cc<=127)
   {
      mask = "255.0.0.0";
System.out.println("Class A IP Address");
System.out.println("SUBNET MASK:\n"+mask);
   }
   if(cc>=128 && cc<=191)
   {
      mask = "255.255.0.0";
System.out.println("Class B IP Address");
System.out.println("SUBNET MASK:\n"+mask);
   }
   if(cc>=192 && cc<=223)
   {
      mask = "255.255.255.0";
System.out.println("Class C IP Address");
System.out.println("SUBNET MASK:\n"+mask);
   }
if(cc>=224 && cc<=239)
   {
mask = "255.0.0.0";
      System.out.println("Class D IP Address Used for multicasting");
```

```
        }
        if(cc>=240 && cc<=254)
        {
      mask = "255.0.0.0";
            System.out.println("Class E IP Address Experimental Use");
        }
      }
      String networkAddr="";
   String lastAddr="";
      String[] ipAddrParts=ip.split("\\.");
      String[] maskParts=mask.split("\\.");
      for(int i=0;i<4;i++){
      int x=Integer.parseInt(ipAddrParts[i]);
      int y=Integer.parseInt(maskParts[i]);
      int z=x&y;
      networkAddr+=z+".";
   int w=z|(y^255);
   lastAddr+=w+".";
      }
   System.out.println("First IP of block: "+networkAddr);
   System.out.println("Last IP of block: "+lastAddr);
  }
}
/*OUTPUT
iotlab@iotlab-Veriton-M200-B360:~$ javac Subnet1.java
iotlab@iotlab-Veriton-M200-B360:~$ java Subnet1
ENTER IP:
226.35.65.23
Class D IP Address Used for multicasting
First IP of block: 226.0.0.0.
```

Last IP of block: 226.255.255.255.

iotlab@iotlab-Veriton-M200-B360:~$ java Subnet1

ENTER IP:

192.168.100.5

Class C IP Address

SUBNET MASK:

255.255.255.0

First IP of block: 192.168.100.0.

Last IP of block: 192.168.100.255.

iotlab@iotlab-Veriton-M200-B360:~$

Conclusion- Thus we have implemented subnetting program .

ASSIGNMENT No 6

Title: Write a program to implement link state /Distance vector routing protocol to find suitable path for transmission.

Objectives:

- Implement **Link State Routing (LSR)** and **Distance Vector Routing (DVR)** protocols in a network simulation.
- The program should compute the shortest path for packet transmission between nodes in a network based on their connectivity and the cost of links.

Problem Statement: To Implement link state /Distance vector routing protocol to find suitable path for transmission.

Outcomes: Understanding of Routing Protocols**:** Gain insight into how LSR and DVR work, the differences between them, and how routing information is propagated through a network.

Tools Required:

Software: cpp

Theory:

Routing algorithm is a part of network layer software which is responsible for deciding which output line an incoming packet should be transmitted on. If the subnet uses datagram internally, this decision must be made anew for every arriving data packet since the best route may have changed since last time. If the subnet uses virtual circuits internally, routing decisions are made only when a new established route is being set up. The latter case is sometimes called session routing, because a rout remains in force for an entire user session (e.g., login session at a terminal or a file).

Routing algorithms can be grouped into two major classes: adaptive and non-adaptive. Nonadaptive algorithms do not base their routing decisions on measurement or estimates of current traffic and topology. Instead, the choice of route to use to get from I to J (for all I and J) is compute in advance, offline, and downloaded to the routers when the network ids booted. This procedure is sometime called static routing.

Adaptive algorithms, in contrast, change their routing decisions to reflect changes in the topology, and usually the traffic as well. Adaptive algorithms differ in where they get information (e.g., locally, from adjacent routers, or from all routers), when they change the routes, and what metric is used for optimization (e.g., distance, number of hops, or estimated transit time).

Two algorithms in particular, distance vector routing and link state routing are the most popular. Distance vector routing algorithms operate by having each router maintain a table (i.e., vector) giving the best known distance to each destination and which line to get there. These tables are updated by exchanging information with the neighbour's. The distance vector routing algorithm is sometimes called by other names, including the distributed Bellman-Ford routing algorithm and the Ford-Fulkerson algorithm, after the researchers who developed it (Bellman, 1957; and Ford and Fulkerson, 1962). It was the original ARPANET routing algorithm and was also used in the Internet under the RIP and in early versions of DEC net and Novell'Ps IPX. AppleTalk and Cisco routers use improved distance vector protocols.

In distance vector routing, each router maintains a routing table indexed by, and containing one entry for, each router in subnet. This entry contains two parts: the preferred outgoing line to use for that destination, and an estimate of the time or distance to that destination. The metric used might be number of hops, time delay in milliseconds, total number of packets queued along the path, or something similar.

The router is assumed to know the "distance" to each of its neighbour. If the metric is hops, the distance is just one hop. If the metric is queue length, the router simply examines each queue. If the metric is delay, the router can measure it directly with special ECHO packets that the receiver just time stamps and sends back as fast as possible.

Algorithm:

1. Start
2. By convention, the distance of the node to itself is assigned to zero and when a node is unreachable the distance is accepted as 999.
3. Accept the input distance matrix from the user (*dm[][]*) that represents the distance between each node in the network.
4. Store the distance between nodes in a suitable varible.
5. Calculate the minimum distance between two nodes by iterating.

- If the distance between two nodes is larger than the calculated alternate available path, replace the existing distance with the calculated distaance.

6. Print the shortest path calculated.
7. Stop.

Link state routing

In link state routing, each router shares its knowledge of its neighborhood with every other router in the internet work.

(i) Knowledge about Neighborhood: Instead of sending its entire routing table a router sends info about its neighborhood only.

(ii) To all Routers: each router sends this information to every other router on the internet work not just to its neighbor .It does so by a process called flooding.

(iii) Information sharing when there is a change: Each router sends out information about the neighbors when there is change.

PROCEDURE: The Dijkstra algorithm follows four steps to discover what is called the shortest path tree(routing table) for each router:The algorithm begins to build the tree by identifying its roots. The root router's trees the router itself. The algorithm then attaches all nodes that can be reached from the root. The algorithm compares the tree's temporary arcs and identifies the arc with the lowest cumulative cost. This arc and the node to which it connects are now a permanent part of the shortest path tree. The algorithm examines the database and identifies every node that can be reached from its chosen node. These nodes and their arcs are added temporarily to the tree. The last two steps are repeated until every node in the network has become a permanent part of the tree.

Implementation-

```
#include <iostream>
using namespace std;
struct node
 {
```

```
        int dist[20];
        int from[20];
}
 route[10];
int main()
{
int dm[20][20], no;
cout << "Enter no of nodes." << endl;
cin >> no;
cout << "Enter the distance matrix:" << endl;
for (int i = 0; i < no; i++)
{
for (int j = 0; j < no; j++)
{
cin >> dm[i][j];
/*  Set distance from i to i as 0 */
dm[i][i] = 0;
route[i].dist[j] = dm[i][j];
route[i].from[j] = j;
}
}
int flag;
do {
flag = 0;
for (int i = 0; i < no; i++) {
for (int j = 0; j < no; j++) {
for (int k = 0; k < no; k++) {
if ((route[i].dist[j]) > (route[i].dist[k] + route[k].dist[j])) {
route[i].dist[j] = route[i].dist[k] + route[k].dist[j];
route[i].from[j] = k;
```

```
flag = 1;
}
}
}
}
} while (flag);
for (int i = 0; i < no; i++) {
cout << "Router info for router: " << i + 1 << endl;
cout << "Dest\tNext Hop\tDist" << endl;
for (int j = 0; j < no; j++)
printf("%d\t%d\t\t%d\n", j+1, route[i].from[j]+1, route[i].dist[j]);
}
return 0;
}
```

Output-

```
Enter no of nodes.
3
Enter the distance matrix:
0 2 99
1 0 99
3 2 0
Router info for router: 1
Dest    Next Hop        Dist
1       1               0
2       2               2
3       3               99
Router info for router: 2
Dest    Next Hop        Dist
1       1               1
2       2               0
3       3               99
Router info for router: 3
Dest    Next Hop        Dist
1       1               3
2       2               2
3       3               0
```

Conclusion: Thus we implemented Distance vector algorithm for finding suitable path for transmission.

ASSIGNMENT NO- 8

Title: Use packet tracer tool for configuration of 3Router Network using one of the following protocol RIP/OSPF/BGP

Objectives:

- Cable a network according to the Topology Diagram.
- Erase the startup configuration and reload a router to the default state.
- Perform basic configuration tasks on a router.
- Configure and activate interfaces.
- Configure RIP/OSPF/BGP routing on all routers..
- Document the RIP/OSPF/BGP configuration.

Problem Statement: To setup network, configure and verify RIP/ OSPF/BGP .

Outcomes: Perform basic configuration tasks on a router.

Tools Required:

Software: Packet Tracer Tool

Theory:

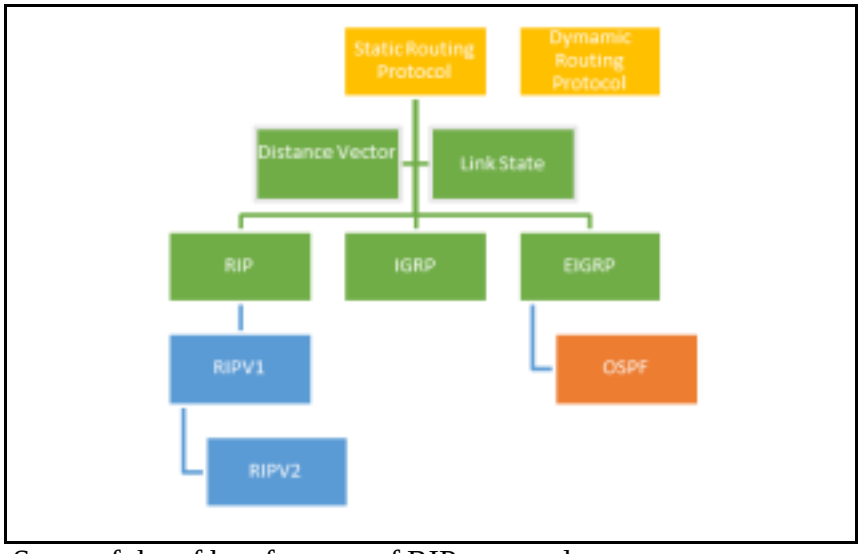
Introduction:-

ROUTING PROTOCOLS: RIP [ROUTING INFORMATION PROTOCOL]

RIP is an open standard routing protocol. It is a distance vectored routing protocols. It is a class-full routing protocol where updates are exchanged through broadcast. The routing table is exchanged every 30 seconds among the routers in the inter-network. The RIP protocol uses hop count as the metric to find the shortest path but the maximum allowable hop count is 15 by default. The RIP protocols is used only for a small network and is ineffective for a large network. The Administrative Distance of RIP is 120.

Routing Protocols are the set of defined rules used by the routers to communicate between source & destination. They do not move the information to the source to a destination, but only update the routing table that contains the information. Network Router protocols help you to specify the way routers communicate with each other. It allows the network to select routes between any two nodes on a computer network.

Types of Routing Protocols :



Some of the of key features of RIP protocol are:

- It supports maximum 15 hops in a path.
- It uses hops count metric to calculate the best path from a source to a destination network.
- It sends routing updates (entire routing table) after every 30 seconds and when the network changes.
- It uses UDP broadcast packets to exchange routing information.
- The Administrative Distance (AD) value of the RIP protocol is 120.
- It has two versions: RIPv1 and RIPv2.

RIP TIMERS To manage the routing performance, RIP uses four different kinds of timers:

· Update timer: It is the time interval after which a router sends it's a copy of the routing table as update to the neighbor routers. The update timer is 30 sec by default.

· Invalid time: It is the time interval after which a router understands that the path to a network is invalid or becomes invalid. The invalid timer is 180 sec by default.

· Hold-down timer: It specifies the amount of time for which the information about the poorer routes are ignored. The hold-down timer is 180sec by default.

· Flush timer: It is the time before the invalid route is purged from the routing table. The flush timer is 240 sec by default.

Dis-advantages of RIP

· It uses more bandwidth as updates are exchanged every 30 seconds where each update contains the complete routing table of the router.

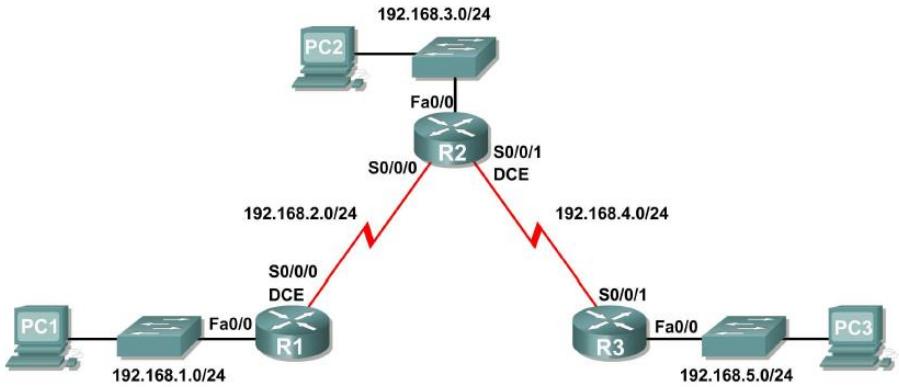
· It does not uses bandwidth as the metric for calculation of the shortest path.

· RIP has a very slow convergence.

· RIP implementation can lead to routing loops in the network.

· RIP is only applicable to small network and is inefficient for larger networks.

Topology Diagram



Scenario A: Running RIPv1 on Classful Networks

Addressing Table

Device	Interface	IP Address	Subnet Mask	Default Gateway
R1	Fa0/0	192.168.1.1	255.255.255.0	N/A
	S0/0/0	192.168.2.1	255.255.255.0	N/A
R2	Fa0/0	192.168.3.1	255.255.255.0	N/A
	S0/0/0	192.168.2.2	255.255.255.0	N/A
	S0/0/1	192.168.4.2	255.255.255.0	N/A
R3	Fa0/0	192.168.5.1	255.255.255.0	N/A
	S0/0/1	192.168.4.1	255.255.255.0	N/A
PC1	NIC	192.168.1.10	255.255.255.0	192.168.1.1
PC2	NIC	192.168.3.10	255.255.255.0	192.168.3.1
PC3	NIC	192.168.5.10	255.255.255.0	192.168.5.1

Task 1: Prepare the Network.

Step 1: Cable a network that is similar to the one in the Topology Diagram.

You can use any current router in your lab as long as it has the required interfaces shown in the topology.

Step 2: Clear any existing configurations on the routers.

Task 2: Perform Basic Router Configurations.

Perform basic configuration of the R1, R2, and R3 routers according to the following guidelines:

1. Configure the router hostname.
2. Disable DNS lookup.
3. Configure an EXEC mode password.
4. Configure a message-of-the-day banner.
5. Configure a password for console connections.
6. Configure a password for VTY connections.

Task 3: Configure and Activate Serial and Ethernet Addresses.

Step 1: Configure interfaces on R1, R2, and R3.

Configure the interfaces on the R1, R2, and R3 routers with the IP addresses from the table under the Topology Diagram.

Step 2: Verify IP addressing and interfaces.

Use the **show ip interface brief** command to verify that the IP addressing is correct and that the interfaces are active.When you have finished, be sure to save the running configuration to the NVRAM of the router.

Step 3: Configure Ethernet interfaces of PC1, PC2, and PC3.

Configure the Ethernet interfaces of PC1, PC2, and PC3 with the IP addresses and default gateways from the table under the Topology Diagram.

Step 4: Test the PC configuration by pinging the default gateway from the PC.

Task 4: Configure RIP.

Step 1: Enable dynamic routing.

To enable a dynamic routing protocol, enter global configuration mode and use the **router** command. Enter **router ?**at the global configuration prompt to a see a list of available routing protocols on your router.

To enable RIP, enter the command **router rip** in global configuration mode.

R1(config)#**router rip**

R1(config-router)#

Step 2: Enter classful network addresses.

Once you are in routing configuration mode, enter the classful network address for each directly connected network, using the **network** command.

R1(config-router)#**network 192.168.1.0**

R1(config-router)#**network 192.168.2.0**

R1(config-router)#

The **network** command:

• Enables RIP on all interfaces that belong to this network. These interfaces will now both send and receive RIP updates.

• Advertises this network in RIP routing updates sent to other routers every 30 seconds.

When you are finished with the RIP configuration, return to privileged EXEC mode and save the current configuration to NVRAM.

R1(config-router)#**end**

%SYS-5-CONFIG_I: Configured from console by console

R1#**copy run start**

Step 3: Configure RIP on the R2 router using the router rip and network commands.

R2(config)#**router rip**

R2(config-router)#**network 192.168.2.0**

R2(config-router)#**network 192.168.3.0**

R2(config-router)#**network 192.168.4.0**

R2(config-router)#**end**

%SYS-5-CONFIG_I: Configured from console by console

R2#**copy run start**

When you are finished with the RIP configuration, return to privileged EXEC mode and save the current configuration to NVRAM.

Step 4: Configure RIP on the R3 router using the router rip and network commands.

R3(config)#**router rip**

R3(config-router)#**network 192.168.4.0**

R3(config-router)#**network 192.168.5.0**

R3(config-router)#**end**

%SYS-5-CONFIG_I: Configured from console by console

R3# **copy run start**

When you are finished with the RIP configuration, return to privileged EXEC mode and save the current configuration to NVRAM.

Task 5: Verify RIP Routing.

Step 1: Use the show ip route command to verify that each router has all of the networks in the topology entered in the routing table.

Routes learned through RIP are coded with an **R** in the routing table. If the tables are not converged as shown here, troubleshoot your configuration. Did you verify that the configured interfaces are active? Did you configure RIP correctly? Return to Task 3 and Task 4 to review the steps necessary to achieve convergence.

R1#**show ip route**

Codes: C - connected, S - static, I - IGRP, R - RIP, M - mobile, B - BGP

D - EIGRP, EX - EIGRP external, O - OSPF, IA - OSPF inter area

N1 - OSPF NSSA external type 1, N2 - OSPF NSSA external type 2

E1 - OSPF external type 1, E2 - OSPF external type 2, E - EGP

i - IS-IS, L1 - IS-IS level-1, L2 - IS-IS level-2, ia - IS-IS inter area

* - candidate default, U - per-user static route, o - ODR

P - periodic downloaded static route

Gateway of last resort is not set

C 192.168.1.0/24 is directly connected, FastEthernet0/0

C 192.168.2.0/24 is directly connected, Serial0/0/0

R 192.168.3.0/24 [120/1] via 192.168.2.2, 00:00:04, Serial0/0/0

R 192.168.4.0/24 [120/1] via 192.168.2.2, 00:00:04, Serial0/0/0

R 192.168.5.0/24 [120/2] via 192.168.2.2, 00:00:04, Serial0/0/0

R1#

R2#**show ip route**

<Output omitted>

R 192.168.1.0/24 [120/1] via 192.168.2.1, 00:00:22, Serial0/0/0

C 192.168.2.0/24 is directly connected, Serial0/0/0

C 192.168.3.0/24 is directly connected, FastEthernet0/0

C 192.168.4.0/24 is directly connected, Serial0/0/1

R 192.168.5.0/24 [120/1] via 192.168.4.1, 00:00:23, Serial0/0/1

R2#

R3#**show ip route**

<Output omitted>

R 192.168.1.0/24 [120/2] via 192.168.4.2, 00:00:18, Serial0/0/1

R 192.168.2.0/24 [120/1] via 192.168.4.2, 00:00:18, Serial0/0/1

R 192.168.3.0/24 [120/1] via 192.168.4.2, 00:00:18, Serial0/0/1

C 192.168.4.0/24 is directly connected, Serial0/0/1

C 192.168.5.0/24 is directly connected, FastEthernet0/0

R3#

Step 2: Use the show ip protocols command to view information about the routing processes.

The **show ip protocols** command can be used to view information about the routing processes that are occurring on the router. This output can be used to verify most RIP parameters to confirm that:

RIP routing is configured

The correct interfaces send and receive RIP updates

The router advertises the correct networks

RIP neighbors are sending updates

R1#**show ip protocols**

Routing Protocol is "rip"

Sending updates every 30 seconds, next due in 16 seconds

Invalid after 180 seconds, hold down 180, flushed after 240

Outgoing update filter list for all interfaces is not set

Incoming update filter list for all interfaces is not set

Redistributing: rip

Default version control: send version 1, receive any version

Interface Send Recv Triggered RIP Key-chain

FastEthernet0/0 1 2 1

Serial0/0/0 1 2 1

Automatic network summarization is in effect

Maximum path: 4

Routing for Networks:

192.168.1.0

192.168.2.0

Passive Interface(s):

Routing Information Sources:

Gateway Distance Last Update

192.168.2.2 120

Distance: (default is 120)

R1#

R1 is indeed configured with RIP. R1 is sending and receiving RIP updates on FastEthernet0/0 and Serial0/0/0. R1 is advertising networks 192.168.1.0 and 192.168.2.0. R1 has one routing information source. R2 is sending R1 updates.

Step 3: Use the debug ip rip command to view the RIP messages being sent and received.

Rip updates are sent every 30 seconds so you may have to wait for debug information to be displayed.

R1#**debug ip rip**

R1#RIP: received v1 update from 192.168.2.2 on Serial0/0/0

192.168.3.0 in 1 hops

192.168.4.0 in 1 hops

192.168.5.0 in 2 hops

RIP: sending v1 update to 255.255.255.255 via FastEthernet0/0 (192.168.1.1)

RIP: build update entries

network 192.168.2.0 metric 1

network 192.168.3.0 metric 2

network 192.168.4.0 metric 2

network 192.168.5.0 metric 3

RIP: sending v1 update to 255.255.255.255 via Serial0/0/0 (192.168.2.1)

RIP: build update entries

network 192.168.1.0 metric 1

The debug output shows that R1 receives an update from R2. Notice how this update includes all the networks that R1 does not already have in its routing table. Because the FastEthernet0/0 interface belongs to the 192.168.1.0 network configured under RIP, R1 builds an update to send out that interface.

The update includes all networks known to R1 except the network of the interface. Finally, R1 builds an update to send to R2. Because of split horizon, R1 only includes the 192.168.1.0 network in the update.

Step 4: Discontinue the debug output with the undebug all command.

R1#**undebug all**

All possible debugging has been turned off

Routing Protocols : OSPF [Open Shortest Path First] :

The OSPF (Open Shortest Path First) protocol is one of a family of IP Routing protocols, and is an Interior Gateway Protocol (IGP) for the Internet, used to distribute IP routing information throughout a single Autonomous System (AS) in an IP network.

The OSPF protocol is a link-state routing protocol, which means that the routers exchange topology information with their nearest neighbors. The topology information is flooded throughout the AS, so that every router within the AS has a complete picture of the topology of the AS. This picture is then used to calculate end-to-end paths through the AS, normally using a variant of the Dijkstra algorithm. Therefore, in a link-state routing protocol, the next hop address to which data is forwarded is determined by choosing the best end-to-end path to the eventual destination.

The main advantage of a link state routing protocol like OSPF is that the complete knowledge of topology

allows routers to calculate routes that satisfy particular criteria. This can be useful for traffic engineering purposes, where routes can be constrained to meet particular quality of service requirements. The main disadvantage of a link state routing protocol is that it does not scale well as more routers are added to the routing domain. Increasing the number of routers increases the size and frequency of the topology updates, and also the length of time it takes to calculate end-to-end routes. This lack of scalability means that a link state routing protocol is unsuitable for routing across the Internet at large, which is the reason why IGPs only route traffic within a single AS.

Each OSPF router distributes information about its local state (usable interfaces and reachable neighbors, and the cost of using each interface) to other routers using a Link State Advertisement (LSA) message. Each router uses the received messages to build up an identical database that describes the topology of the AS.

From this database, each router calculates its own routing table using a Shortest Path First (SPF) or Dijkstra algorithm. This routing table contains all the destinations the routing protocol knows about, associated with a next hop IP address and outgoing interface.

● The protocol recalculates routes when network topology changes, using the Dijkstra algorithm, and minimises the routing protocol traffic that it generates. ● It provides support for multiple paths of equal cost.
● It provides a multi-level hierarchy (two-level for OSPF) called "area routing," so that information about the topology within a defined area of the AS is hidden from routers outside this area. This enables an additional level of routing protection and a reduction in routing protocol traffic.

● All protocol exchanges can be authenticated so that only trusted routers can join in the routing exchanges for the AS.

Routing Protocols : BFP [Border Gateway Protocol] :

Border Gateway Protocol (BGP) is used to Exchange routing information for the internet and is the protocol used between ISPs which are different ASes. The protocol can connect together any internetwork of an autonomous system using an arbitrary topology. The only requirement is that each AS has at least one router that is able to run BGP and that router connects to at least one another AS's BGP router. BGP's main function is to exchange network reach-ability information with other BGP systems. Border Gateway Protocol

constructs an autonomous systems' graph based on the information exchanged between BGP routers.

Characteristics of Border Gateway Protocol (BGP) :

●Inter-Autonomous System Configuration: The main role of BGP is to provide communication between two autonomous systems.

● BGP supports Next-Hop Paradigm.

● Coordination among multiple BGP speakers within the AS (Autonomous System).

● Path Information: BGP advertisement also includes path information, along with the reachable destination and next destination pair.

● Policy Support: BGP can implement policies that can be configured by the administrator. For example :- a router running BGP can be configured to distinguish between the routes that are known within the AS and those which are known from outside the AS.

● Runs Over TCP.

● BGP conserves network Bandwidth.

● BGP supports CIDR.

● BGP also supports Security.

Functionality of Border Gateway Protocol (BGP) :

BGP peers perform 3 functions, which are given below.

1. The first function consists of initial peer acquisition and authentication. Both the peers established a TCP connection and performed message exchange that guarantees both sides have agreed to communicate.
2. The second function mainly focuses on sending negative or positive reach-ability information.
3. The third function verifies that the peers and the network connection between them are functioning correctly.

FEATURES OF TOOL USED : CISCO PACKET TRACER TOOL :

Packet Tracer is a cross-platform visual simulation tool designed by Cisco Systems that allows users to create network topologies and imitate modern computer networks. The software allows users to simulate

the configuration of Cisco routers and switches using a simulated command line interface.

Cisco Packet Tracer as the name suggests, is a tool built by Cisco. This tool provides a network simulation to practice simple and complex networks. As Cisco believes, the best way to learn about networking is to do it. The main purpose of Cisco Packet Tracer is to help in learning the principles of networking with hands-on experience as well as develop Cisco technology specific skills. Since the protocols are implemented in software only method, this tool cannot replace the hardware Routers or Switches.

ALGORITHM :

Step 1 : START

Step 2 : Prepare the network using routers, switches and cables.

Step 3 : Clear any existing configurations on the routers and perform the basic router configurations such as providing IPv4 addresses and disabling DNS lookup.

Step 4 : Configure and Activate Serial and Ethernet Addresses of the PC's added to the network.

Step 5 : Configure the OSPF. To enable OSPF, enter the command **router OSPF** in global configuration mode.

R1(config)#**router OSPF**

R1(config-router)#

Step 6 : Enter the classful network address for each directly connected network, using the **network** command.

R1(config-router)#**network 192.168.1.0**

R1(config-router)#**network 192.168.2.0**

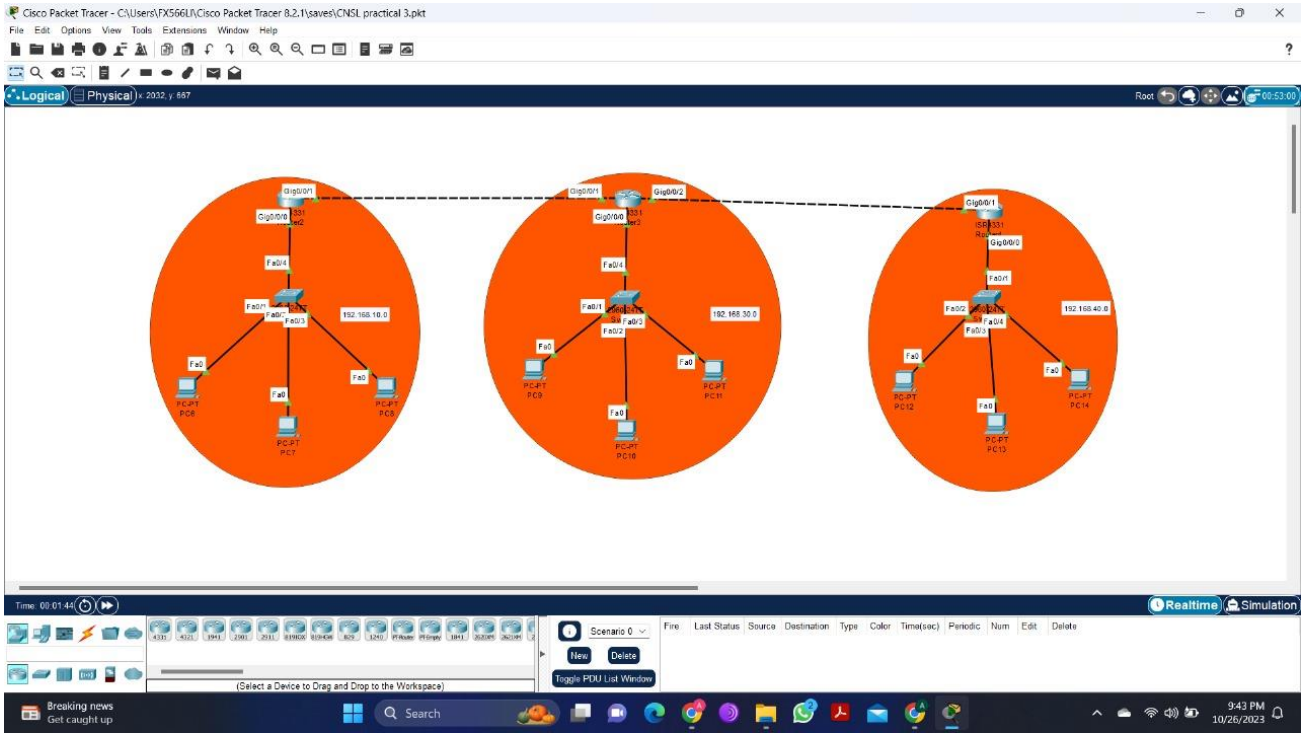
R1(config-router)#

Step 7 : Use the same procedure as in Step 6 for all the routers in the network.

Step 8 : Verify the OSPF routing using the packets.

Step 9 : STOP.

Implementation-



Conclusion: Thus we were able to setup network, configure and verify RIP/OSPF/BGP

ASSIGNMENT No-8

Title: Write a program using TCP socket for wired network for following

a. Say Hello to Each other (For all students)

b. File transfer (For all students)

c. Calculator (Arithmetic) (50% students)

d. Calculator (Trigonometry) (50% students)

Objectives: To perform socket programming with help of different systems to check the connection establishment for TCP

Problem Statement: To Implement link state /Distance vector routing protocol to find suitable path for transmission.

Outcomes: We can send message from one system to another , transfer file and perform the calculator operation.

Tools Required:

Software: cpp

THEORY:

TCP:

The Transmission Control Protocol provides a communication service at an intermediate level between an application program and the Internet Protocol. It provides host-to-host connectivity at the Transport Layer of the Internet model.

The client server model

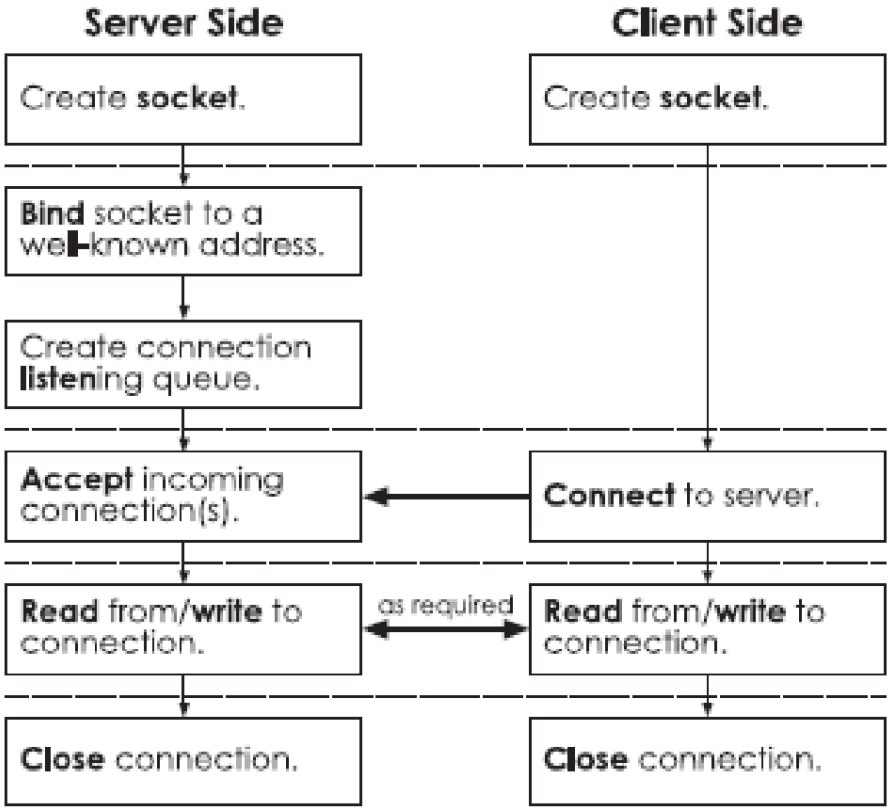
Most interprocess communication uses the client server model. These terms refer to the two processes which will be communicating with each other. One of the two processes, the client, connects to the other process, the server, typically to make a request for information. A socket is one end of an interprocess communication channel. The two processes each establish their own socket.

The steps involved in establishing a socket on the client side are as follows:

1. Create a socket with the socket( ) system call
2. Connect the socket to the address of the server using the connect( ) system call
3. Send and receive data. There are a number of ways to do this, but the simplest is to use the read ( ) and write ( ) system calls.

The steps involved in establishing a socket on the server side are as follows:

1. Create a socket with the socket ( ) system call
2. Bind the socket to an address using the bind ( ) system call. For a server socket on the Internet, an address consists of a port number on the host machine.
3. Listen for connections with the listen ( ) system call
4. Accept a connection with the accept ( ) system call. This call typically blocks until a client connects with the server.
5. Send and receive data



Sockets are one of the most important IPC mechanisms on UNIX. Originally introduced in 4.2BSD as a generalization of pipes, they later became the basis for the UNIX networking subsystem. Sockets arethe only IPC mechanism that allows communication between processes running on different machines. Essentially, it is an end-point of communication which may be bound to a name.But enough with the bland introduction. A socket is just a way to allow processes to talk to one another.

TCP and UDP

As you should remember from networking, on top of IP, there are two major transport protocols on topof which all other protocols are built: TCPand UDP These act as transportationmechanisms for other,higher-level, protocols.TCP is a reliable, connection-oriented protocol that transmits data as a stream of bytes. UDP, on theother hand, is an unreliable, connectionless protocol that sends data in chunks calleddatagrams

Creating a socket

Both the client and server need to create a socket before they can do anything. The client uses its socketto connect to a server whilst the server uses its socket for listening for new connections. Socket creation isdone using thesocket()call

socket()

Creates a new socket. Returns a file descriptor representing the socket end-point, or-1

, if an error occurs.

int socket(intaf, int type, int protocol);

af :is the address family to use with the socket. This could be either

AF_UNIXfor the UNIX address family(for local IPC), or

AF_INETfor the Internet address family (for network communication). TCP

requires that you useAF_INET,

type: is the socket type. This can beSOCK_STREAMfor connection-oriented, stream-based protocols likeTCP,SOCK_DGRAMfor datagram-based, connectionless protocols like UDP, orSOCK_RAWwhereyou want to use your own transportation protocol

protocol :is the transport protocol to use. It’s best to pass0, which lets the system decide.

Binding the socket to a well-known address

For clients to connect to a server, they need to know its address, but sockets are created without an addressof their own and so must be assigned to one that the client will know about. This is done bybindingasocket to awell-known addresson the system. For instance, web-servers are usually bound to port 80 asthis is the well-known port for the HTTP protocol

structsockaddr

This is a generic socket address structure provided to allow flexibility in passing socket addresses to functions.

```
struct sockaddr
{
    unsigned short sa_family;   // Address family tag.
    char           sa_data[14]; // Padding.
};
```

structsockaddr_in

structsockaddr_inis a specialised version ofstructsockaddrespecially for the

AF_INETaddress family.

```
struct sockaddr_in
{
    unsigned short sin_family;  // Set to AF_INET.
    unsigned short sin_port;    // Port number to bind to.
    struct in_addr sin_addr;    // IP address.
    char           sin_zero[8]; // Padding.
};
```

structin_addr

structin_addrrepresents an IP address. Why this structure exists and wasn't just incorporated directly intostructsockaddr_inis anybody's guess.Setting its one field, s_addr, toINADDR_ANYwill leave it up to the server to choose an appropriate host IP address, and this is usually the best thing to do.

```
struct in_addr
{
    unsigned long s_addr;    // IP address.
};
```

bind()

Binds a socket to a well-known address. This will return0if successful and-1

if not.

int bind(intfd, structsockaddr* addr, intlen);

fd: is the file descriptor of the socket to bind.

addr: points to the address to bind the socket to.

len: is the length of the address in bytes.

intBindSocket(intfd, unsigned short port)

```
{
    struct sockaddr_in addr;

    addr.sin_family      = AF_INET;
    addr.sin_addr.s_addr = INADDR_ANY;  // Let the host decide.
    addr.sin_port        = htons(port); // Port to bind to.

    return bind(fd, (struct sockaddr*) &addr, sizeof(addr));
}
```

listen()

Makes the socket listen for incoming connections, and sets up a connection queue for the socket.

int listen(intfd, intlen);

fd: is the file descriptor of the socket to put in passive mode.

Len: is the length of the connection queue **5**is the usual value used.

gethostbyname()

Allows you to discover a host's details (including its address) by specifying its name. This returns astructure specifying the host's details, orNULLif it fails.

structhostent* gethostbyname(char* name);

connect()

Connects a socket to a given server, putting the socket inactive mode. Returns0 if successful, else-1

int connect(intfd, structsockaddr* addr, intlen);

fd: is the file descriptor of the socket to connect.

Addr: points to the address of the server to connect to.

len: is the length in bytes of the address

Accepting incoming connections

After the connection queue has been created, the server can accept incoming connections from clientswishing to talk to it. To accept one, it must call

accept()

Accepts a single incoming connection.Returns a file descriptor corresponding to the new connection, or-1if an error occurs.

int accept(intfd, structsockaddr* addr, int* len);

fd: is the file descriptor of the socket listening for incoming connections.

addr: is an address structure to hold the address of the client making the connection. Pass NULLhereifyou don't care about getting this information.

len: is pointer to a variable holding the length in bytes of the address structure passed in addr. On return-ing, this will hold the actual length of the client address. If you passed in NULLtoaddr, passNULLin here too.

Reading and Writing data

After the client has connected to the server and the server has accepted the connection, they can start sending data back and forth. This is done with good old read() and write() . There are, however, some caveats attached to reading data, however. .

write()

Writes data to a file descriptor.

int write(intfd, void* buf, unsigned int n);

fd: is the file descriptor to write to.

buf: is a buffer containing data to write.

n: is the number of bytes from the buffer to write.

read()

Reads data from a file descriptor. It returns the number of bytes actually read, or-1 if an error occurs.

int read(intfd, void* buf, unsigned int n);

fd:is the file descriptor to read from.

buf:is a buffer to write the data read to.

n:is the size in bytes of the buffer.

Closing the socket

Once the client or server is finished with a socket, it should call**close()**to deallocate it.

close()

Closes a file descriptor.

int close(intfd);

fd: is the file descriptor to close.

No.	TCP	UDP
1	This Connection oriented protocol	This is connection-less protocol
2	The TCP connection is byte stream	The UDP connection is a message stream
3	It does not support multicasting and broadcasting	It supports broadcasting
4	It provides error control and flow control	The error control and flow control is not provided
5	TCP supports full duplex transmission	UDP does not support full duplex transmission
6	It is reliable service of data transmission	This is an unreliable service of data transmission
7	The TCP packet is called as segment	The UDP packet is called as user datagram.

Implementation :

Server.cpp:

```
#include <iostream>
#include <string>
#include <cstring>
#include <unistd.h>
#include <fcntl.h>
#include <sys/socket.h>
#include <netinet/in.h>
#include <muParser.h>   // Include muParser (C++ library)
#include <arpa/inet.h>
#include <fstream>
#define PORT 8080
#define BUFFER_SIZE 1024

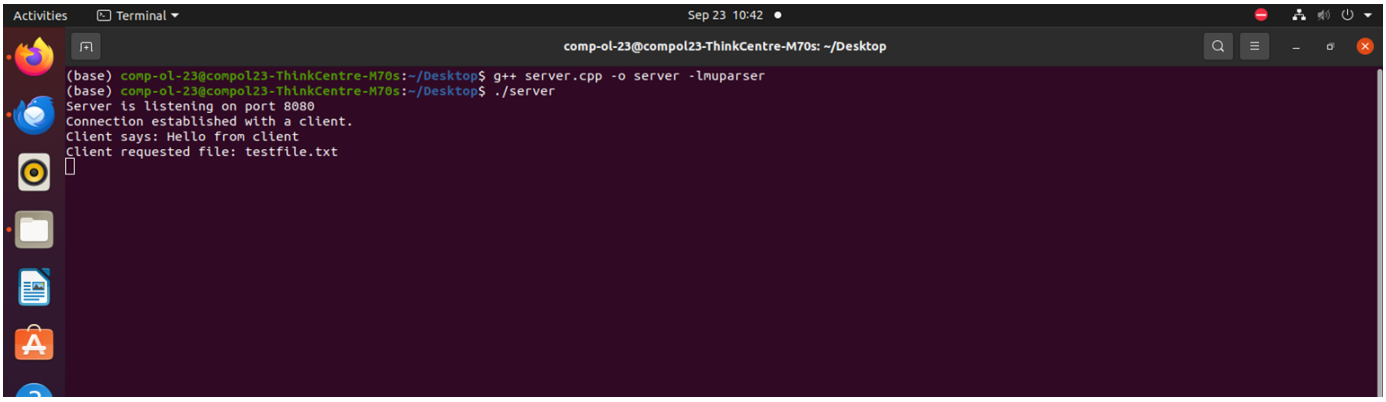

// Function to evaluate a mathematical expression using muParser
```

```
double evaluate_expression(const std::string &expression) {
    double result = 0;
    mu::Parser p;  // Create parser object
    try {
        p.SetExpr(expression); // Set the expression to be evaluated
        result = p.Eval();     // Evaluate the expression
    } catch (mu::Parser::exception_type &e) {
        std::cerr << "Error: " << e.GetMsg() << std::endl;
    }
    return result;
}
void handle_client(int new_socket) {
    char buffer[BUFFER_SIZE] = {0};
    // Say Hello to each other
    send(new_socket, "Hello from server", strlen("Hello from server"), 0);
    recv(new_socket, buffer, BUFFER_SIZE, 0);
    std::cout << "Client says: " << buffer << std::endl;
    // File transfer
    bzero(buffer, BUFFER_SIZE);
    recv(new_socket, buffer, BUFFER_SIZE, 0);
    std::string file_name(buffer);
    std::cout << "Client requested file: " << file_name << std::endl;
    std::ifstream file(file_name);
    if (!file.is_open()) {
        send(new_socket, "File not found.", strlen("File not found."), 0);
    } else {
        send(new_socket, "File found, sending now...", strlen("File found, sending now..."), 0);
        std::string file_content((std::istreambuf_iterator<char>(file)), std::istreambuf_iterator<char>());
        send(new_socket, file_content.c_str(), file_content.size(), 0);
    }
```

```
    // Calculator
    while (1) {
        bzero(buffer, BUFFER_SIZE);
        recv(new_socket, buffer, BUFFER_SIZE, 0);
        if (strcmp(buffer, "exit") == 0)
            break;
        // Evaluate the expression using muParser and send back the result
        double result = evaluate_expression(buffer);
        std::string result_str = "Result: " + std::to_string(result);
        send(new_socket, result_str.c_str(), result_str.length(), 0);
    }
    close(new_socket);
}
int main() {
    int server_fd, new_socket;
    struct sockaddr_in address;
    int addrlen = sizeof(address);
    // Creating socket file descriptor
    if ((server_fd = socket(AF_INET, SOCK_STREAM, 0)) == 0) {
        perror("socket failed");
        exit(EXIT_FAILURE);
    }
    // Forcefully attaching socket to the port
    address.sin_family = AF_INET;
    address.sin_addr.s_addr = INADDR_ANY;
    address.sin_port = htons(PORT);
    // Bind the socket to the network address and port
    if (bind(server_fd, (struct sockaddr *)&address, sizeof(address)) < 0) {
        perror("bind failed");
        exit(EXIT_FAILURE);
```

```
    }
    // Start listening for connections
    if (listen(server_fd, 3) < 0) {
        perror("listen failed");
        exit(EXIT_FAILURE);
    }
    std::cout << "Server is listening on port " << PORT << std::endl;
    while (1) {
        if ((new_socket = accept(server_fd, (struct sockaddr *)&address, (socklen_t *)&addrlen)) < 0) {
            perror("accept failed");
            exit(EXIT_FAILURE);
        }
        std::cout << "Connection established with a client." << std::endl;
        handle_client(new_socket);
    }
    return 0;
}
```

Output for server:



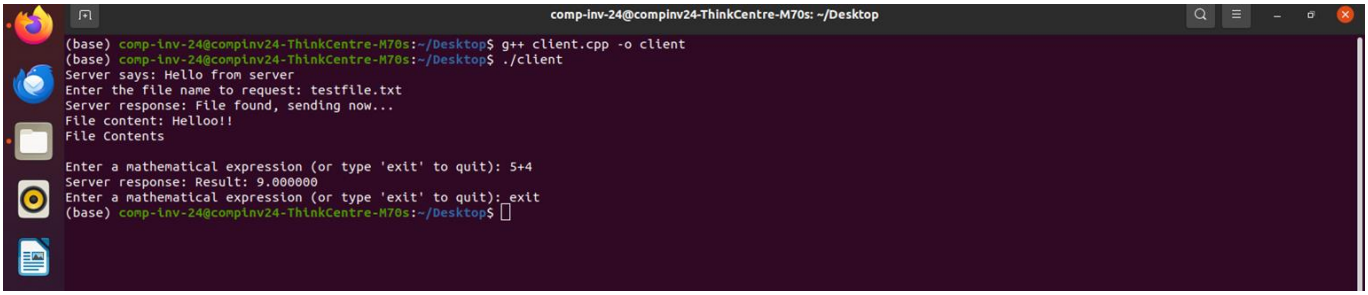
client.cpp:

```
#include <iostream>
`#include <string>
```

```
#include <cstring>
#include <unistd.h>
#include <arpa/inet.h>
#define PORT 8080
#define BUFFER_SIZE 1024
int main() {
    int sock = 0;
    struct sockaddr_in serv_addr;
    char buffer[BUFFER_SIZE] = {0};
    // Create socket
    if ((sock = socket(AF_INET, SOCK_STREAM, 0)) < 0) {
        std::cerr << "Socket creation error" << std::endl;
        return -1;
    }
    serv_addr.sin_family = AF_INET;
    serv_addr.sin_port = htons(PORT);
    // Convert IPv4 and IPv6 addresses from text to binary form
    if (inet_pton(AF_INET, "172.16.0.169", &serv_addr.sin_addr) <= 0) {
        std::cerr << "Invalid address / Address not supported" << std::endl;
        return -1;
    }
    // Connect to server
    if (connect(sock, (struct sockaddr *)&serv_addr, sizeof(serv_addr)) < 0) {
        std::cerr << "Connection failed" << std::endl;
        return -1;
    }
    // Say Hello to each other
    recv(sock, buffer, BUFFER_SIZE, 0);
    std::cout << "Server says: " << buffer << std::endl;
    send(sock, "Hello from client", strlen("Hello from client"), 0);
```

```
    // Request a file from server
    std::cout << "Enter the file name to request: ";
    std::string file_name;
    std::getline(std::cin, file_name);
    send(sock, file_name.c_str(), file_name.size(), 0);
    bzero(buffer, BUFFER_SIZE);
    recv(sock, buffer, BUFFER_SIZE, 0);
    std::cout << "Server response: " << buffer << std::endl;
   if (strcmp(buffer, "File found, sending now...") == 0) {
        bzero(buffer, BUFFER_SIZE);
        recv(sock, buffer, BUFFER_SIZE, 0);
        std::cout << "File content: " << buffer << std::endl;
    }
    // Calculator
    while (1) {
        std::cout << "Enter a mathematical expression (or type 'exit' to quit): ";
        std::string expression;
        std::getline(std::cin, expression);
        send(sock, expression.c_str(), expression.size(), 0);
        if (expression == "exit")
            break;
        bzero(buffer, BUFFER_SIZE);
        recv(sock, buffer, BUFFER_SIZE, 0);
        std::cout << "Server response: " << buffer << std::endl;
    }
    close(sock);
    return 0;
}
```

output for client:



testfile.txt:

Helloo!!

File Contents

Conclusion: Thus we have successfully implemented the socket programming for TCP using C.

ASSIGNMENT No 9

Title: Write a program using UDP Sockets to enable file transfer (Script, Text, Audio and Video one file each) between two machines.

Objectives: To perform socket programming with help of different systems to check the connection establishment for UDP

Problem Statement: To Implement connection establishment between two machines for UDP socket.

Outcomes: We can send message from one system to another , transfer file between two machines.

Tools Required:

Software: cpp

THEORY:

UDP:

UDP (User Datagram Protocol) is a communication protocol that offers a limited amount of service when messages are exchanged between computers in a network that uses the Internet Protocol (IP). UDP is an alternative to the Transmission Control Protocol (TCP) and, together with IP, is sometimes referred to as UDP/IP. Like the Transmission Control Protocol, UDP uses the Internet Protocol to actually get a data unit (called a datagram) from one computer to another. Unlike TCP, however, UDP does not provide the service of dividing a message into packets (datagrams) and reassembling it at the other end. Specifically, UDP doesn't provide sequencing of the packets that the data arrives in. This means that the application program that uses UDP must be able to make sure that the entire message has arrived and is in the right order. Network applications that want to save processing time because they have very small data units to exchange (and therefore very little message reassembling to do) may prefer UDP to TCP. The Trivial File Transfer Protocol (TFTP) uses UDP instead of TCP.

With a UDP socket a connection is NOT made, instead the sender just issues a message to its destination and hopes it gets there! The message uses a datagram of fixed length, often termed a record. Since there is no connection between client and server the client can send a datagram to one server and then immediately send a datagram to another server using the same socket UDP is a connectionless protocol.

Trivial File Transfer Protocol (TFTP) is a simple, lock-step, file transfer protocol which allows a client to get from or put a file onto a remote host.

TFTP is a simple protocol for transferring files, implemented on top of the UDP/IP protocols using IANA registered port number 69. TFTP was designed to be small and easy to implement, and therefore it lacks most of the advanced features offered by more robust file transfer protocols. TFTP only reads and writes files from or to a remote server. It cannot list, delete, or rename files or directories and it has no provisions for user authentication. Today TFTP is generally only used on local area networks (LAN).

The header files:

The Berkeley socket development library has many associated header files. They include:

<sys/socket.h>

Definitions for the most basic of socket structures with the BSD

socket API <sys/types.h>

Basic data types associated with structures within the BSD

socket API <netinet/in.h>

Definitions for the socketaddr_in{} and other base data structures.<sys/un.h>

Definitions and data type declarations for SOCK_UNIX streams

UDP:

UDP consists of a connectionless protocol with no guarantee of delivery. UDP packets may arrive out of order, become duplicated and arrive more than once, or even not arrive at all. Due to the minimal guarantees involved, UDP has considerably less overhead than TCP. Being connectionless means that there is no concept of a stream or connection between two hosts, instead, data arrives in datagrams.

UDP address space, the space of UDP port numbers (in ISO terminology, the TSAPs), is completely disjoint from that of TCP ports.

Server:

Code may set up a UDP server on port 7654 as follows:

```
sock = socket(PF_INET,SOCK_DGRAM,0);
sa.sin_addr.s_addr = INADDR_ANY;

sa.sin_port = htons(7654);
```

```
bound = bind(sock,(struct sockaddr *)&sa, sizeof(struct
sockaddr)); if (bound < 0)
fprintf(stderr,
"bind():
%s\n",strerror(errno)); listen(sock,3);
```

bind() binds the socket to an address/port pair.

listen() sets the length of the new connections queue.

```
while (1)
{
printf ("recv test....\n");
recsize = recvfrom(sock, (void *)hz, 100, 0, (struct sockaddr *)&sa, fromlen);
printf ("recsize: %d\n ",recsize);
if (recsize < 0)
fprintf(stderr, "%s\n", strerror(errno));
sleep(1);
printf("datagram: %s\n",hz);
}
```

This infinite loop receives any UDP datagrams to port 7654 using recvfrom(). It uses the parameters:

- socket
- pointer to buffer for data
- size of buffer
- flags (same as in read or other receive socket function)l address struct of sending peer
- length of address struct of sending peer.

Client:

A simple demo to send an UDP packet containing "Hello World!" to address 127.0.0.1, port 7654 might look like this:

```
#include <stdio.h>
#include <errno.h>
```

```
#include <string.h>
#include <sys/socket.h>
#include <sys/types.h>
#include <netinet/in.h>
int main(int argc, char *argv[])
{
int sock;
struct sockaddr_in sa;
int bytes_sent, buffer_length;
char buffer[200];
sprintf(buffer, "Hello World!");
buffer_length = strlen(buffer) + 1;
sock = socket(PF_INET, SOCK_DGRAM, 0);
sa.sin_family = AF_INET;
sa.sin_addr.s_addr = htonl(0x7F000001);
sa.sin_port = htons(7654);
bytes_sent = sendto(sock, buffer, buffer_length, 0, &sa,
sizeof(struct sockaddr_in) );
if(bytes_sent < 0)
printf("Error sending packet: %s\n", strerror(errno) );
return 0;
}
```

In this code, buffer provides a pointer to the data to send, and buffer_length specifies the size of the buffer contents.

Typical UDP client code

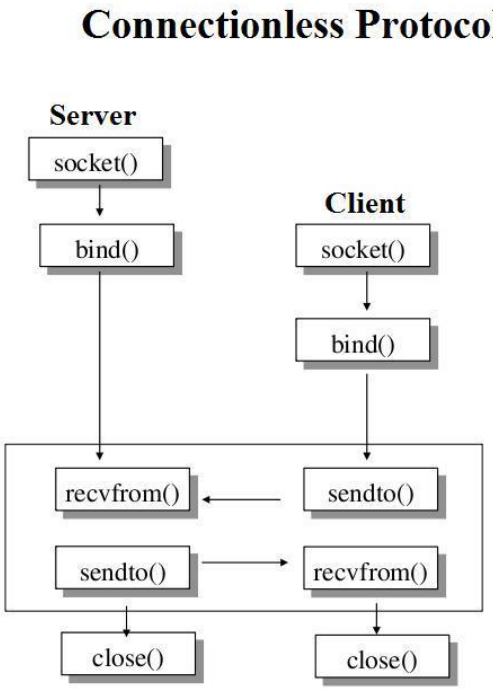
1. Create UDP socket to contact server (with a given hostname and service port number)
2. Create UDP packet.
3. Call send(packet), sending request to the server.
4. Possibly call receive(packet) (if we need a reply).

Typical UDP Server code

1. Create UDP socket listening to a well known port number.
2. Create UDP packet buffer
3. Call receive(packet) to get a request, noting the address of the client.
4. Process request and send reply back with send(packet).

APPLICATION

Socket programming is essential in developing any application over a network.



Implementation :

Server code :-

```
include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <fcntl.h>
#define PORT 5005
#define BUFFER_SIZE 65507
int main() {
    int sockfd;
    struct sockaddr_in serverAddr;
    char buffer[BUFFER_SIZE];
    char *filename = "server.txt";  // Specify the file to send
    // Create UDP socket
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd < 0) {
        perror("Socket creation failed");
        return 1;
    }
    // Prepare the sockaddr_in structure
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_port = htons(PORT);
    serverAddr.sin_addr.s_addr = inet_addr("172.16.1.142"); // Change to server's IP if needed
    // Send filename to server
    if (sendto(sockfd, filename, strlen(filename), 0, (struct sockaddr*)&serverAddr, sizeof(serverAddr)) < 0)
{
        perror("Failed to send filename");
        close(sockfd);
```

```
        return 1;
    }
    // Open the file to send
    int inFile = open(filename, O_RDONLY);
    if (inFile < 0) {
        perror("Failed to open file for reading");
        close(sockfd);
        return 1;
    }
    // Send file data
    ssize_t bytesRead;
    while ((bytesRead = read(inFile, buffer, BUFFER_SIZE)) > 0) {
        if (sendto(sockfd, buffer, bytesRead, 0, (struct sockaddr*)&serverAddr, sizeof(serverAddr)) < 0) {
            perror("Failed to send file data");
            break;
        }
    }
    // Close the input file
    close(inFile);
    printf("File %s sent successfully.\n", filename);
    close(sockfd);
    return 0;
}
```

output :-

```
(base) comp-inv-27@compinv27-ThinkCentre-M70s:~/Desktop$ gcc server.c
(base) comp-inv-27@compinv27-ThinkCentre-M70s:~/Desktop$ ./a.out
File server.txt sent successfully.
```

Client code :-

```
#include <stdio.h>
```

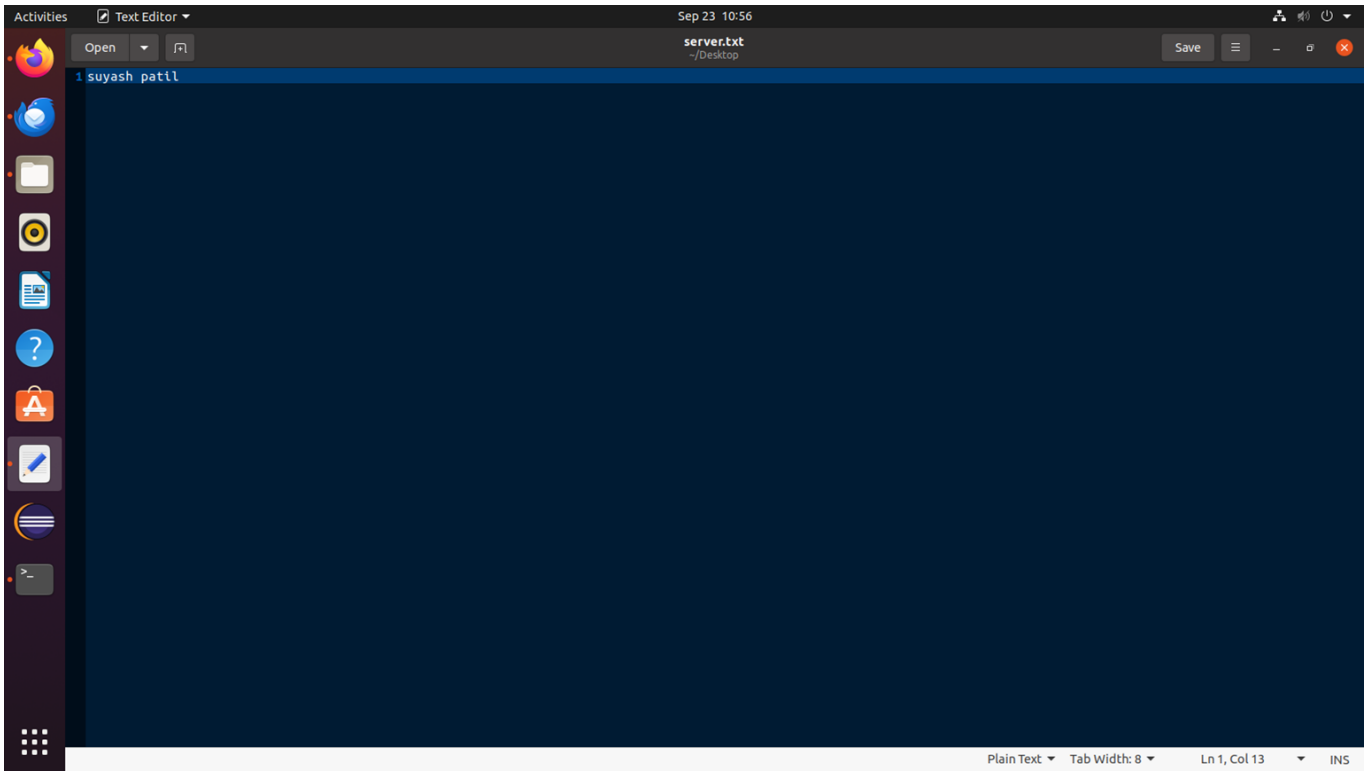
```
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <arpa/inet.h>
#include <fcntl.h>
#include <signal.h>
#define PORT 5005
#define BUFFER_SIZE 65507
int sockfd;
void handle_signal(int signal) {
    printf("\nReceived signal %d, closing socket.\n", signal);
    if (sockfd > 0) {
        close(sockfd);
    }
    exit(0);
}
int main() {
    struct sockaddr_in serverAddr, clientAddr;
    socklen_t addrLen = sizeof(clientAddr);
    char buffer[BUFFER_SIZE];
    // Create UDP socket
    sockfd = socket(AF_INET, SOCK_DGRAM, 0);
    if (sockfd < 0) {
        perror("Socket creation failed");
        return 1;
    }
    // Set up signal handling
    signal(SIGINT, handle_signal);
    signal(SIGTERM, handle_signal);
    // Prepare the sockaddr_in structure
```

```
    serverAddr.sin_family = AF_INET;
    serverAddr.sin_addr.s_addr = INADDR_ANY;
    serverAddr.sin_port = htons(PORT);
    // Bind the socket
    if (bind(sockfd, (struct sockaddr*)&serverAddr, sizeof(serverAddr)) < 0) {
        perror("Bind failed");
        close(sockfd);
        return 1;
    }
    printf("Server listening on port %d\n", PORT);
    // Receive filename
    int receivedBytes = recvfrom(sockfd, buffer, BUFFER_SIZE, 0, (struct sockaddr*)&clientAddr,
&addrLen);
    if (receivedBytes < 0) {
        perror("Failed to receive filename");
        close(sockfd);
        return 1;
    }
    buffer[receivedBytes] = '\0';
    printf("Receiving file: %s\n", buffer);
    // Open the output file
    int outFile = open(buffer, O_WRONLY | O_CREAT | O_TRUNC, 0644);
    if (outFile < 0) {
        perror("Failed to open file for writing");
        close(sockfd);
        return 1;
    }
    // Receive file data
    while (1) {
        receivedBytes = recvfrom(sockfd, buffer, BUFFER_SIZE, 0, (struct sockaddr*)&clientAddr,
```

```
&addrLen);
        if (receivedBytes < 0) {
            perror("Failed to receive file data");
            break;
        }
        if (receivedBytes == 0) {
            break; // End of file
        }
        if (write(outFile, buffer, receivedBytes) < 0) {
            perror("Failed to write to file");
            break;
        }
    }
    close(outFile);
    printf("File %s received successfully.\n", buffer);
    close(sockfd);
    return 0;
}
```

output :-

```
base) comp-inv-28@compinv28-
(base) comp-inv-28@compinv28-
Server listening on port 5005
Receiving file: server.txt
```



Conclusion : Thus we have successfully implemented the socket programming for UDP using C.

ASSIGNMENT No 11

Title: Write a program for DNS lookup. Given an IP address input, it should return URL & vice versa.

Objectives: To know how to check DNS from any URL, perform a program for DNS lookup.

Problem Statement: Need to write a program for DNS lookup where we will give input as an IP address and it will show the name of DNS.

Outcomes: after entering URL or Web link will get the host name of that particular DNS.

Tools Required:

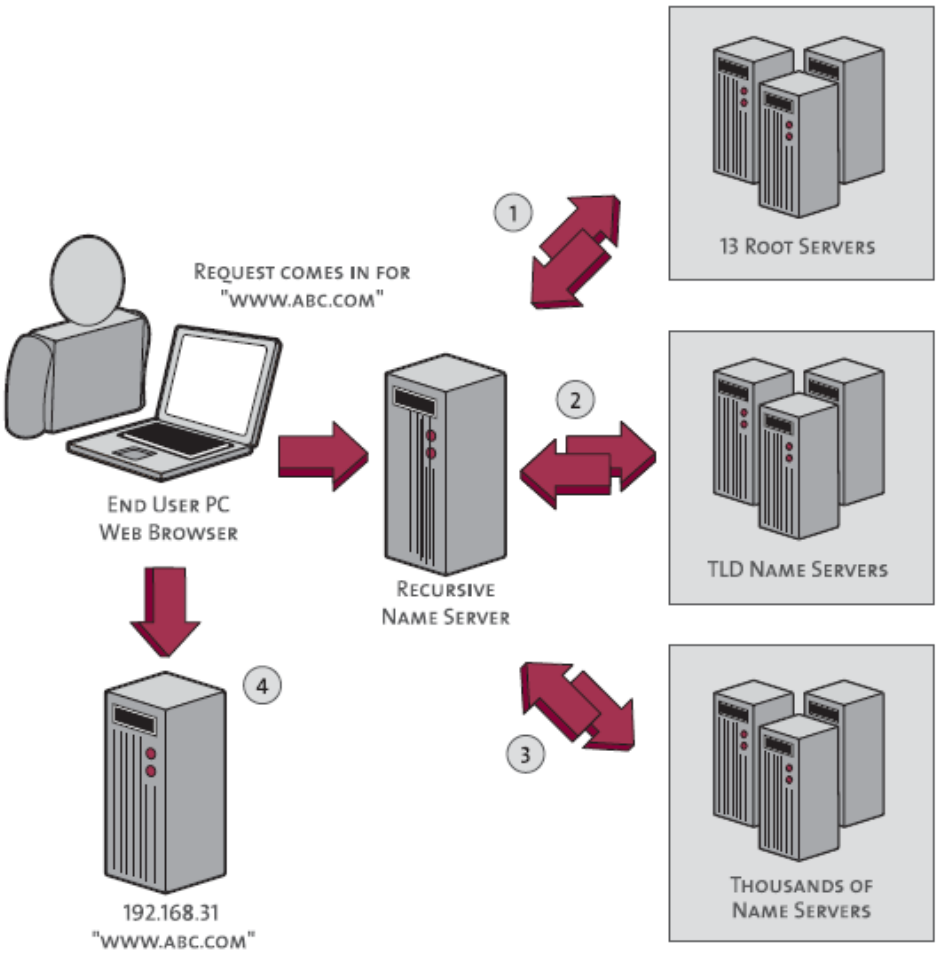
Software: Java

Theory:-A DNS lookup, in a general sense, is the process by which a DNS record is returned from a DNS server. This is like looking up a phone number in a phone book - that is why it is referred to as a "lookup". Interconnected computers, servers and smart phones need to know how to translate the email addresses and domain names people use into meaningful numerical addresses. A DNS lookup performs this function.

The basic idea of DNS is that humans can't easily remember long strings of digits like machines can, but can much more easily remember words. So, when you type in a domain name like www.techopedia.com, the request is forwarded to a DNS server (whether locally or at an ISP), which returns the corresponding IP address. This address is then used by all the computers and routers to channel the request and responses of a user's session. The result is the user sees web pages as expected or has email show up in an in-box.

The two types of DNS lookups are forward DNS lookups and reverse DNS lookups.Forward DNS lookup is using an Internet domain name to find an IP address. Reverse DNS lookup is using an Internet IP address to find a domain name. When you enter the address for a Web site at your browser (the address is formally called the Uniform Resource Locator, or URL), the address is transmitted to a nearby router which does a forward DNS lookup in a routing table to locate the IP address. Forward DNS (which stands for domain name system) lookup is the more common lookup since most users think in terms of domain names rather than IP addresses. However, occasionally you may see a Web page with a URL in which the domain name part is expressed as an IP address (sometimes called a dot address) and want to be able to see its domain name. An Internet facility that lets you do either forward or reverse DNS lookup yourself is called nslookup. It comes with some operating systems or you can download the program and install it in your computer.

First your computer queries the name server (DNS server) it is set up to use. This is the recursive name server shown above.



The name server doesn't know the IP address for www.abc.com, so it will start the following chain of queries before it can report back the IP address to your computer (the numbers below correspond to the numbers in the image).

1. Query the **Internet root servers** to get the name servers for the .com TLD.
2. Query the .com **TLD name servers** to get the authoritative name servers for abc.com.
3. Query the **authoritative name servers for abc.com** to finally get the IP address for the host www.abc.com, then return that IP address to your computer.
4. Done! Now that your computer has the IP address for www.abc.com, it can access that host.

Implementation :

```
import java.net.*;
import java.util.*;


public class IPDemo
{
 public static void main(String[] args){
  String host;
  Scanner ch = new Scanner(System.in);
  System.out.print("1.Enter Host Name \n2.Enter IP address \nChoice=");
  int choice = ch.nextInt();
  if(choice==1)
  {
  Scanner input = new Scanner(System.in);
  System.out.print("\n Enter host name: ");
  host = input.nextLine();
  try {
   InetAddress address = InetAddress.getByName(host);
   System.out.println("IP address: " + address.getHostAddress());
   System.out.println("Host name : " + address.getHostName());
   System.out.println("Host name and IP address: " + address.toString());
  }
  catch (UnknownHostException ex) {
      System.out.println("Could not find " + host);
  }
  }
  else
  {
  Scanner input = new Scanner(System.in);
  System.out.print("\n Enter IP address: ");
```

```
 host = input.nextLine();
 try {
  InetAddress address = InetAddress.getByName(host);
  System.out.println("Host name : " + address.getHostName());
  System.out.println("IP address: " + address.getHostAddress());
  System.out.println("Host name and IP address: " + address.toString());


 }
 catch (UnknownHostException ex) {
     System.out.println("Could not find " + host);
 }
 }
  }
}
```

```
java -cp /tmp/StnqkTUrmk/IPDemo
1.Enter Host Name
2.Enter IP address
Choice=1

 Enter host name: www.amazon.com
IP address: 3.167.165.162
Host name : www.amazon.com
Host name and IP address: www.amazon.com/3.167.165.162

=== Code Execution Successful ===
```

Conclusion:- Hence we have implemented DNSlookup.

ASSIGNMENT NO 12

Title: Installing and configure DHCP server and write a program to install the software on remote machine.

Objectives: to check how the DHCP server works

Problem Statement: We have to install DHCP server to configure the files which relate to DHCP

Outcomes: we can check the status of DHCP server after entering IP address of machine.

Tools Required:

Software: Linux OS and DHCP installation commands

Theory :

DHCP (Dynamic Host Configuration Protocol) is a protocol that lets network administrators manage centrally and automate the assignment of IP (Internet Protocol) configurations on a computer network. When using the Internet's set of protocols (TCP/IP), in order for a computer system to communicate to another computer system it needs a unique IP address. Without DHCP, the IP address must be entered manually at each computer system. DHCP lets a network administrator supervise and distribute IP addresses from a central point.

The purpose of DHCP is to provide the automatic (dynamic) allocation of IP client configurations for a specific time period (called a lease period) and to eliminate the work necessary to administer a large IP network. When connected to a network, every computer must be assigned a unique address. However, when adding a machine to a network, the assignment and configuration of network (IP) addresses has required human action.

The computer user had to request an address, and then the administrator would manually configure the machine. Mistakes in the configuration process are easy for novices to make, and can cause difficulties for both the administrator making the error as well as neighbors on the network. Also, when mobile computer users travel between sites, they have had to relive this process for each different site from which they connected to a network. In order to simplify the process of adding machines to a network and assigning unique IP addresses manually, there is a need to automate the task.

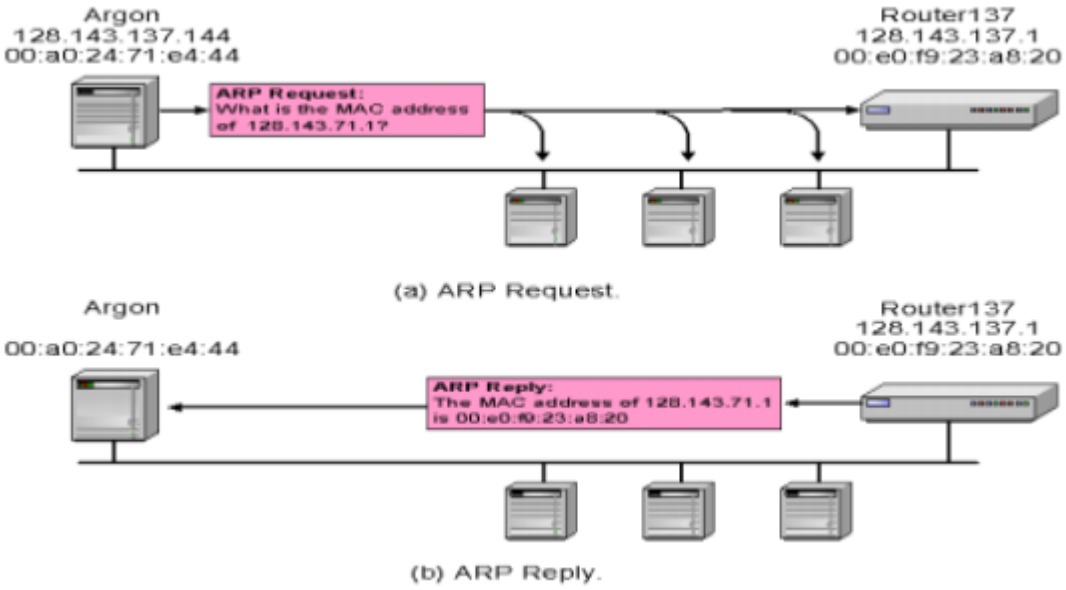
The introduction of DHCP alleviated the problems associated with manually assigning TCP/IP client addresses. Network administrators have quickly appreciated the importance, flexibility and ease-of-use offered in DHCP. Advantages of DHCP:- DHCP has several major advantages over manual configurations. Each computer gets its configuration from a "pool" of available numbers automatically for a specific time period (called a leasing period), meaning no wasted numbers. When a computer has finished with the address, it is released for another computer to use. Configuration information can be administered from a single point. Major network resource changes (e.g. a router changing address), requires only the DHCP server be updated with the new information, rather than every system.

DHCP message types:

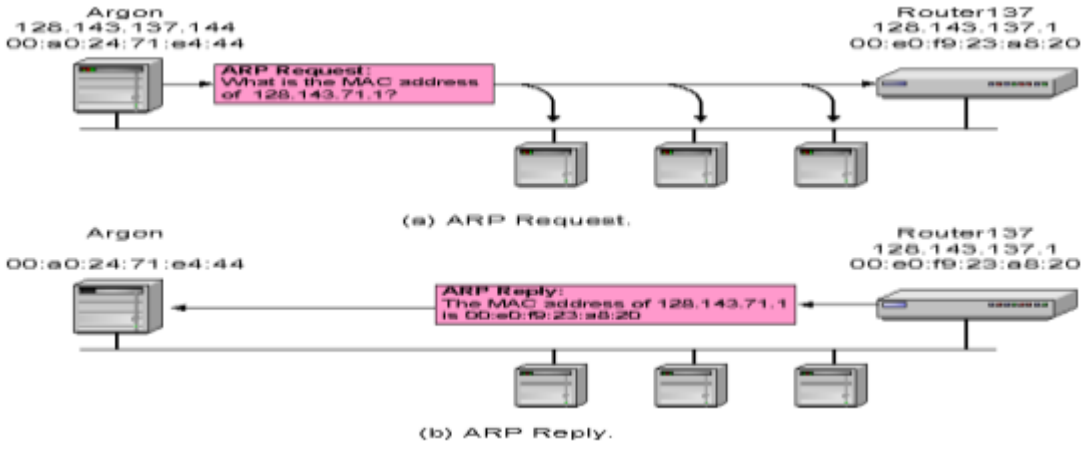
Value	Message Type
1	DHCPDISCOVER
2	DHCPOFFER
3	DHCPREQUEST
4	DHCPDECLINE
5	DHCPACK
6	DHCPNAK
7	DHCPRELEASE
8	DHCPINFORM

DHCP Operations:-

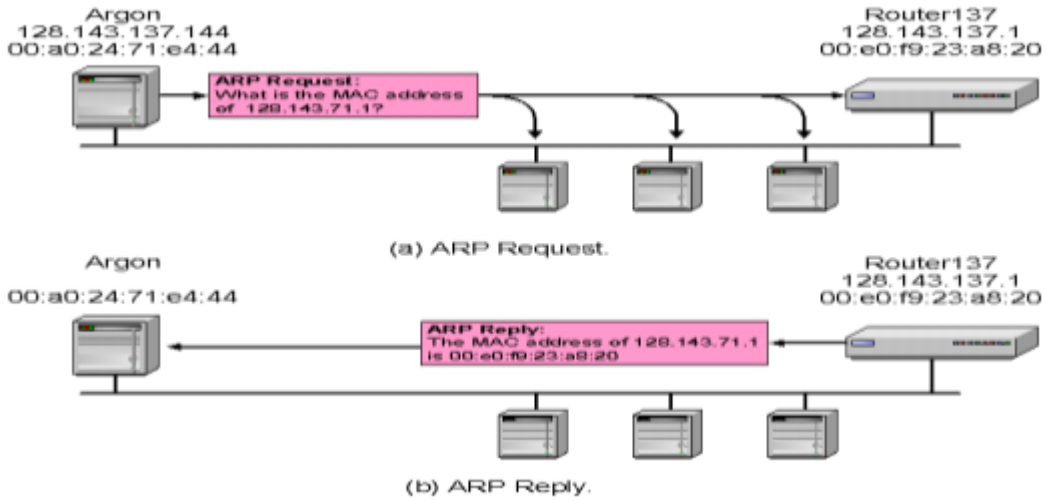
1. DHCP Discover



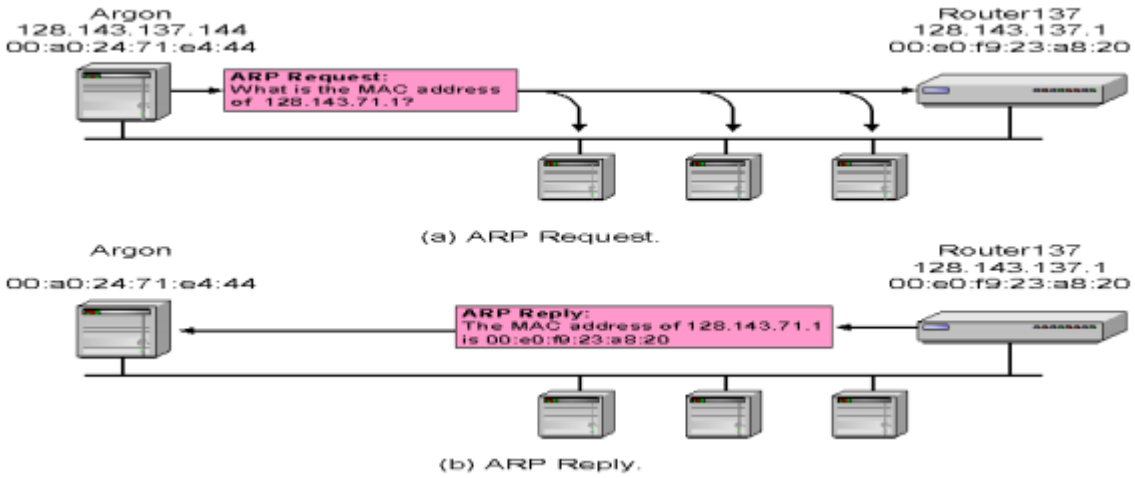
2. DHCP Offer



3. DHCP Discover: At this time, the DHCP client can start to use the IP address



4. DHCP Release: At this time, the DHCP client has released the IP address



Installing DHCP in Ubuntu:

Open terminal and type following commands:-

1. sudo apt-get install isc-dhcp-server

2. sudo gedit /etc/dhcp/dhcpd.conf then make changes in file.... default-lease-time 600; max-lease-time 7200; option subnet-mask 255.255.255.0; option broadcast-address 10.1.32.255; subnet 192.168.1.0 netmask 255.255.255.0 range 10.1.32.10 10.1.32.20; }

3. save file and close

4. again on terminal give following commands.... sudo service isc-dhcp-server restart sudo service isc-dhcp-server start

5. On another PC in Internet properties change to Obtain IP address automatically and then check the IP address.

Conclusion: Hence we Installed and Configured DHCP and studied Installation of Software on remote Machine.

ASSIGNMENT No 13

Title: Capture packets using Wireshark, write the exact packet capture filter expressions to accomplish the following and save the output in file:

1. Capture all TCP traffic to/from Facebook, during the time when you log in to your Facebook account
2. Capture all HTTP traffic to/from Facebook, when you log in to your Facebook account
3. Write a DISPLAY filter expression to count all TCP packets (captured under item #1) that have the flags SYN, PSH, and RST set. Show the fraction of packets that had each flag set.
4. Count how many TCP packets you received from / sent to Face book, and how many of each were also HTTP packets.

Objectives: to trace the packets with help of wireshark where TCP , HTTP traffic need to capture and study

Problem Statement: we need to capture packets while log in to facebook account and count the TCP packets received by wireshark

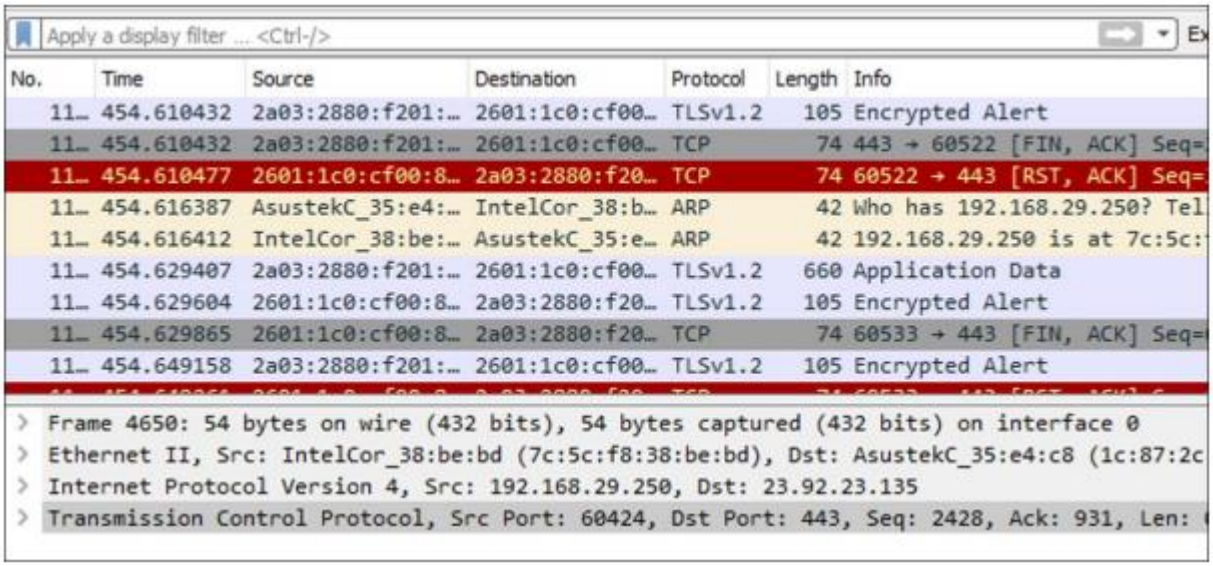
Outcomes: We will get packets captured by tool where TCP traffic , HTTP traffic from Facebook account.

Tools Required:

Software: Wireshark

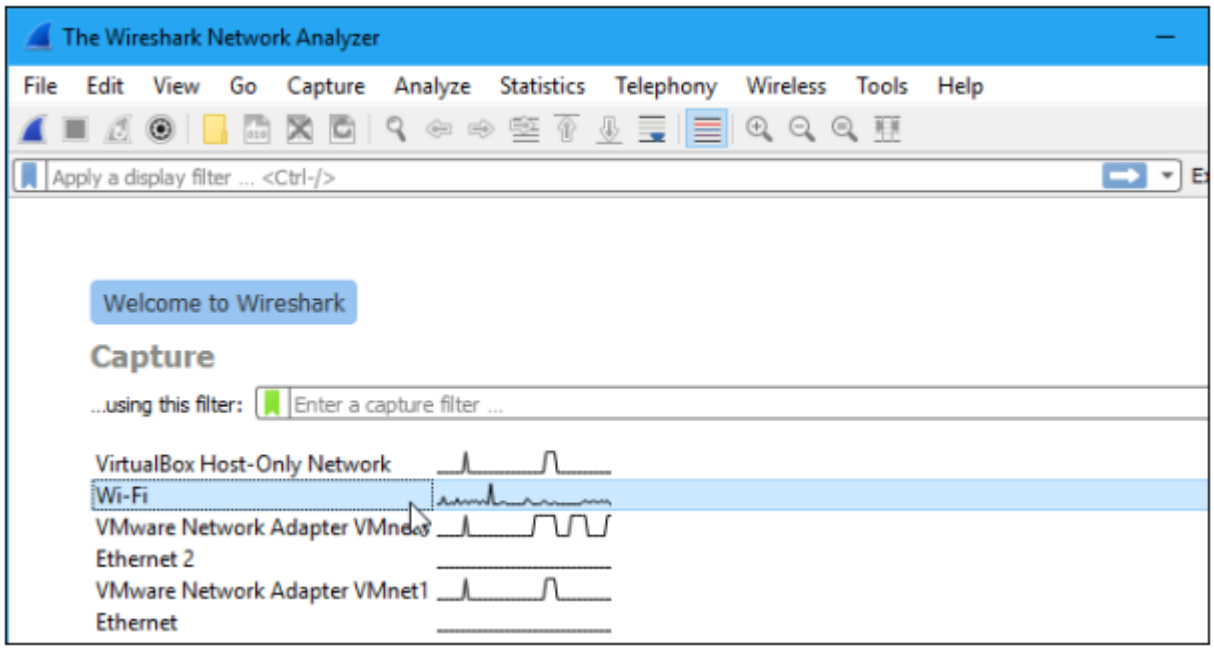
Theory :

Wireshark, a network analysis tool formerly known as Ethereal, captures packets in real time and display them in human-readable format. Wireshark includes filters, color coding, and other features that let you dig deep into network traffic and inspect individual packets. This tutorial will get you up to speed with the basics of capturing packets, filtering them, and inspecting them. You can use Wireshark to inspect a suspicious program's network traffic, analyze the traffic flow on your network, or troubleshoot network problems. You can download Wireshark for Windows or macOS from https://www.wireshark.org/ If you're using Linux or another UNIX-like system, you'll probably find Wireshark in its package repositories. For example, if you're using Ubuntu, you'll find Wireshark in the Ubuntu Software Center. Just a quick warning: Many organizations don't allow Wireshark and similar tools on their networks. Don't use this tool at work unless you have permission

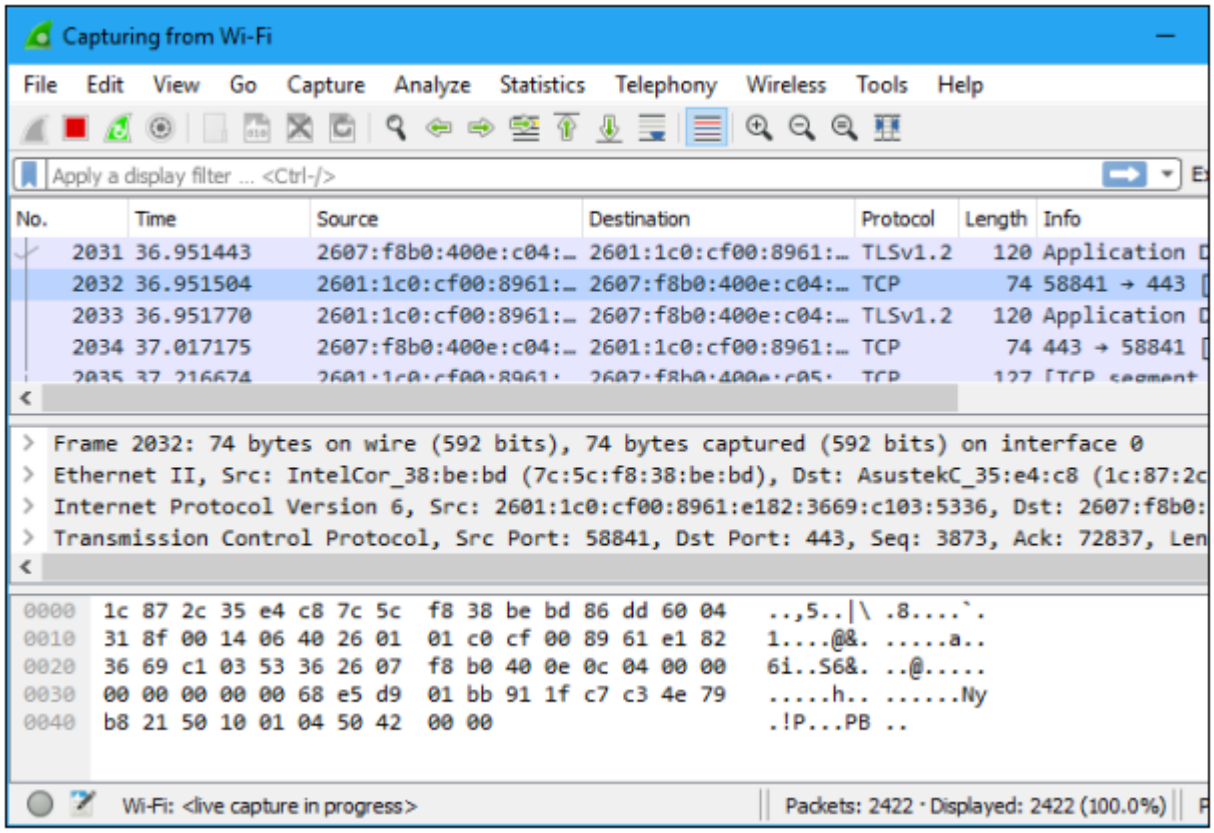


CAPTURING PACKETS

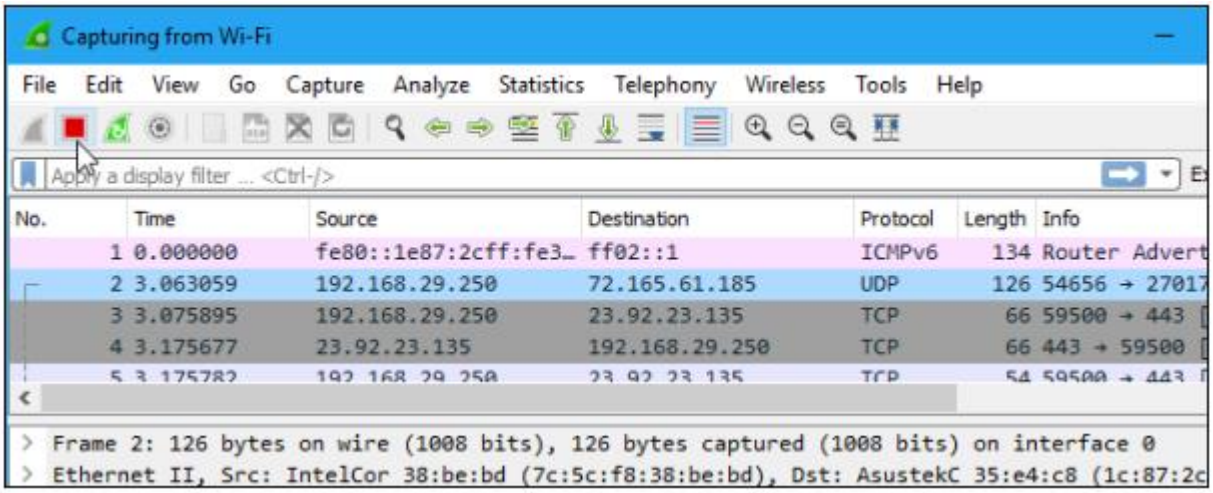
After downloading and installing Wireshark, you can launch it and double-click the name of a network interface under Capture to start capturing packets on that interface. For example, if you want to capture traffic on your wireless network, click your wireless interface. You can configure advanced features by clicking Capture > Options, but this isn't necessary for now.



As soon as you click the interface's name, you'll see the packets start to appear in real time. Wireshark captures each packet sent to or from your system. 48 If you have promiscuous mode enabled—it's enabled by default—you'll also see all the other packets on the network instead of only packets addressed to your network adapter. To check if promiscuous mode is enabled, click Capture > Options and verify the "Enable promiscuous mode on all interfaces" checkbox is activated at the bottom of this window.

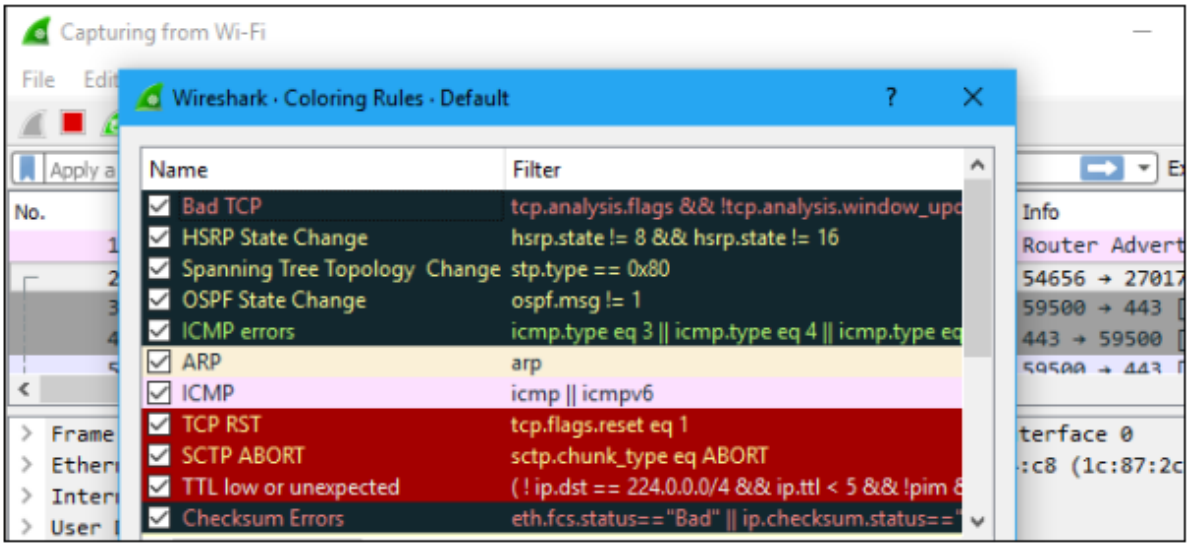


Click the red "Stop" button near the top left corner of the window when you want to stop capturing traffic.

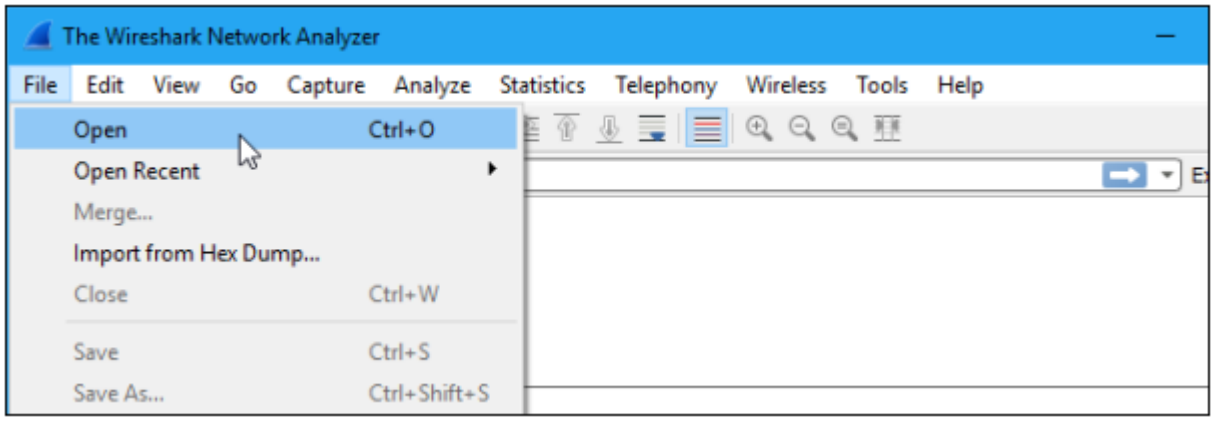


Color Coding :

You'll probably see packets highlighted in a variety of different colors. Wireshark uses colors to help you identify the types of traffic at a glance. By default, light purple is TCP traffic, light blue is UDP traffic, and black identifies packets with errors—for example, they could have been delivered out of order. To view exactly what the color codes mean, click View > Coloring Rules. You can also customize and modify the coloring rules from here, if you like.

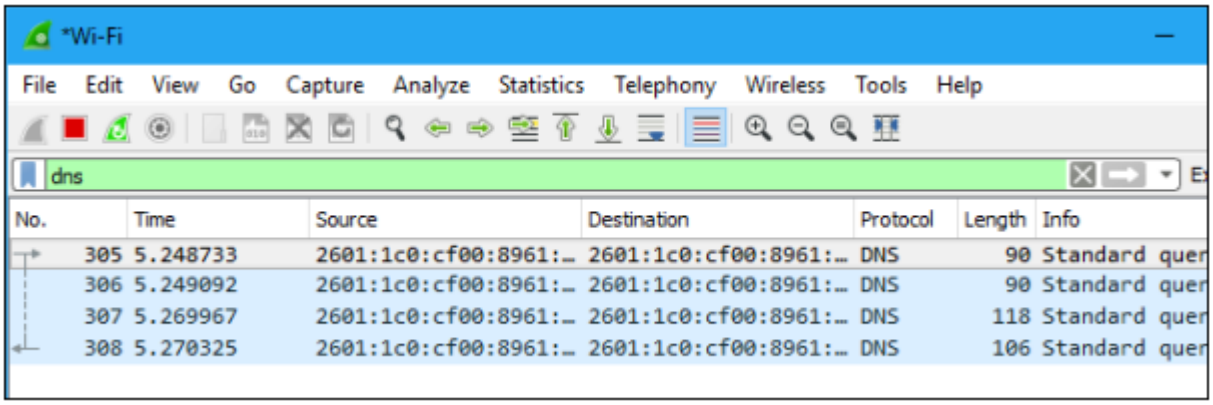


If there's nothing interesting on your own network to inspect, Wireshark's wiki has you covered. The wiki contains a page of sample capture files that you can load and inspect. Click File > Open in Wireshark and browse for your downloaded file to open one. You can also save your own captures in Wireshark and open them later. Click File > Save to save your captured packets.

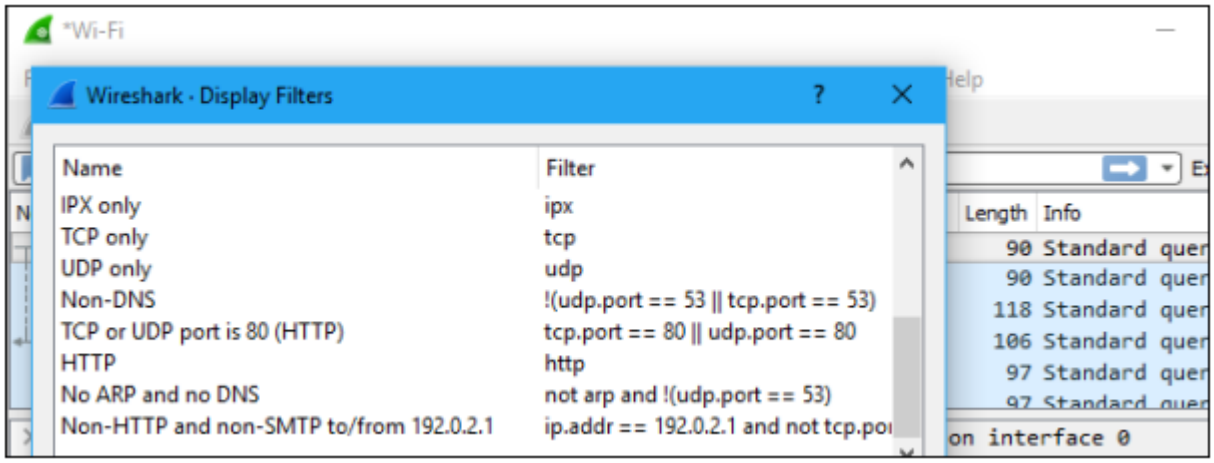


Filtering Packets :

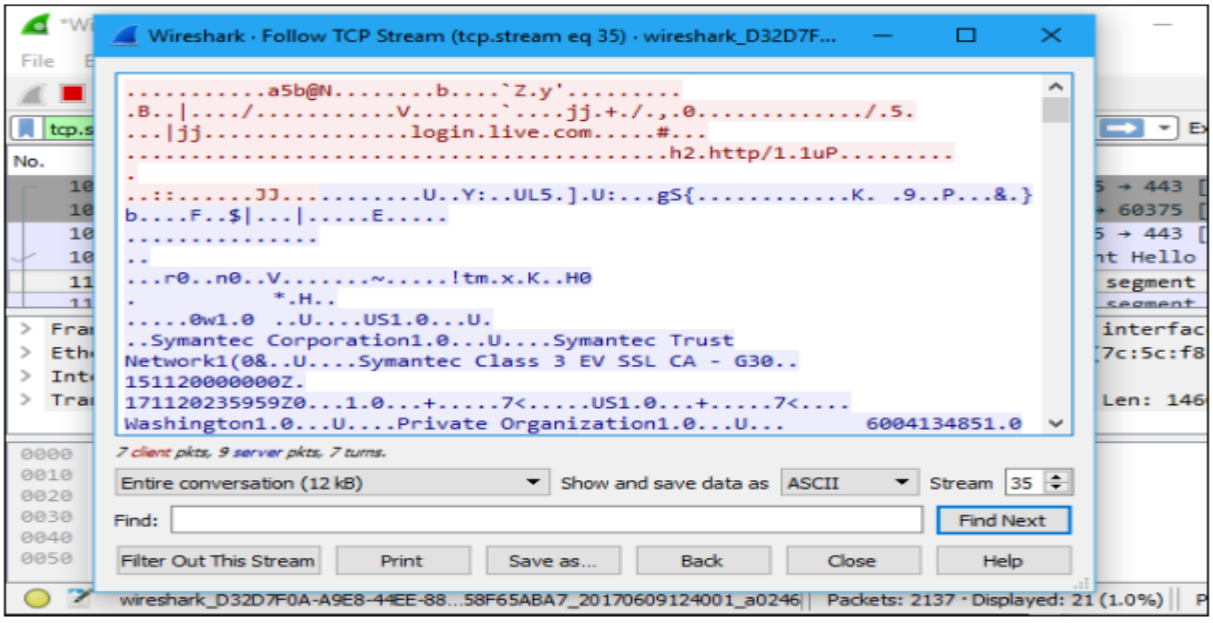
If you're trying to inspect something specific, such as the traffic a program sends when phoning home, it helps to close down all other applications using the network so you can narrow down the traffic. Still, you'll likely have a large amount of packets to sift through. That's where Wireshark's filters come in. The most basic way to apply a filter is by typing it into the filter box at the top of the window and clicking Apply (or pressing Enter). For example, type "dns" and you'll see only DNS packets. When you start typing, Wireshark will help you autocomplete your filter.



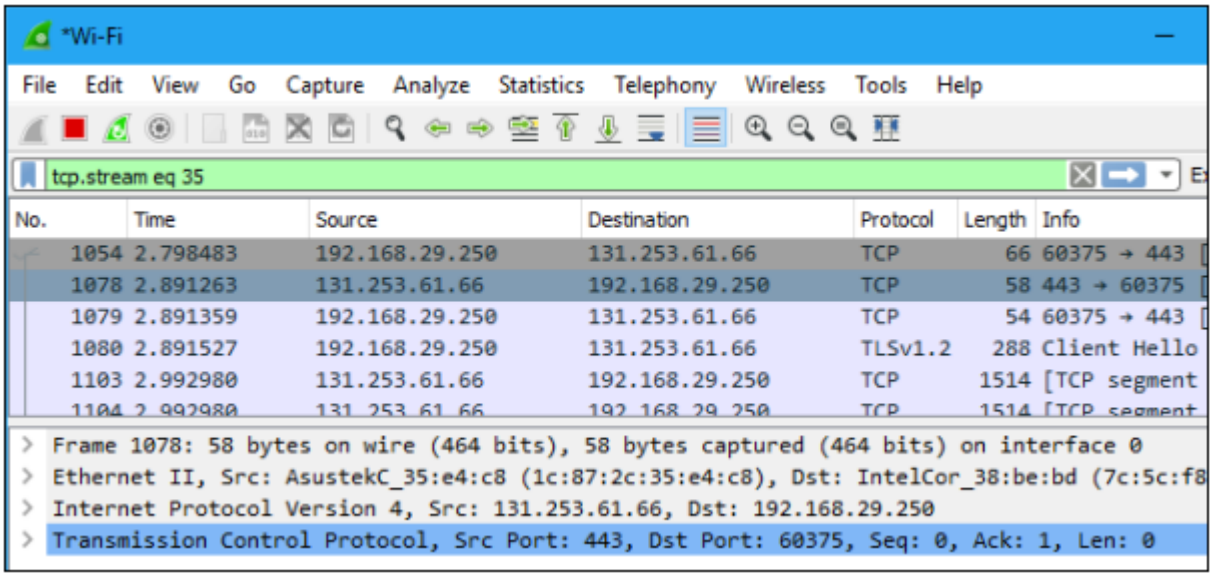
You can also click Analyze > Display Filters to choose a filter from among the default filters included in Wireshark. From here, you can add your own custom filters and save them to easily access them in the future. For more information on Wireshark's display filtering language, read the Building display filter expressions page in the official Wireshark documentation.



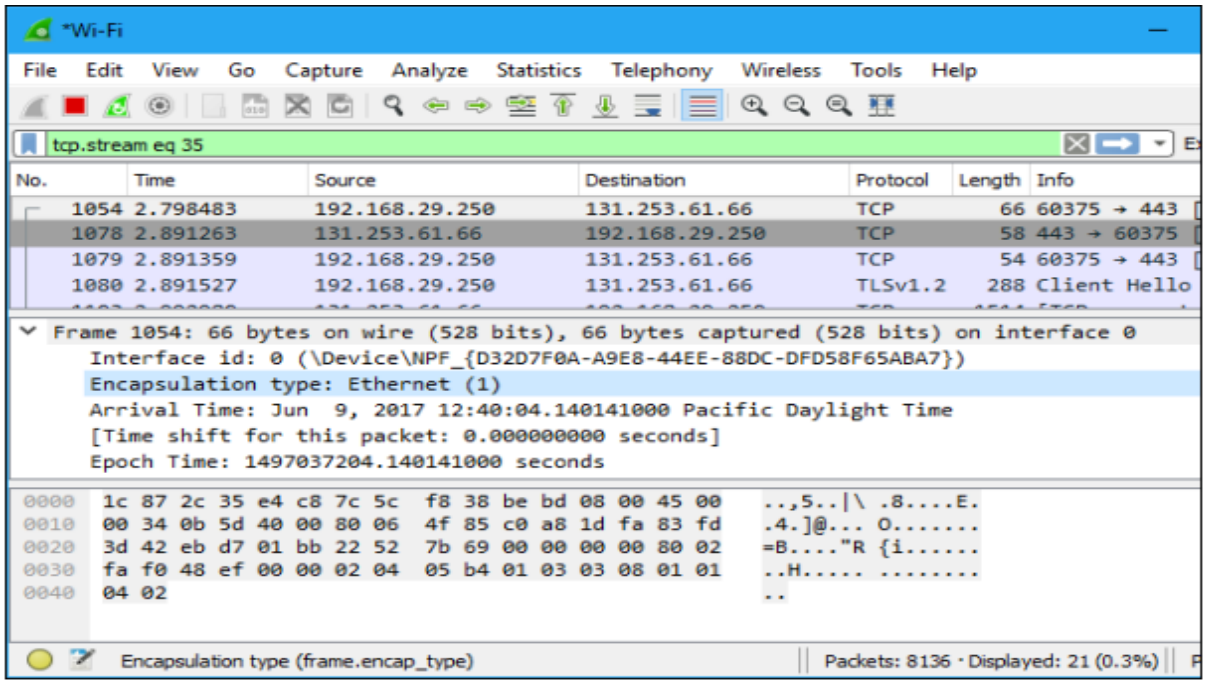
Another interesting thing you can do is right-click a packet and select Follow > TCP Stream. You'll see the full TCP conversation between the client and the server. You can also click other protocols in the Follow menu to see the full conversations for other protocols, if applicable.



Close the window and you'll find a filter has been applied automatically. Wireshark is showing you the packets that make up the conversation.

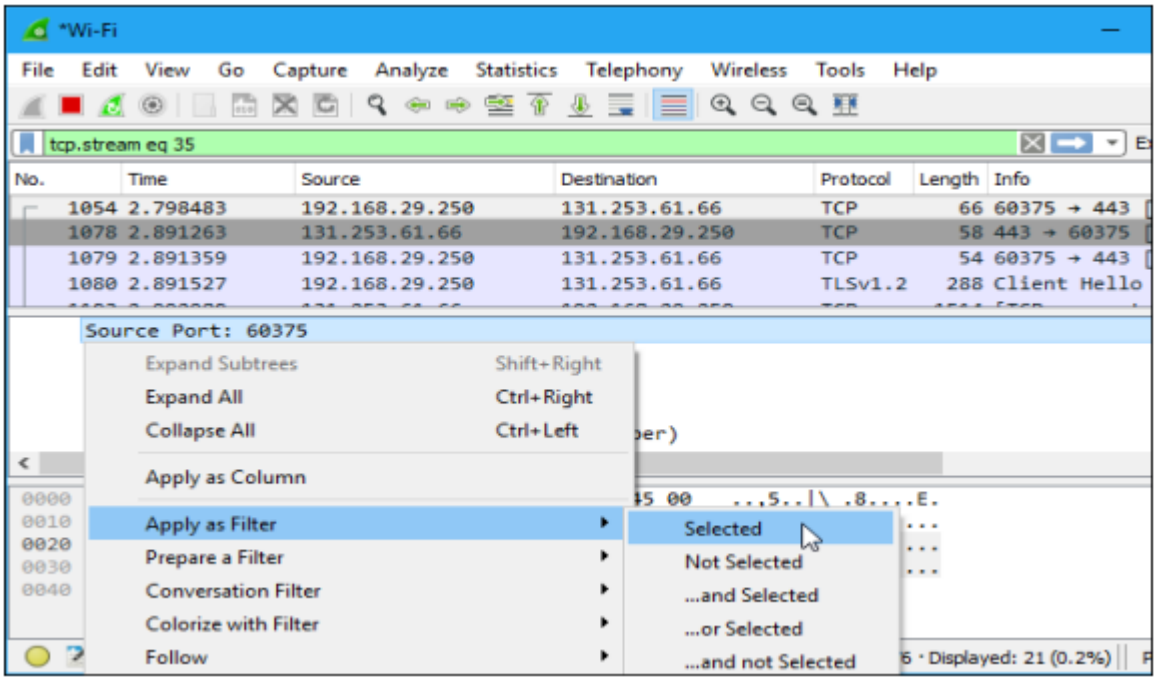


INSPECTING PACKETS : Click a packet to select it and you can dig down to view its details.



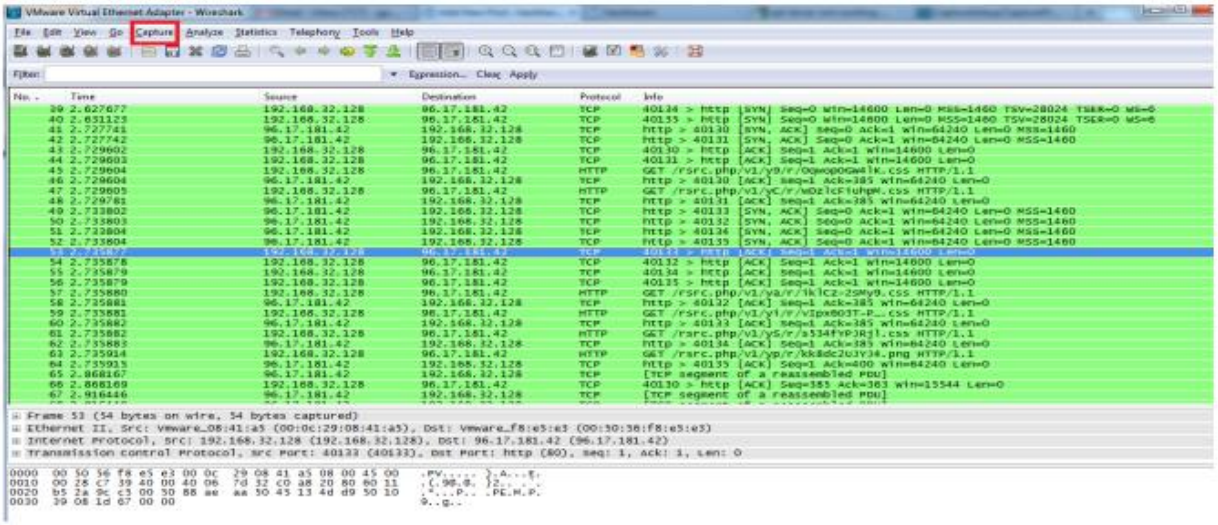
You can also create filters from here — just right-click one of the details and use the Apply as Filter submenu

to create a filter based on it.

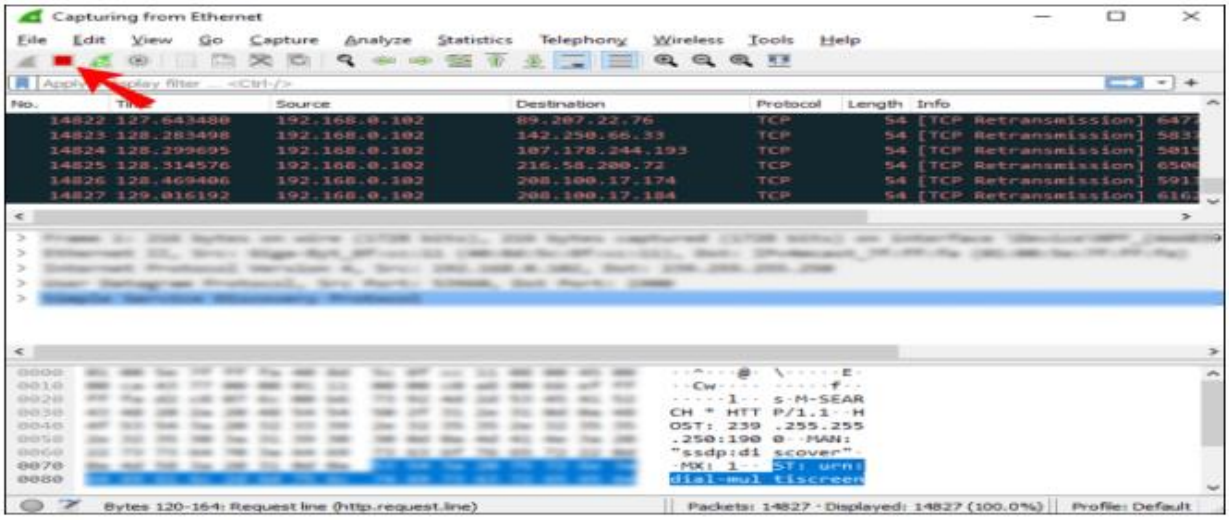


Wireshark is an extremely powerful tool, and this tutorial is just scratching the surface of what you can do with it. Professionals use it to debug network protocol implementations, examine security problems and inspect network protocol internals.

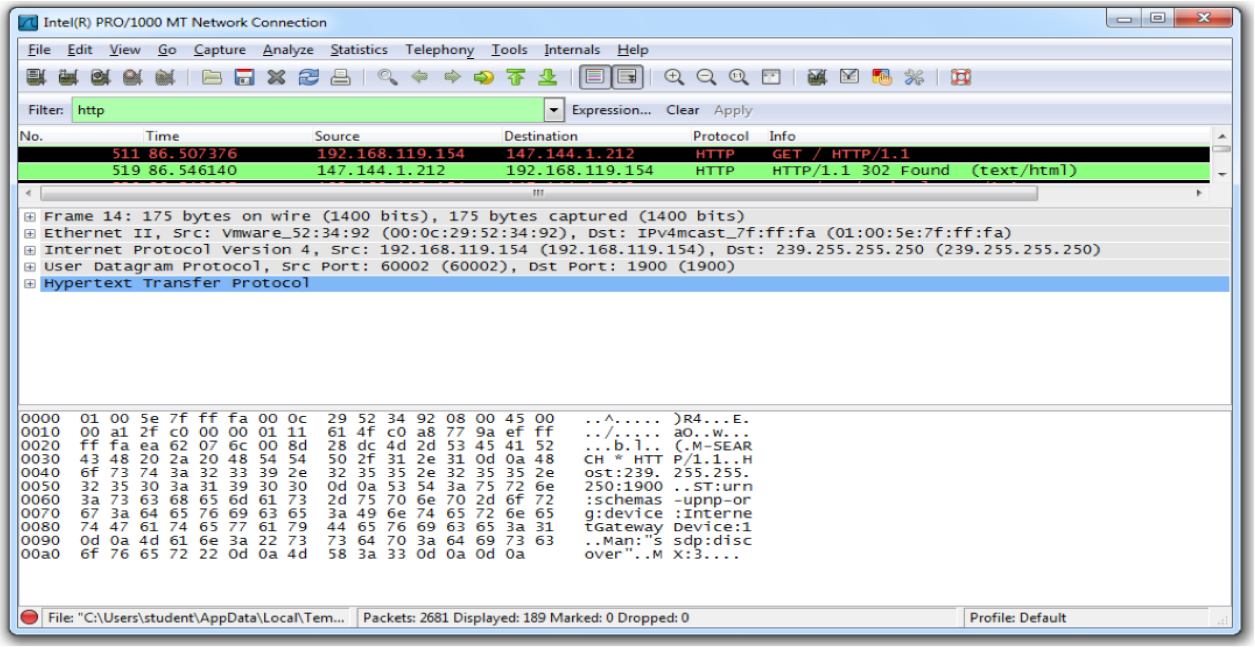
OUTPUT/SCREENSHOT



TCP TRAFFIC



HTTP TRAFFIC



Conclusion : Hence we have implemented the practical where with help of Wireshark we traced TCP traffic, HTTP Trafic.

Assignment No-14

Title: Study and analyze the performance of HTTP,HTTPS,FTP protocol using Packet Tracer Tool.

Objectives: to study the protocols such as HTTP, HTTPS , FTP with CISCO packet tracer.

Problem Statement: we need to study and analyze HTTP protocols.

Outcomes: we can peform HTTP connection and check the process of message transfer.

Tools Required:

Software: CISCO packet tracer

Theory:

HTTP:

HTTP stands for HyperText Transfer Protocol. It was invented by Tim Berner. HyperText is the type of text which is specially coded with the help of some standard coding language called HyperText Markup Language (HTML). HTTP/2 is the latest version of HTTP, which was published in May 2015.

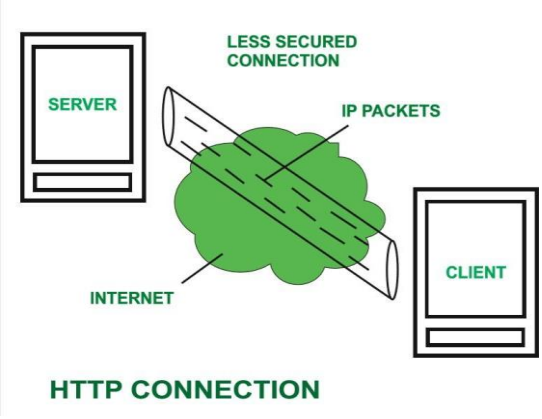
The protocols that are used to transfer hypertext between two computers is known as HyperText Transfer Protocol.

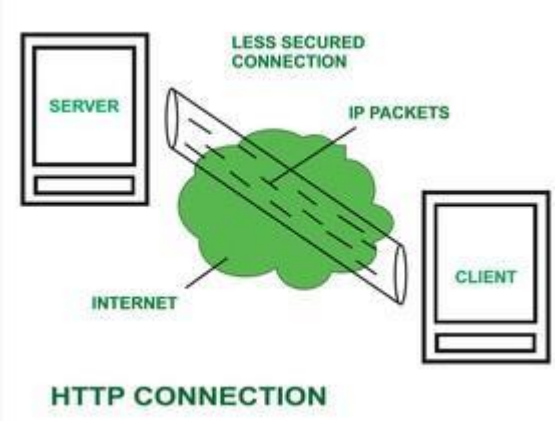
HTTP provides a standard between a web browser and web server to establish communication. It is a set of rules for transferring data from one computer to another. Data such as text, images, and other multimedia files are shared on the World Wide Web.

Whenever a web user opens their web browser, the user indirectly uses HTTP. It is an application protocol which is used for distributed, collaborative, hypermedia information systems.

First of all, whenever we want to open any website then first we open a web browser. After that we will type the URL of that website (e.g., www.facebook.com). This URL is now sent to Domain Name Server (DNS). Then DNS first checks records for this URL in their database, then DNS will return the IP address to the web browser corresponding to this URL. Now the browser is able to send requests to the actual server.

After the server sends data to the client, the connection will be closed. If we want something else from server we should have to re-establish the connection between client and server.





HTTPS:

HTTPS stands for HyperText Transfer Protocol Secure. HTTP Secure (HTTPS), could be a combination of the Hypertext Transfer Protocol with the SSL/TLS convention to supply encrypted communication and secure distinguishing proof of an arranged web server.

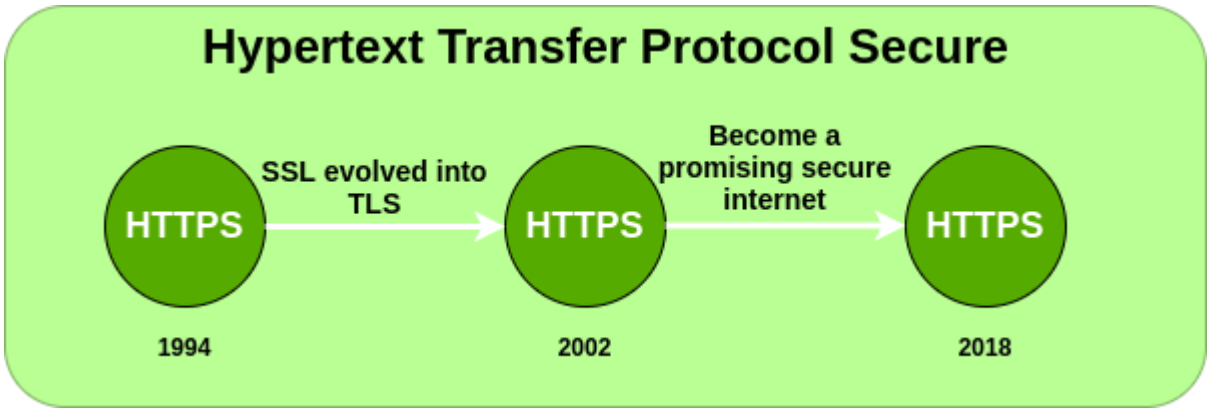
If the URL of that site is just HTTP, at that point anything you're perusing or whatever points of interest you're putting on that site, on the off chance that a programmer needs to take your data.

Therefore, HTTPS is more secure than HTTP because HTTPS is certified by the SSL(Secure Socket Layer).

Whatever website you are visiting on the internet, if its URL is HTTP, then that website is not secure.

If a website has an SSL certificate installed then the URL of that website will be HTTPS and that website will be completely secure. You can give any information about your credit card, debit cards, OTP and anything else.

HTTP Improved years:

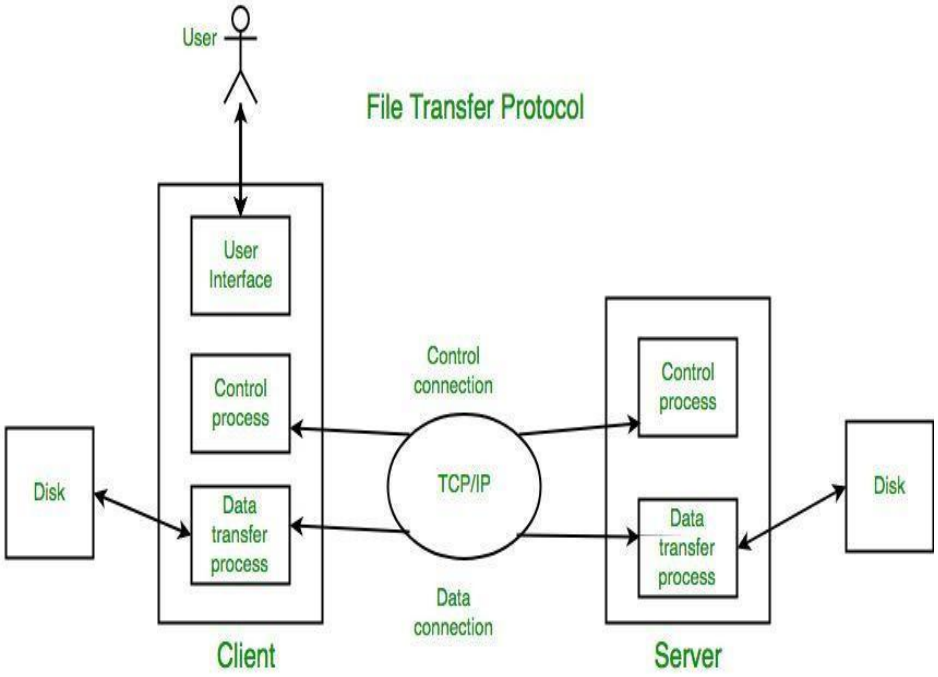


Characteristics of HTTPS:

- Security: Nowadays there's a lot of cyber-attacks on the web. And online instalments have also expanded. That's why we need to be secure.If there is no security in any website,then no one will use that website.
- Need of SSL: Some SEO specialists accept that by introducing SSL on the site, there are a few SEO benefits from Google. And by applying SSL, the positioning of the site in Google is additionally boosted.
- Authentication: HTTPS encrypts all message substance, including the HTTP headers and the request/response data.The verification perspective of HTTPS requires a trusted third party to sign server-side digital certificates.
- Browsing Privately: HTTPS is presently utilised more frequently by web clients than the first non-secure HTTP, fundamentally to ensure page genuineness on all sorts of websites,secure accounts and to keep client communications.

FTP:

File Transfer Protocol(FTP) is an application layer protocol which moves files between local and remote file systems. It runs on the top of TCP, like HTTP. To transfer a file, 2 TCP connections are used by FTP in parallel: control connection and data connection.



What is a control connection?

For sending control information like user identification, password, commands to change the remote directory, commands to retrieve and store files, etc., FTP makes use of control connection. The control connection is initiated on port number 21.

What is data connection?

For sending the actual file, FTP makes use of data connection. A data connection is initiated on port number 20.

FTP sends the control information out-of-band as it uses a separate control connection. Some protocols send their request and response header lines and the data in the same TCP connection. For this reason, they are said to send their control information in-band. HTTP and SMTP are such examples.

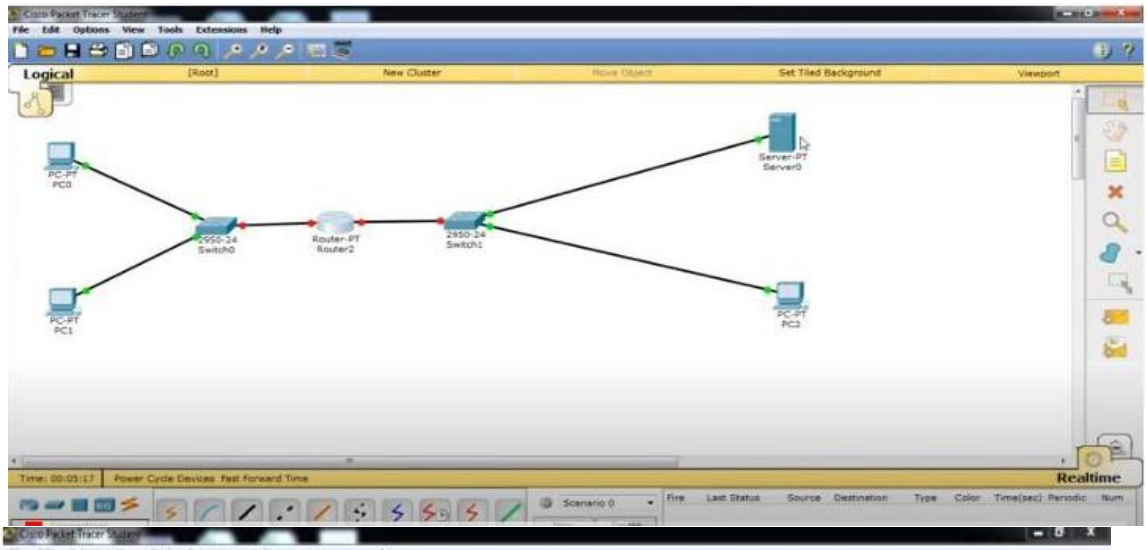
FTP Session :

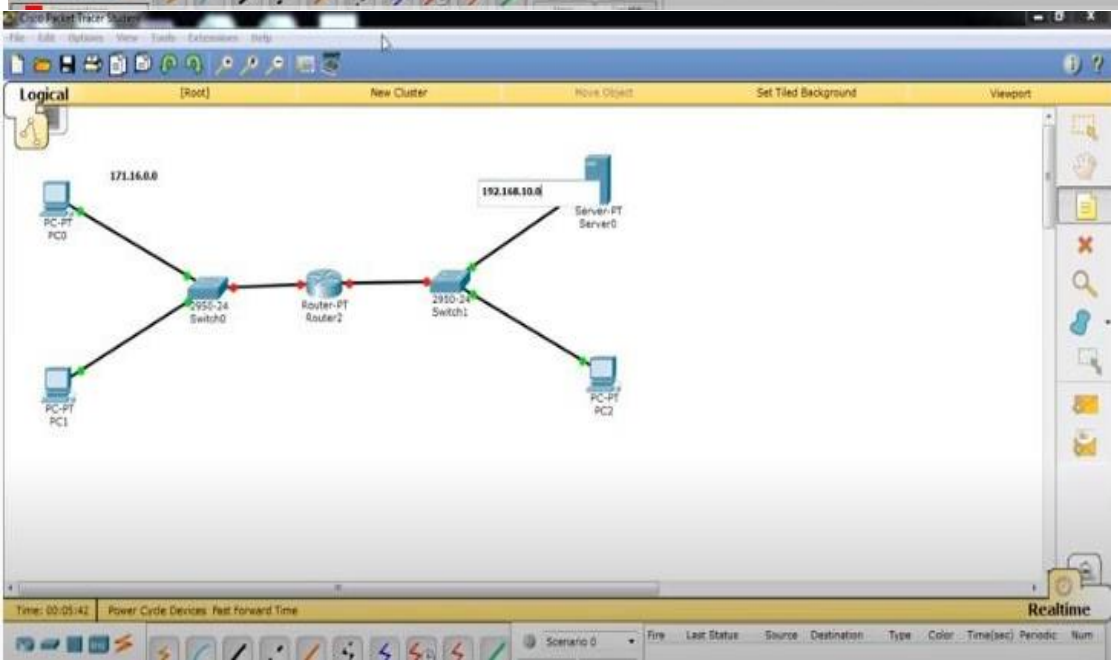
When a FTP session is started between a client and a server, the client initiates a control TCP connection with the server side. The client sends control information over this. When the server receives this, it initiates a data connection to the client side. Only one file can be sent over one data connection. But the control connection remains active throughout the user session. As we know HTTP is stateless i.e. it does not have to keep track of any user state. But FTP needs to maintain a state about its user throughout the session.

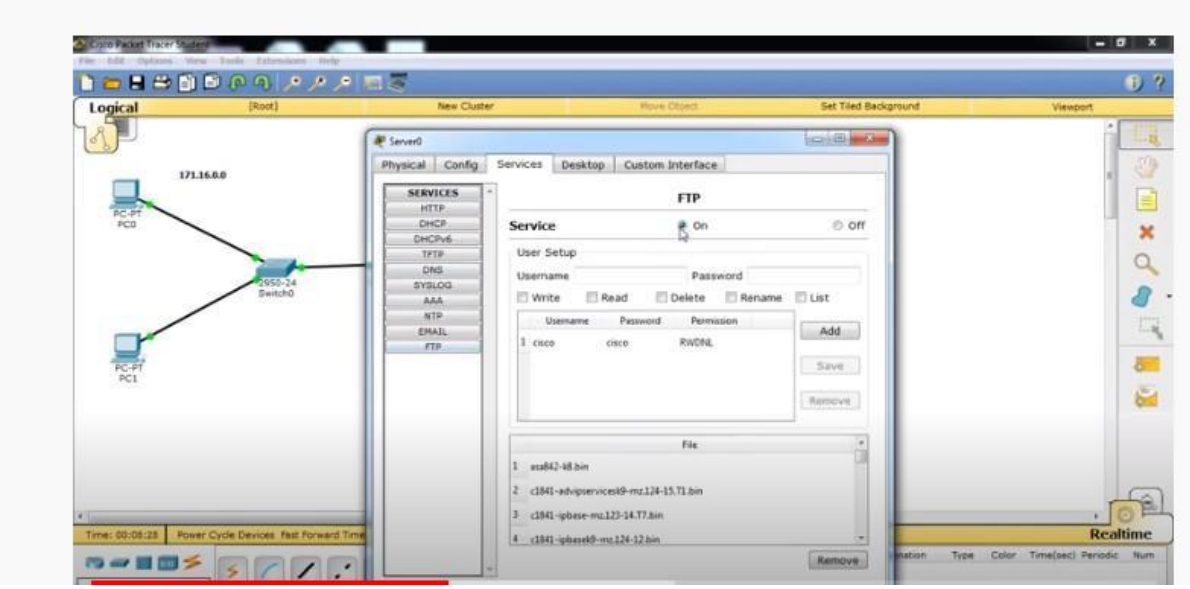
Data Structures : FTP allows three types of data structures :

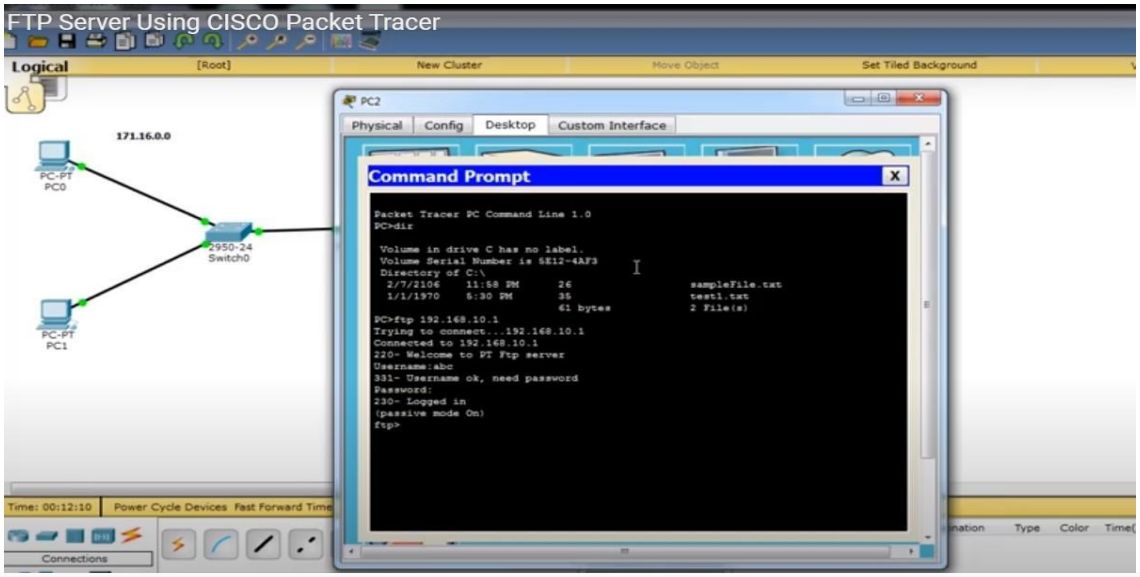
1. File Structure – In file-structure there is no internal structure and the file is considered to be a continuous sequence of data bytes.
2. Record Structure – In record-structure the file is made up of sequential records.
3. Page Structure – In page-structure the file is made up of independent indexed pages.

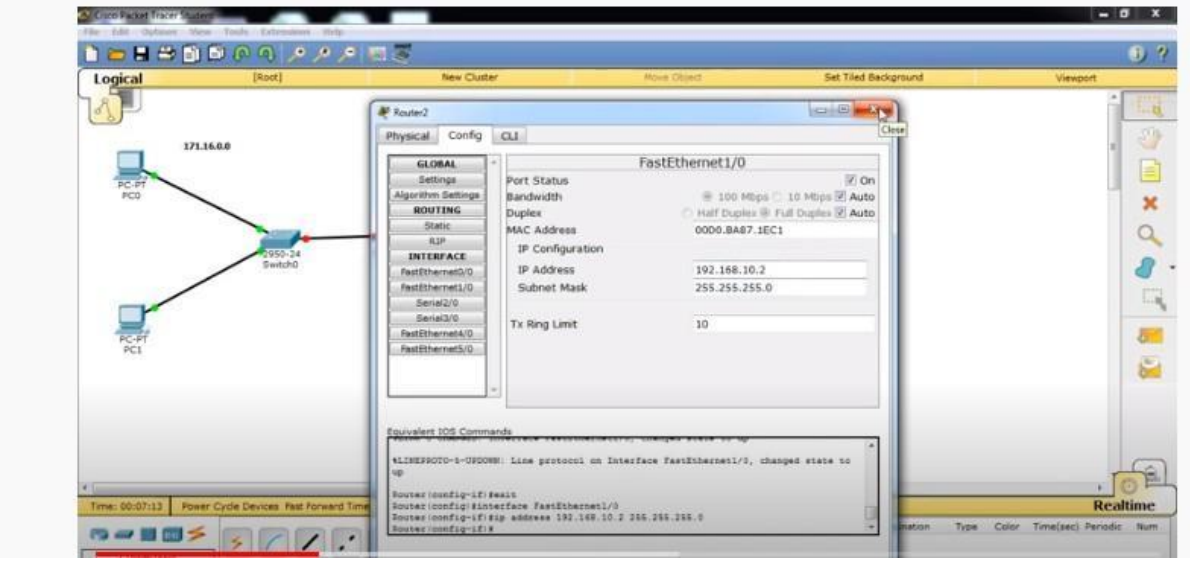
Implementation Screenshots:

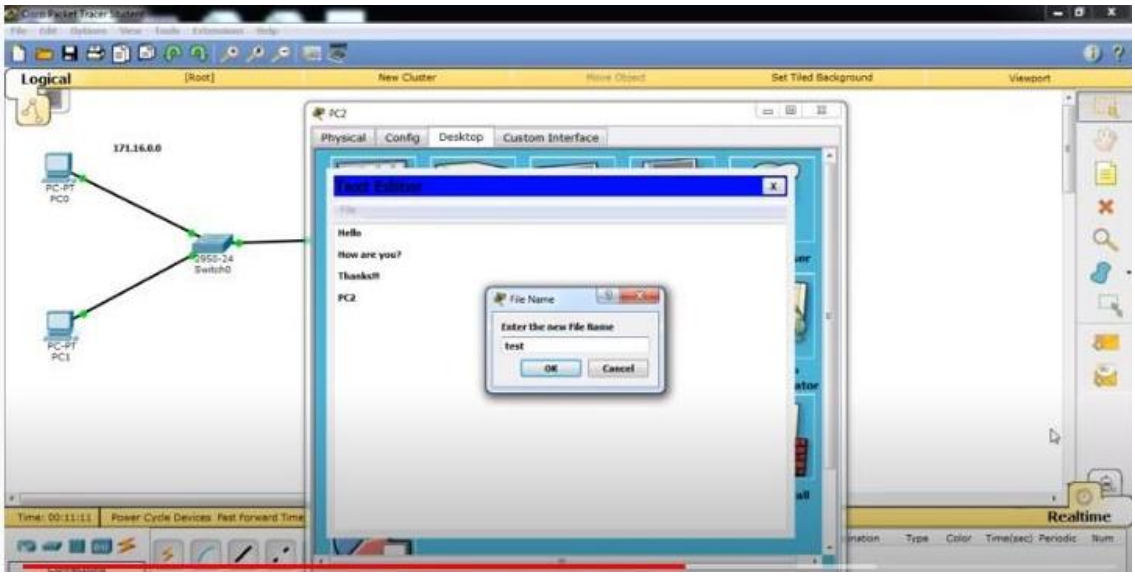


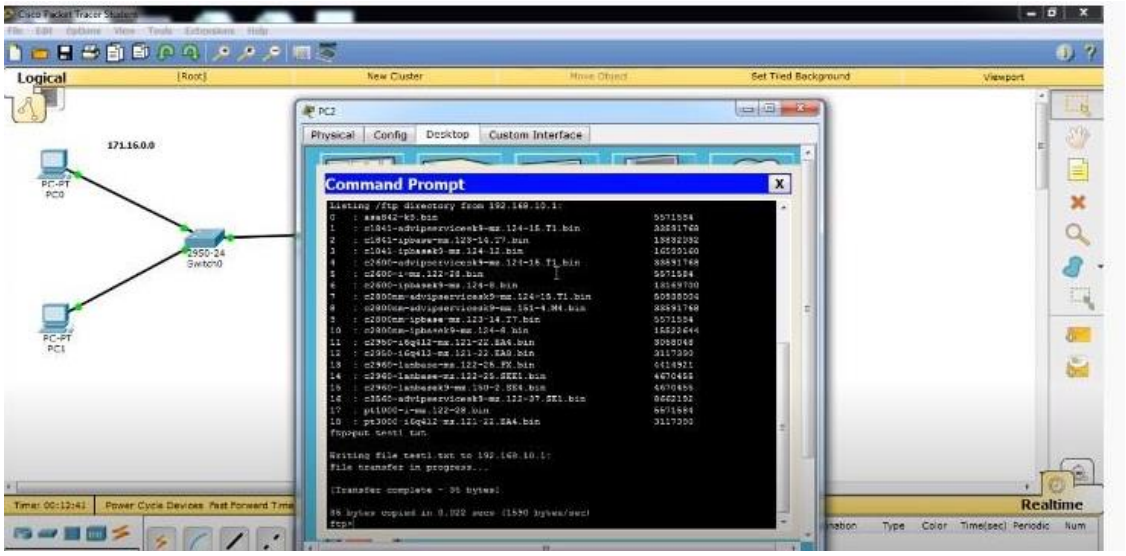



Logical
Server0
Physical
Config
Services
Desktop
Custom Interface
SERVICES
HTTP
DHCP
DHCPv6
TFTP
DNS
SYSLOG
AAA
NTP
EMAIL
FTP
FTP
Service
On
Off
User Setup
Username
Password
Write
Read
Delete
Rename
List
Add
Save
Remove
171.16.0.0
PC-PT PC0
PC-PT PC1
2950-24 Switch0
Realtime


FTP Server Using CISCO Packet Tracer
Logical
PC2
Physical
Config
Desktop
Custom Interface
Command Prompt
Packet Tracer PC Command Line 1.0
PC>dir
Volume in drive C has no label.
Volume Serial Number is 5E12-4AF3
Directory of C:\
2/7/2106 11:58 PM 26 sampleFile.txt
1/1/1970 5:30 PM 35 test1.txt
61 bytes 2 File(s)
PC>ftp 192.168.10.1
Trying to connect...192.168.10.1
Connected to 192.168.10.1
220- Welcome to PT Ftp server
Username:abc
331- Username ok, need password
Password:
230- Logged in
(passive mode On)
ftp>
171.16.0.0
PC-PT PC0
PC-PT PC1
2950-24 Switch0


Logical
Router2
Physical
Config
CLI
GLOBAL
Settings
Algorithm Settings
ROUTING
Static
RIP
INTERFACE
FastEthernet0/0
FastEthernet1/0
Serial2/0
Serial3/0
FastEthernet4/0
FastEthernet5/0
FastEthernet1/0
Port Status
On
Bandwidth
100 Mbps
10 Mbps
Auto
Duplex
Half Duplex
Full Duplex
Auto
MAC Address
0000.8A87.1EC1
IP Configuration
IP Address
192.168.10.2
Subnet Mask
255.255.255.0
Tx Ring Limit
10
Equivalent IOS Commands
%LINEPROTO-5-UPDOWN: Line protocol on Interface FastEthernet1/0, changed state to up
Router(config-if)#exit
Router(config)#interface FastEthernet1/0
Router(config-if)#ip address 192.168.10.2 255.255.255.0
Router(config-if)#
171.16.0.0
PC-PT PC0
PC-PT PC1
2950-24 Switch0
Realtime





Conclusion: We implemented and analyzed the performance of HTTP,HTTPS,FTP protocol using Packet Tracer Tool.

ASSIGNMENT No-15

Title: SSL protocol by capturing the packets using Wireshark tool

Objectives: capturing the packets using Wireshark tool while visiting any SSLsecured website(banking ,e-commerce etc)

Problem Statement: To study SSL protocol by capturing the packets using Wireshark tool while visiting any SSLsecured website(banking ,e-commerce etc)

Outcomes: we will get SSL protocol after tracing packet from wireshark tool.

Tools Required:

Software: Wireshark

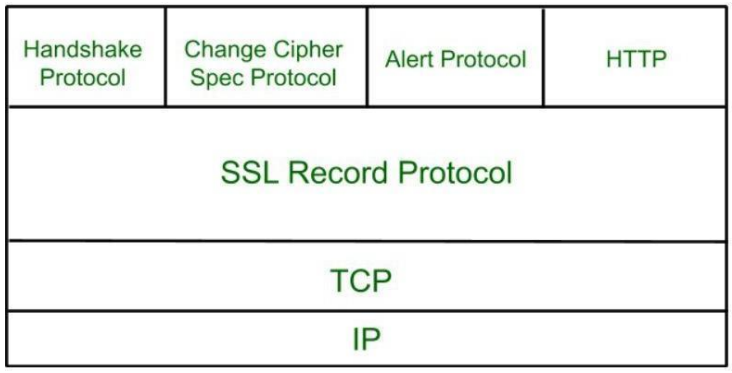
Theory:

Secure Socket Layer (SSL) provides security to the data that is transferred between web browser and server. SSL encrypts the link between a web server and a browser which ensures that all data passed between them remains private and free from attack.

Secure Socket Layer Protocols:

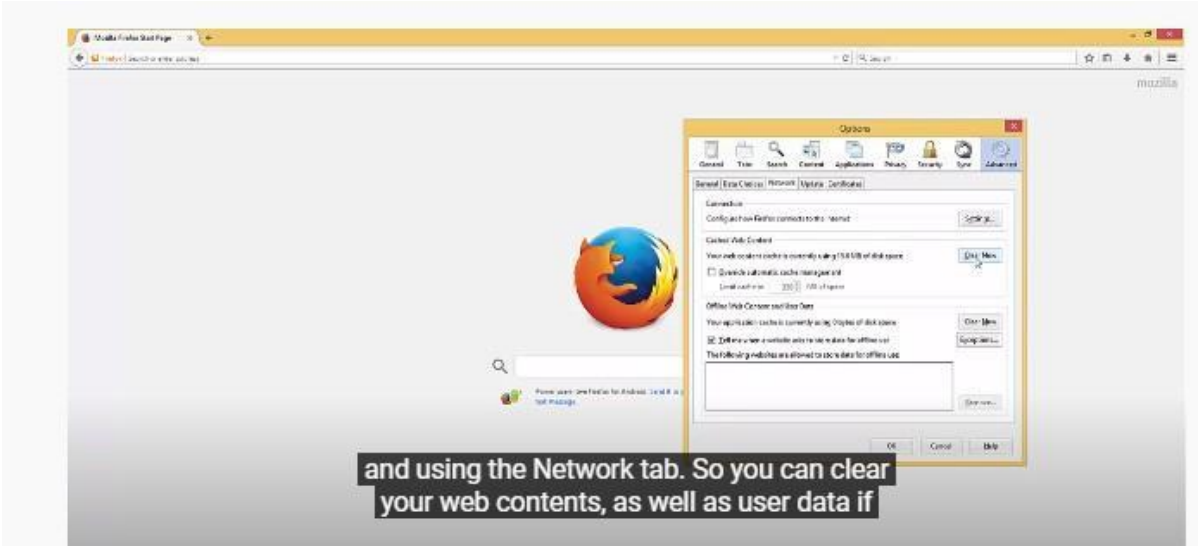
- SSL record protocol
- Handshake protocol
- Change-cipher spec protocol
- Alert protocol

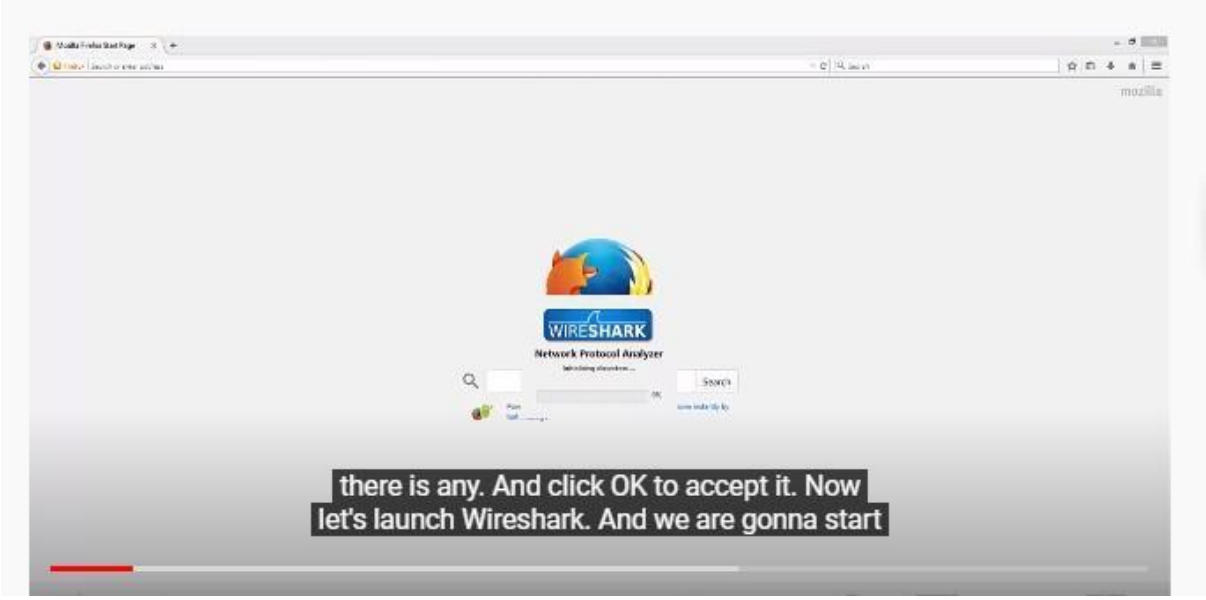
SSL Protocol Stack:

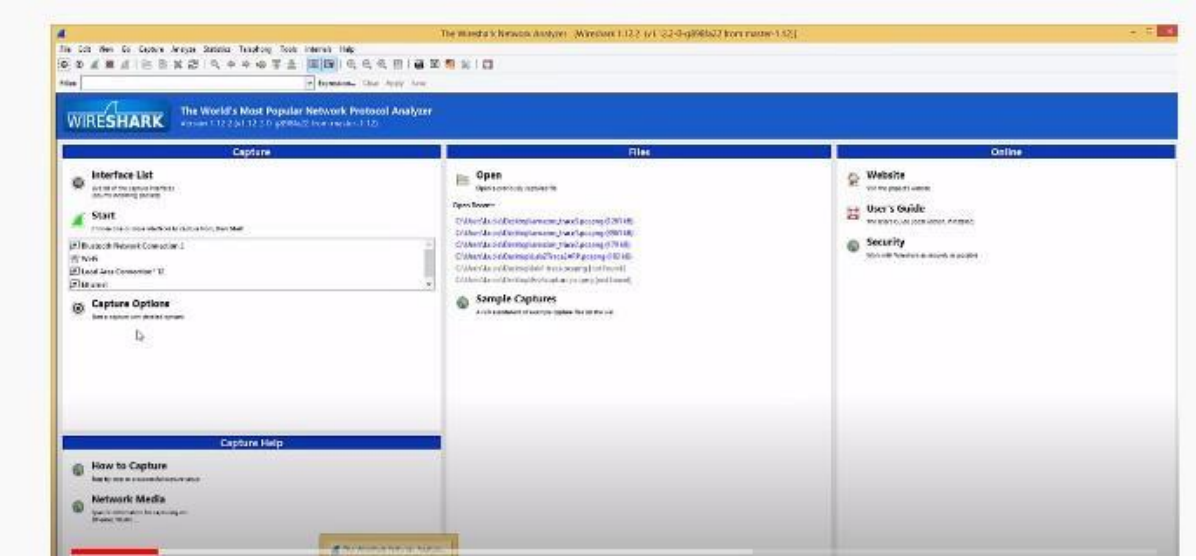


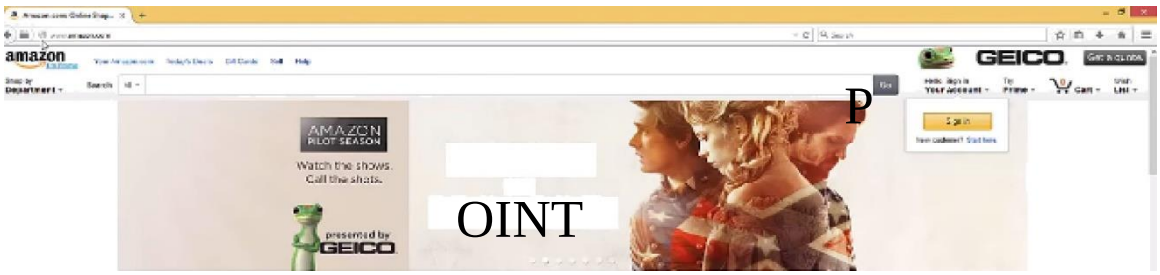
Screenshots:

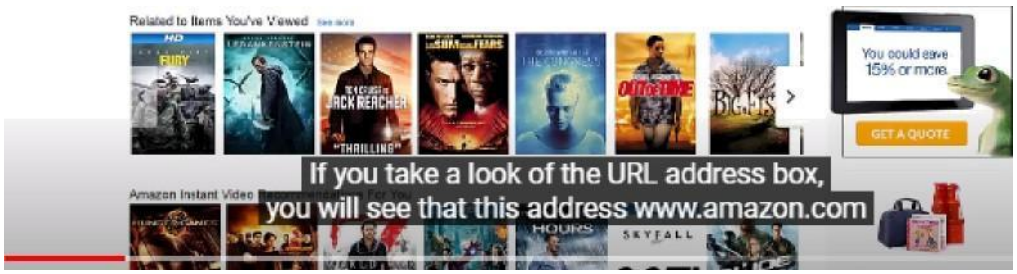

you can go to Options of your browser, and
make sure you are at the Advanced category

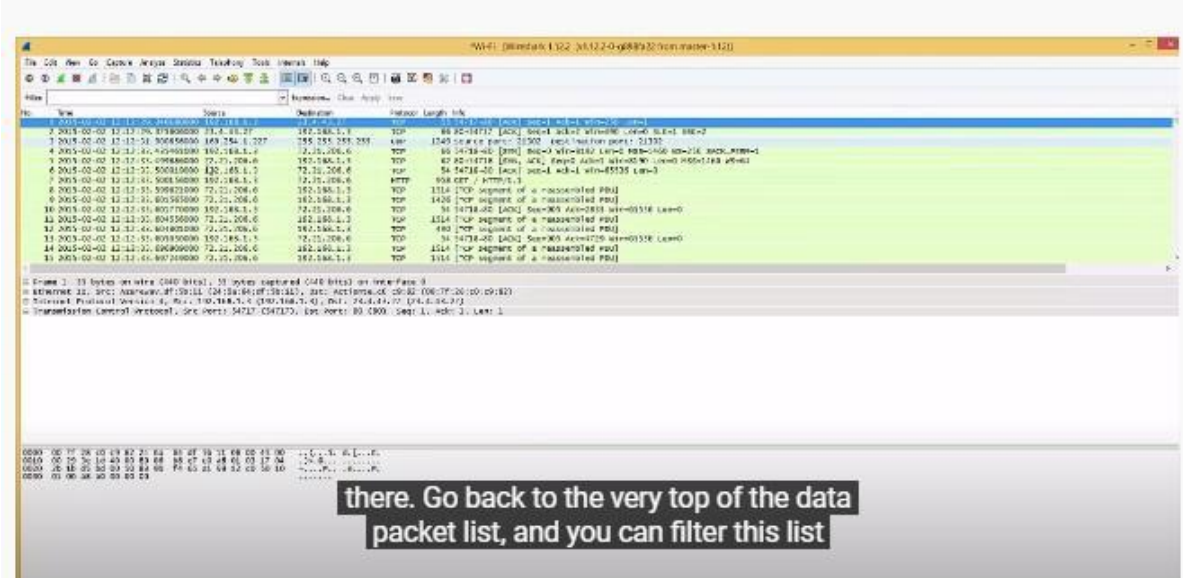

and using the Network tab. So you can clear
your web contents, as well as user data if

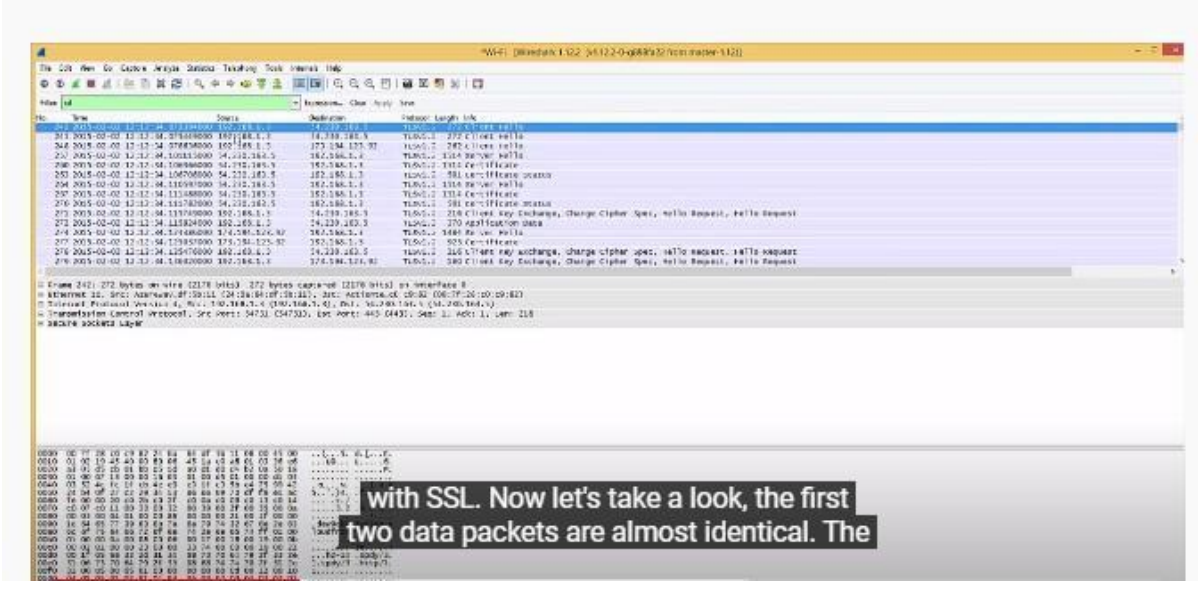

WIRESHARK
Network Protocol Analyzer
there is any. And click OK to accept it. Now
let's launch Wireshark. And we are gonna start

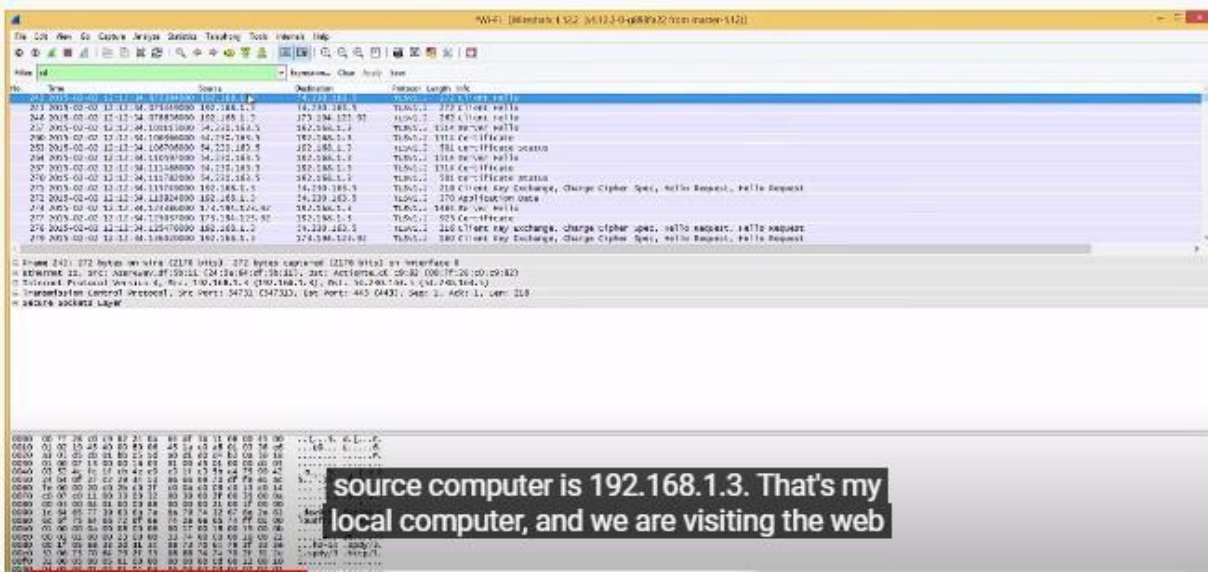

WIRESHARK
The World's Most Popular Network Protocol Analyzer
Capture
Interface List
Start
Capture Options
Files
Open
Sample Captures
Online
Website
User's Guide
Security
Capture Help
How to Capture
Network Media

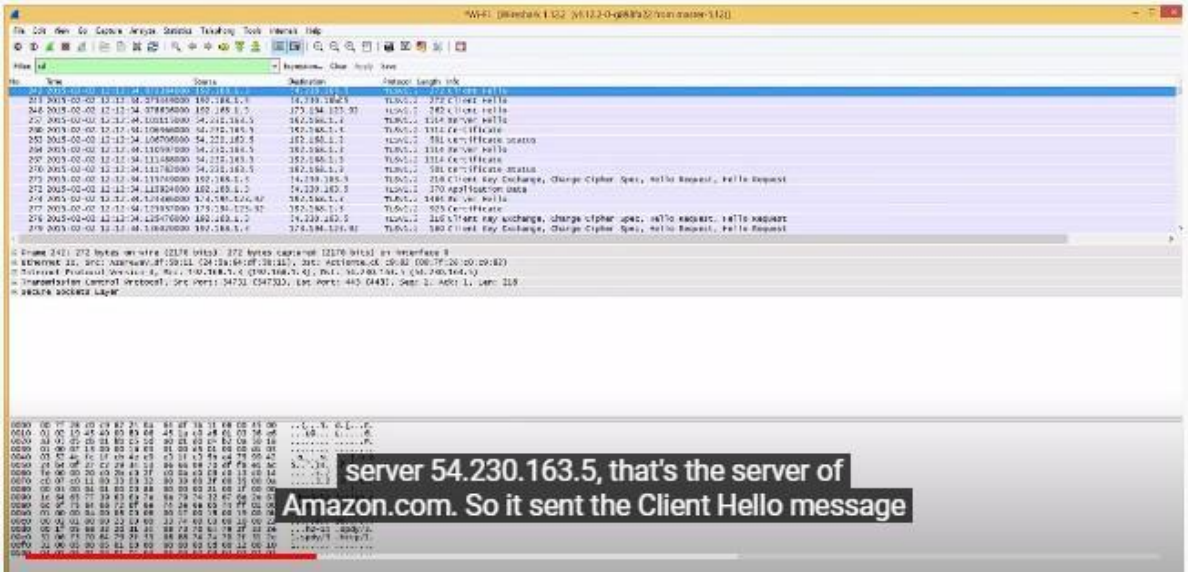

amazon
GEICO
AMAZON
PILOT SEASON
Watch the shows.
Call the shots.
presented by
GEICO
OINT

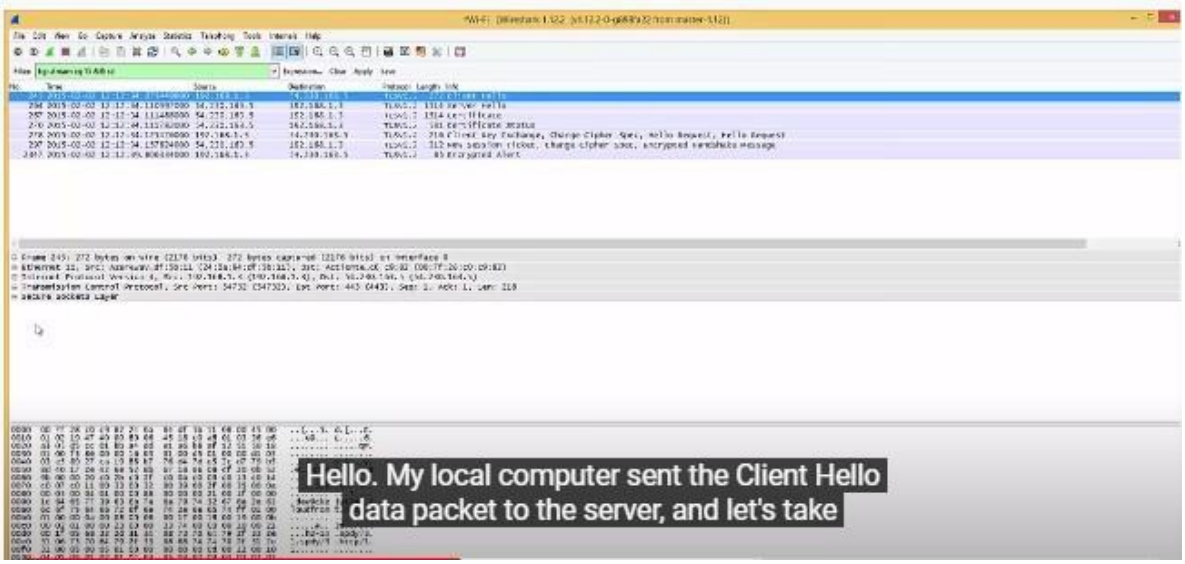

Related to Items You've Viewed
You could save 15% or more.
GET A QUOTE
Amazon Instant Video
If you take a look of the URL address box,
you will see that this address www.amazon.com

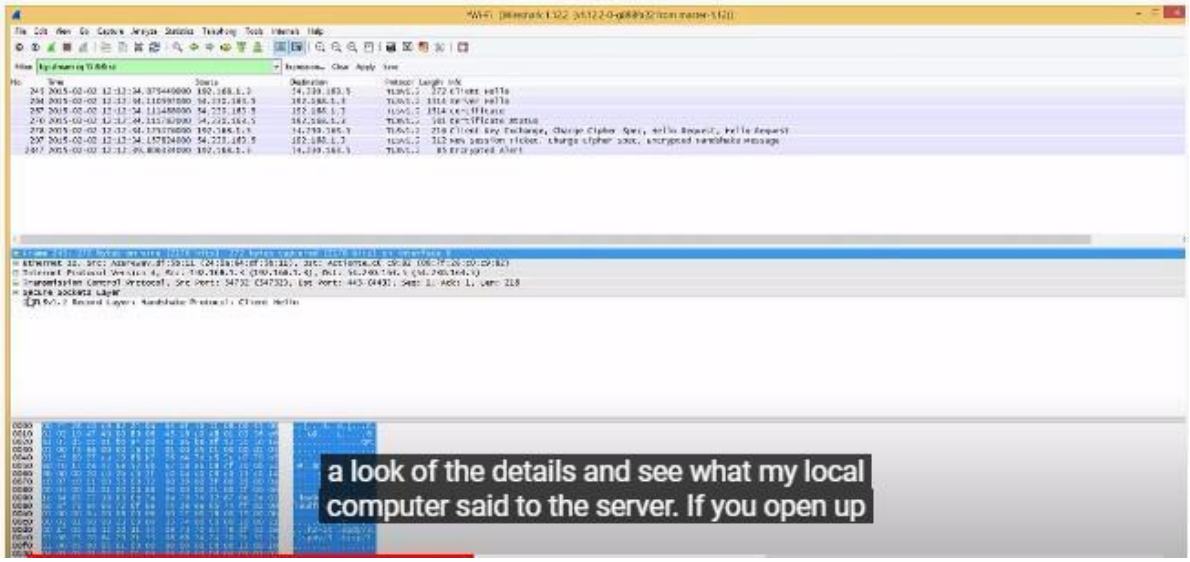

there. Go back to the very top of the data
packet list, and you can filter this list

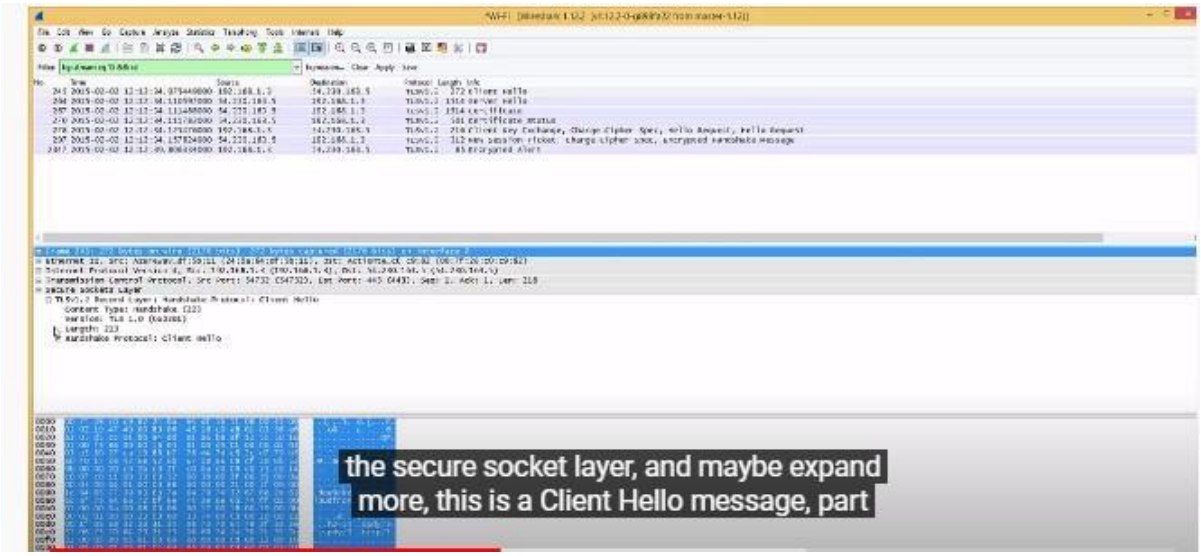

with SSL. Now let's take a look, the first
two data packets are almost identical. The

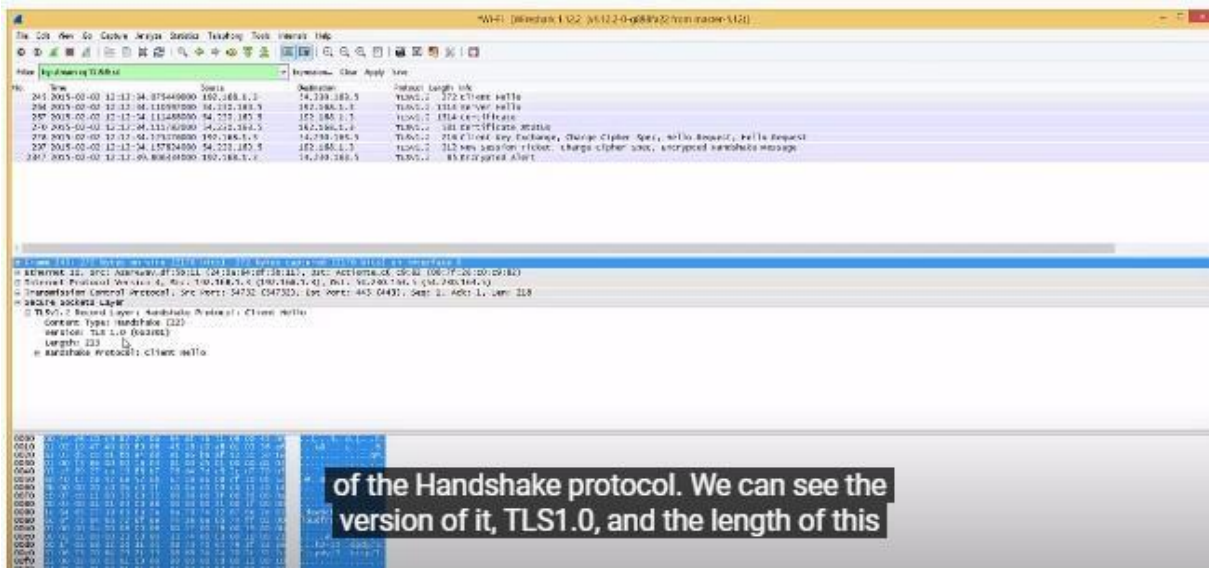

source computer is 192.168.1.3. That's my
local computer, and we are visiting the web

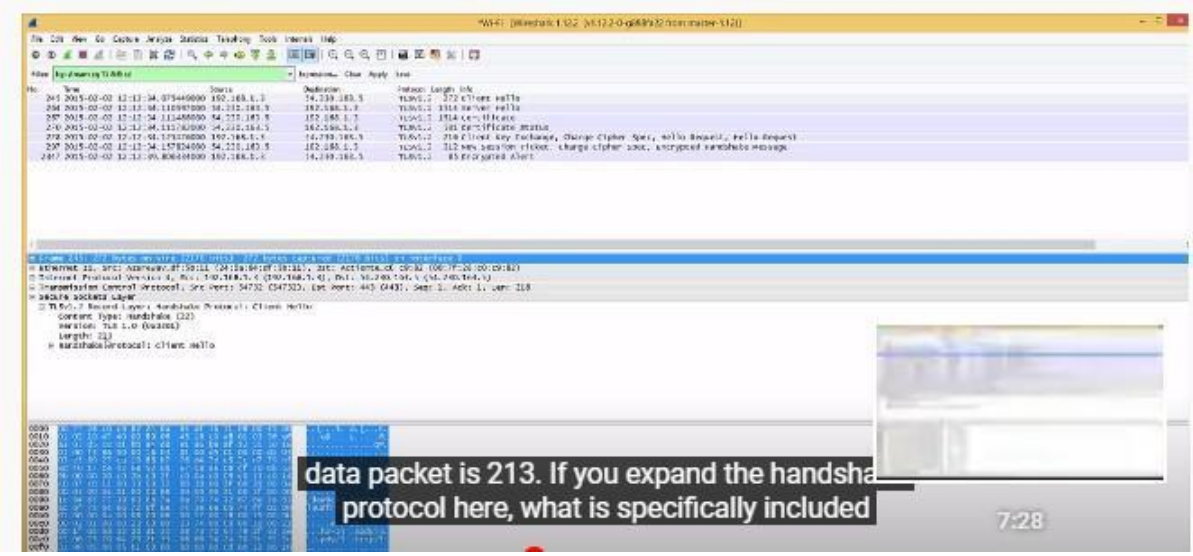

server 54.230.163.5, that's the server of
Amazon.com. So it sent the Client Hello message

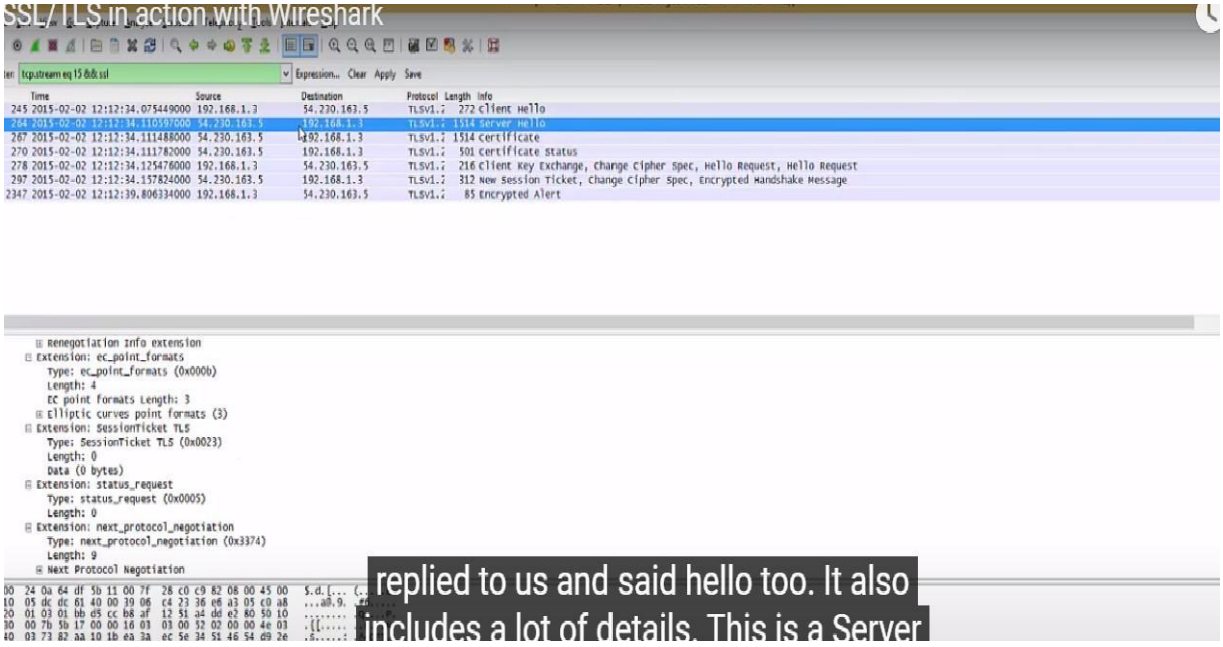

Hello. My local computer sent the Client Hello
data packet to the server, and let's take


a look of the details and see what my local
computer said to the server. If you open up


the secure socket layer, and maybe expand
more, this is a Client Hello message, part


of the Handshake protocol. We can see the
version of it, TLS1.0, and the length of this





Conclusion: We studied SSL protocol by capturing the packets using Wireshark tool while visiting any SSLsecured website(banking ,e-commerce etc)

ASSIGNMENT No. 16

Title: To illustrate the steps for implementation of S/MIME email security through Microsoft office outlook.

Objectives: To illustrate the steps for implementation of S/MIME email security through Microsoft office outlook.

Problem Statement : Illustrate the steps for implementation of S/MIME email security through Microsoft office outlook.

Outcomes: we will get S/MIME email security through outlook.

Tools Required:

Software: Microsoft office outlook

Theory :

Scenario 1: Exchange Online

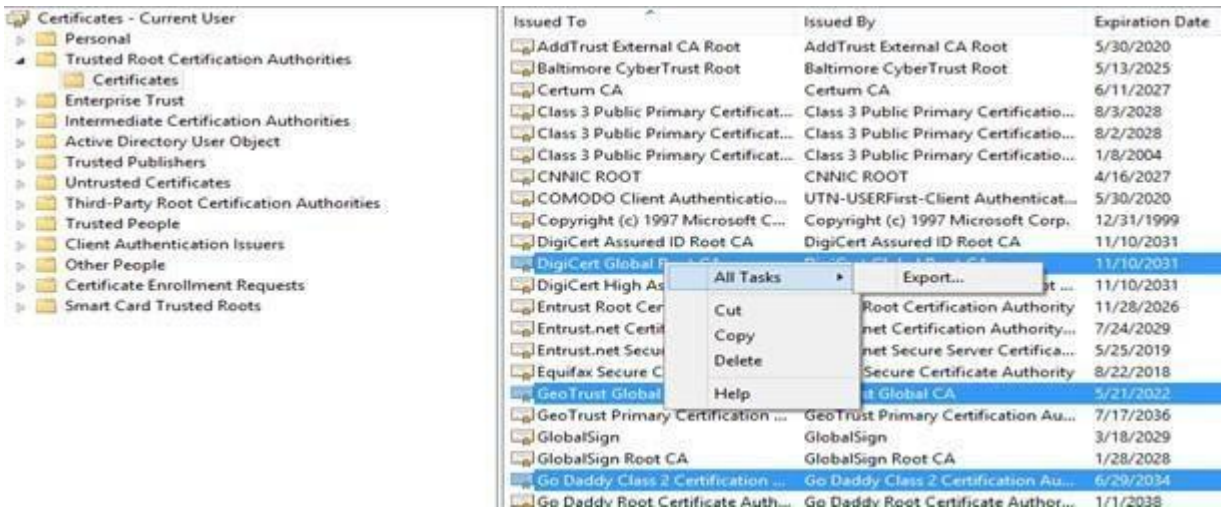
In this scenario, all the users are hosted on cloud and there is no on-premises Exchange organization.

Requirements

1. .SST File (Serialized store): The SST file contains all the root and intermediate certificates that are used when validating the S/MIME message in Office 365. The .SST file is created from certificate store explained below.
2. End user's certificate for signing and encrypting the message issued from Certificate Authorities(CA) either Windows based CA or Third party CA.

Configuration

Remember that in Exchange Online, only the SST will be used for S/MIME certificate validation. **1. Create a .SST file for the Trusted Root CA / Intermediate CA of the certificate issued to the users:** You can use either Certificate MMC or PowerShellcmdlets to export SST file. I am using Certificate console to export the .SST here: Open **certmgr.msc** snap-in, expand **Trusted Root Certificate Authorities > Certificates** > select the CA Certificates which issued the certificates to end users for S/MIME and right click > **All Tasks** > **Export…**



Select **Microsoft Serialized Certificate Store(.SST) > Click Next and save the SST file:**



0. Upload .SST to office 365 server: Update the SST on office 365 exchange server by executing the following commands using remote PowerShell.

(Example: $sst = Get-Content TenantRoot.sst -Encoding Byte) Set-SmimeConfig -SMIMECertificateIssuingCA $sst

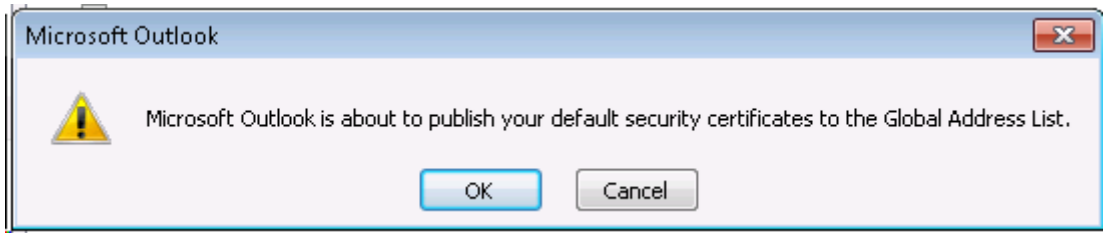
0. Publish user's certificate to the Exchange Online GAL (Global Address List) using Outlook. If not published, users will not be able to exchange S/MIME encrypted messages.

On the **File** menu in Outlook 2013, click **Options**. On the **Outlook Options** window, click **Trust Center**, click **Trust Center Settings**..., and then click **Email Security**.

- In the **Trust Center window**, click **Settings**… (Here, you need to choose certificate issued by the CA you are going to use for S/MIME).
- In the Change Security Settings window, type the Security Settings Name (you can name it anything) and choose Signing and Encryption certificate. Select the appropriate certificate assigned in previous steps, leave the Algorithm default and click OK.



- Once the information is selected, you will notice the Default Setting is populated with Security Settings Name. Now you can click the Publish to GAL button. To publish the certificate to the GAL, click OK.



0. To confirm the certificate is published in AAD (Azure Active Directory), connect to Exchange Online

using remote PowerShelland run following command. Check to make sure that the UserSMimeCertificate attribute is populated with the certificate information. If not, return to step 4.

Get-Mailbox <user> | FL or FT *user*

```
PS C:\Users\suku> Get-Mailbox UserA |ft *user*

ExchangeUserAccountControl    UserPrincipalName           UserSMimeCertificate
--------------------------    -----------------           --------------------
None                          UserA@sukuoncloud.onmicros... {48 130 9 194 6 9 42 134 7...
```

0. Once you confirm the end user has the certificate on their machine under **certificates > personal** store and also published in AAD, the users can use Outlook, OWA, or EASto send and receive S/MIME messages.

Scenario 2: Exchange Hybrid

In Exchange Hybrid topology, some mailboxes are homed on-premises and some mailboxes are homed online, and users share the same e-mail address space.

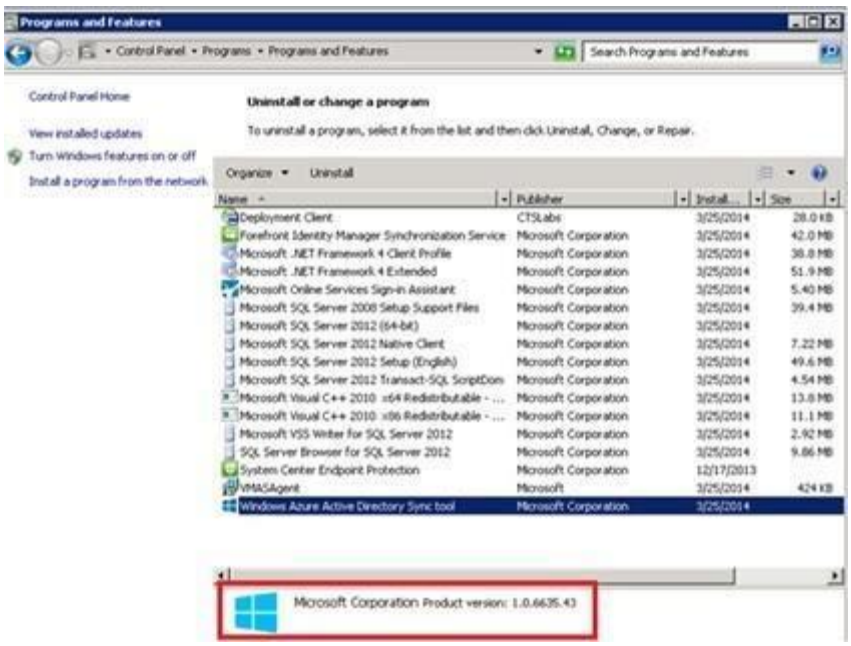
Requirements:

1. Public Key Infrastructure (PKI). You can use Active Directory Certificate Services to issue certificates to the end users.

2. SST File (Microsoft serialized certificate store). Tenant admins will have to configure their tenant in O365 with signing certificates issuing CA & Intermediate certs information. They will have to produce a SST file, which is a collection of certificates, and then later import it into O365 to validate S/MIME.
3. DirSync. You will need version 6593.0012 or higher of the DirSync tool. DirSync is used to synchronize the Active Directory user object to the Azure AD, so that cloud users can also see the certificate information of recipients when performing S/MIME (encrypt) operation.

You can verify the DirSync version following these steps:

- Open Control Panel.
- Click Programs.
- Click Programs and Features.
- Click Windows Azure Active Directory Sync tool.
- Check the version as the screenshot below:

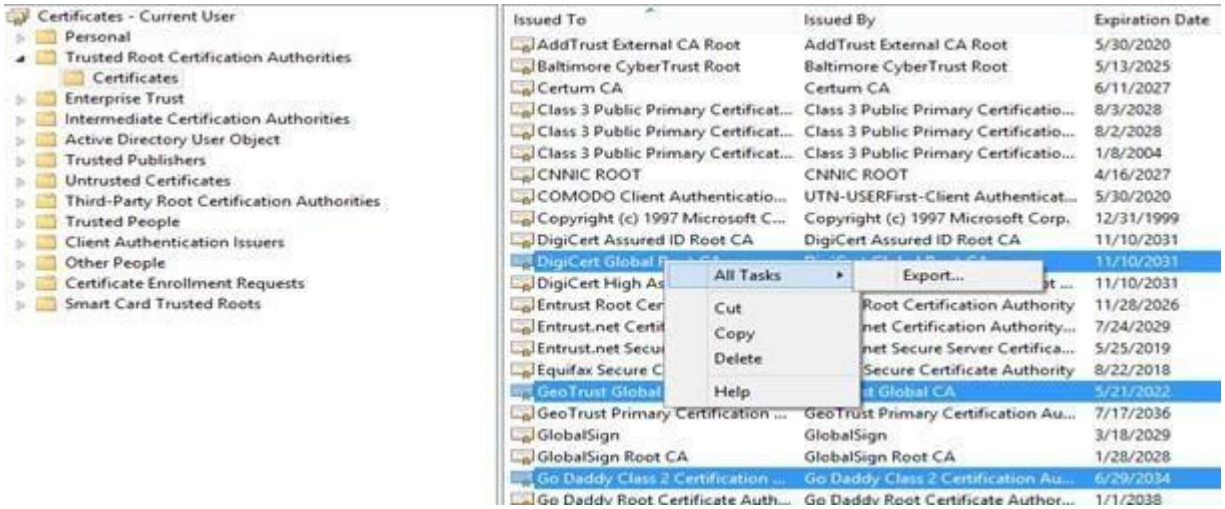


Configuration:

1. Public Key Infrastructure (PKI) The users in your organization must have certificates issued for digitally signing and encryption purposes. You can either install Certificate Authority On-premises to issue certificates to the end users or have third party certificates issued to them. There are two attributes in a user object where certificate information stored:

1) *UserCertificate* and *2) UserSMimeCertificate. UserCertificate*is populated automatically in on-premises deployment with a Windows root CA. This is populated at the time the user enrolls for a user certificate. This could be done manually for each user, or an administrator can set a GPO to automatically enroll all users. Certificates are stored in the *userSMimeCertificate* attribute when an Outlook client publishes a certificate to GAL. Outlook 2010 and above will populate both attributes with the same certificate http://support.microsoft.com/kb/2840546. But Outlook 2007 and below will not. http://support.microsoft.com/kb/822504 **2.**When setting a SST file, remember in Exchange online, only the SST will be used for S/MIME certificate validation. **Create a SST file for the Trusted Root CA / Intermediate CA of the certificate issued to the users:** You can use either Certificate MMC or PowerShellcmdlets to export the SST file. I am using the Certificate console to export the SST here: Open **certmgr.msc** snap-in, Expand **Trusted Root Certificate Authorities > Certificates** > select the CA Certificates which issued the certificates to end users for S/MIME,

and right click > **All Tasks** > **Export**…



Select SST > Click Next and save the SST file:



Upload .SST to Office 365 server: Update the SST on **Office 365 Exchange** server by running the commands below using remote PowerShell: (Example: $sst = Get-Content TenantRoot.sst -Encoding Byte) Set-SmimeConfig -

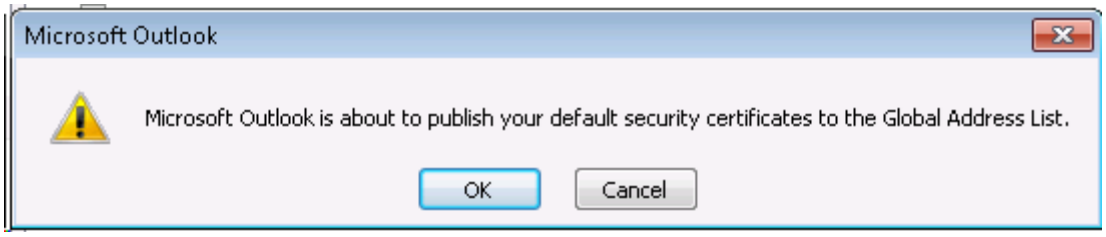
SMIMECertificateIssuingCA $sst

0. If end users are issued third party certificates, they can publish the certificate information to the GAL by following these steps:

- On the File menu in Outlook 2013, click Options.
- On the Outlook Options window, click Trust Center, click Trust Center Settings..., then Email Security.
- On Trust Center window, click Settings… (Here, you need to choose which certificate you are going to use for S/MIME).
- In the Change Security Settings window, type the Security Settings Name (you can name it anything), Choose Signing and Encryption certificate, select the appropriate certificate assigned in previous steps, leave the Algorithm default, and click OK.



- Once the information is selected, you will notice the Default Setting is populated with Security Settings Name. Now you can click the Publish to GAL button. To publish the certificate to the GAL, click OK.



- To confirm that the certificate is published in AAD (Azure Active Directory), connect to Exchange Online using remote PowerShell and run the following command. Check to see if the UserSMimeCertificate attribute is populated with the certificate information. If not, return to step 4.

Get-Mailbox <user> | FL or FT *user*

```
PS C:\Users\suku> Get-Mailbox UserA |ft *user*

ExchangeUserAccountControl    UserPrincipalName            UserSMimeCertificate
--------------------------    -----------------            --------------------
None                          UserA@sukuoncloud.onmicros... {48 130 9 194 6 9 42 134 7...
```

If Windows Certificate Authority is used, then the CA will publish the certificate information into the user object. In both cases, you need to use DirSync to replicate the on-premises Active Directory information to the cloud so that cloud users can exchange S/MIME messages.

0. After the above steps, your end users can use Outlook, OWA, or EASto send and receive S/MIME messages.

S/MIME Supported Clients

All the client machines should have the PKI issued user certificate installed under (whichever is applicable) Certificates - Current User - Personal - Certificates - Trusted Root Certification Authorities - Certificates - Intermediate Certification Authorities - Certificates If the PKI issued certificate is not available, users will not be able to send digitally signed messages or decrypt the S/MIME encrypted messages. **Outlook Web App:**

- OWA for S/MIME - Supported only on Windows Vista or greater with browser IE9 and above. Not supported on other browsers or on MOWA (Mobile for Outlook Web Access).
- Third party certificates aren't supported for OWA S/MIME; only Windows Certificate Authority issued certificates are supported.
- To use Outlook Web Access with the S/MIME control, the client system on which the user is running Internet Explorer must have Outlook Web Access with the S/MIME control installed. S/MIME functionality in Outlook Web Access

cannot be used on a system that does not have Outlook Web Access with the S/MIME control installed.

Conclusion: Illustrated the steps for implementation of S/MIME email security through Microsoft office outlook.

ASSIGNMENT No 17

Title: To study the IPsec (ESP and AH) protocol by capturing the packets using Wireshark tool.

Objectives: we have to study IPsec(ESP and AH) protocol by capturing packets.

Problem Statement : .

Outcomes: we will get ESP and AH protocol after tracing packet via wireshark.

Tools Required:

Software: Wireshark

Problem Statement : - To study the IPsec (ESP and AH) protocol by capturing the packets using Wireshark tool.

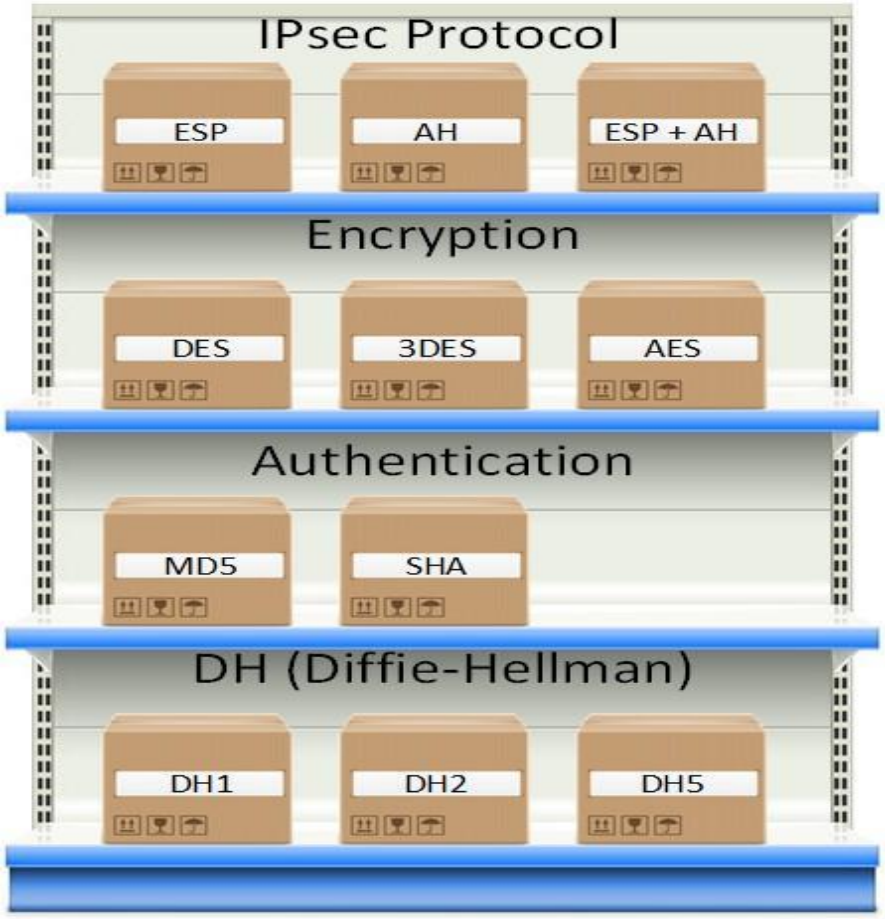
Theory : -

IPsec (Internet Protocol Security) is a framework that helps us to protect IP traffic on the network layer. Why? Because the IP protocol itself doesn't have any security features at all. IPsec can protect our traffic with the following features:

- **Confidentiality**: by encrypting our data, nobody except the sender and receiver will be able to read our data.
- **Integrity**: we want to make sure that nobody changes the data in our packets. By calculating a hash value, the sender and receiver will be able to check if changes have been made to the packet.
- **Authentication**: the sender and receiver will authenticate each other to make sure that we are really talking with the device we intend to.
- **Anti-replay**: even if a packet is encrypted and authenticated, an attacker could try to capture these packets and send them again. By using sequence numbers, IPsec will not transmit any duplicate packets.

As a framework, IPsec uses a variety of protocols to implement the features I described above. Here's an

overview.



IPsec can be used on many different devices, it's used on routers, firewalls, hosts and servers. Here are some examples how you can use it:

- Between two routers to create a site-to-site VPN that "bridges" two LANs together.
- Between a firewall and windows host for remote access VPN.
- Between two linux servers to protect an insecure protocol like telnet.

There are **two phases** to build an IPsec tunnel:

- **IKE phase 1**
- **IKE phase 2**

In IKE phase 1, two peers will negotiate about the encryption, authentication, hashing and other protocols that they want to use and some other parameters that are required. In this phase, an **ISAKMP (Internet Security Association and Key Management Protocol)** session is established. This is also called the

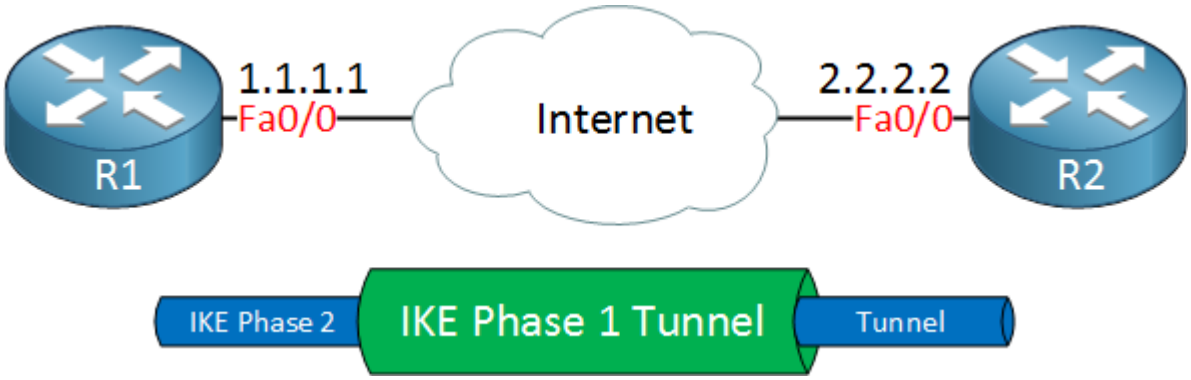
ISAKMP tunnel or **IKE phase 1** tunnel.

The collection of parameters that the two devices will use is called a **SA (Security Association)**. Here's an example of two routers that have established the IKE phase 1 tunnel:

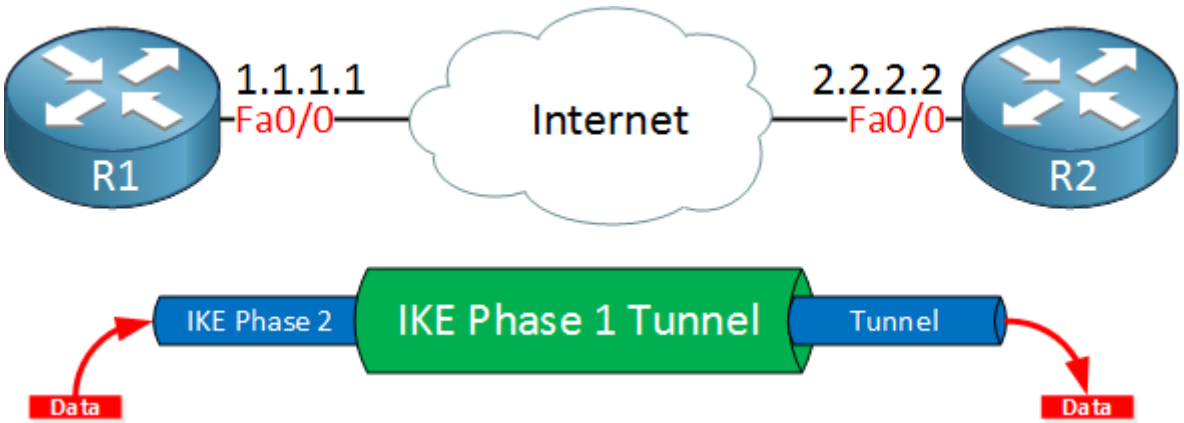


The IKE phase 1 tunnel is only used for **management traffic**. We use this tunnel as a *secure method to establish the second tunnel* called the **IKE phase 2 tunnel** or **IPsec tunnel** and for management traffic like keepalives.

Here's a picture of our two routers that completed IKE phase 2:



Once IKE phase 2 is completed, we have an IKE phase 2 tunnel (or IPsec tunnel) that we can use to protect our user data. This user data will be sent through the IKE phase 2 tunnel:



IKE builds the tunnels for us but it doesn't authenticate or encrypt user data. We use two other protocols for this:

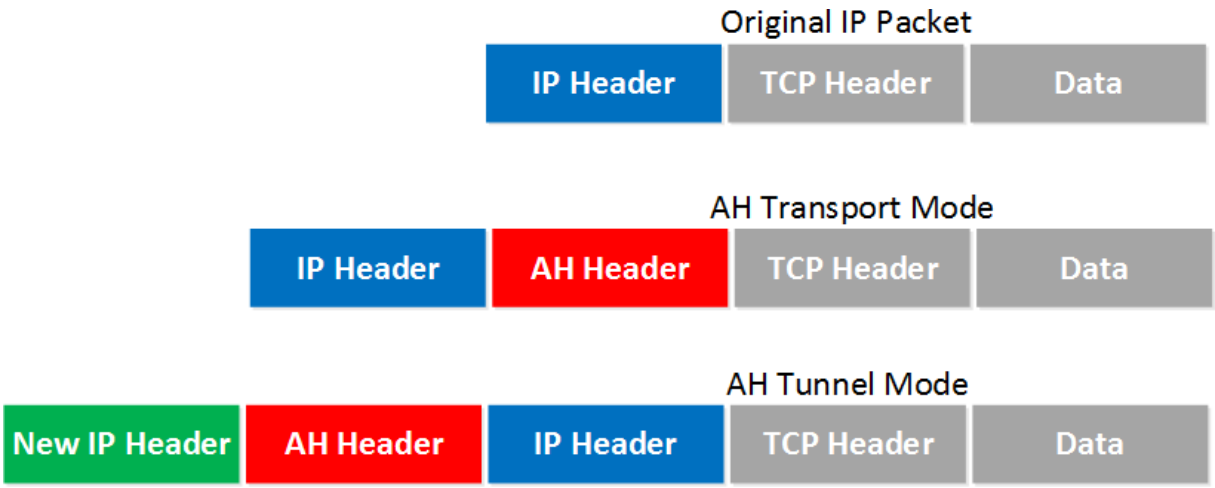
- **AH (Authentication Header)**
- **ESP (Encapsulating Security Payload)**

AH and ESP both offer authentication and integrity but only **ESP supports encryption**. Because of this, ESP is the most popular choice nowadays.

Both protocols support two different modes:

- **Transport mode**
- **Tunnel mode**

The main difference between the two is that with transport mode we will use the **original IP header** while in tunnel mode, we use a new **IP header**. Here's an example to help you visualize this:

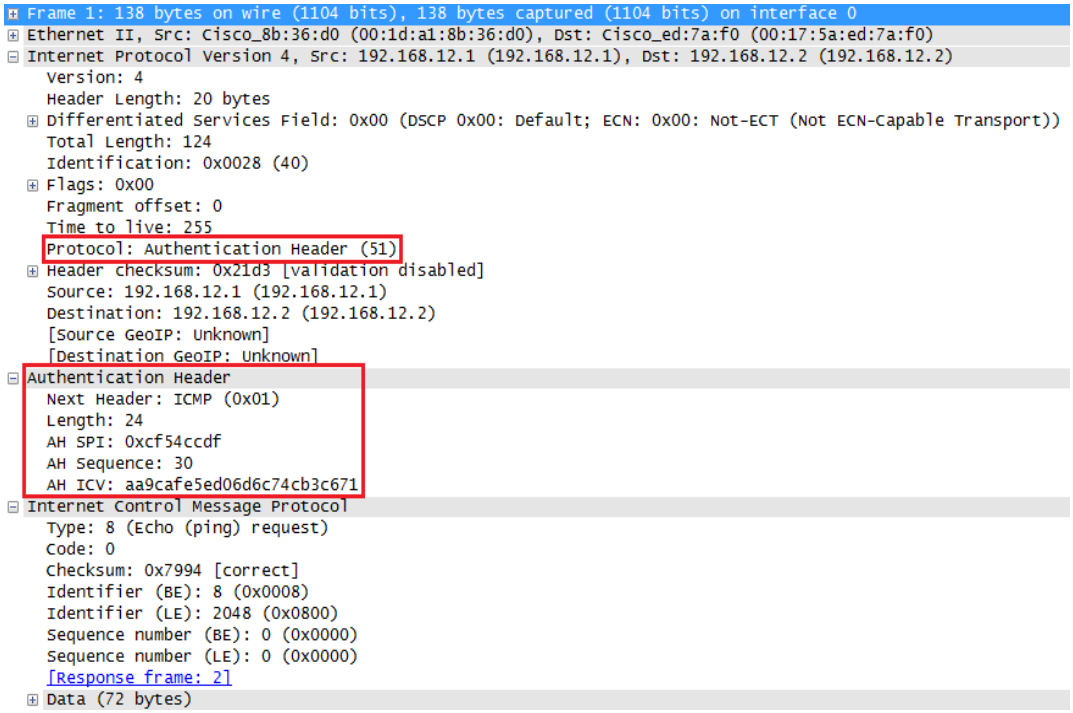


Transport mode is often between two devices that want to protect some insecure traffic (example: telnet traffic). Tunnel mode is typically used for site-to-site VPNs where we need to encapsulate the original IP

packet since these are mostly private IP addresses and can't be routed on the Internet. I will explain these two modes in detail later in this lesson.

The entire process of IPsec consists of five steps:

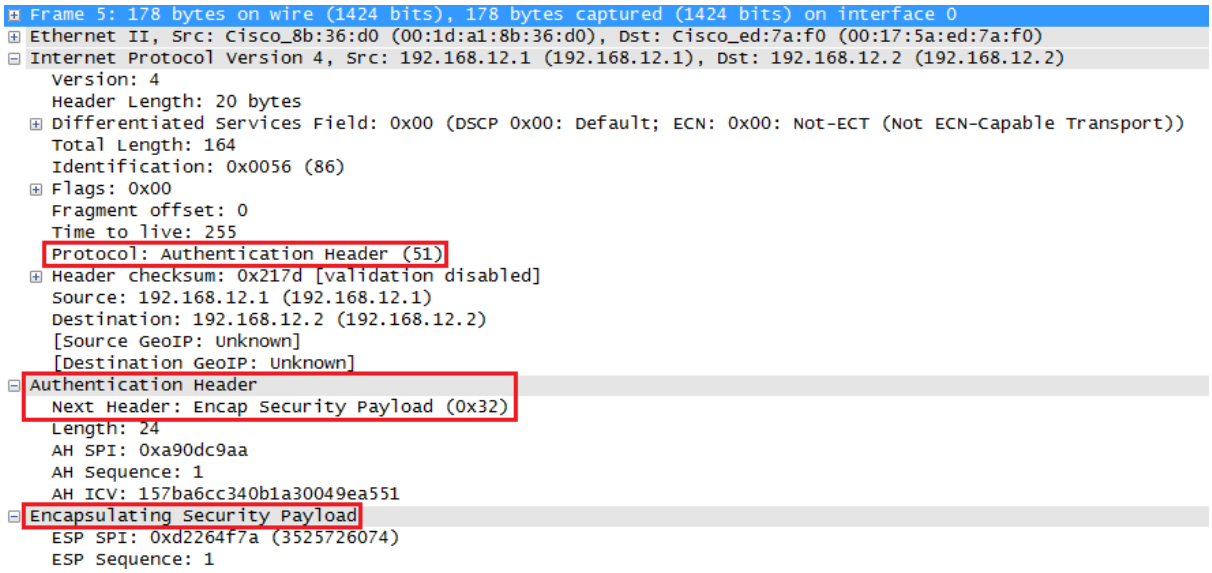
- **Initiation**: something has to trigger the creation of our tunnels. For example when you configure IPsec on a router, you use an access-list to tell the router what data to protect. When the router receives something that matches the access-list, it will start the IKE process. It's also possible to manually initiate the tunnel.
- **IKE phase 1**: we negotiate a security association to build the IKE phase 1 tunnel (ISAKMP tunnel).
- **IKE phase 2**: within the IKE phase 1 tunnel, we build the IKE phase 2 tunnel (IPsec tunnel).
- **Data transfer**: we protect user data by sending it through the IKE phase 2 tunnel.
- **Termination**: when there is no user data to protect then the IPsec tunnel will be terminated after awhile.
- Authentication Header Protocol



AH and ESP

With transport mode we will use the original IP header, followed by an AH and ESP header. The transport layer, payload and ESP trailer will be encrypted.

Because we also use AH, the entire IP packet is authenticated. Here's what it looks like in wireshark:



Conclusion : Hence we have studied ESP and AH Protocol via wireshark.